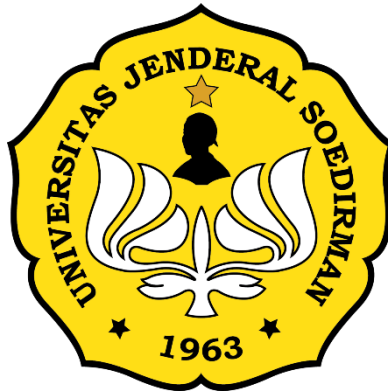


LAPORAN PROJECT ALGORITMA DAN PEMROGRAMAN

Perbandingan Implementasi Algoritma dan Struktur Logika Program Python dan R pada Enam Studi Kasus Komputasional



Dosen Pengampu:

Lutfiah Maharani Siniwi, S.Stat., M.Stat.

Disusun Oleh Kelompok E:

Winny Anindita Natasya	(K1D025045)
Almass Rahma Shaliha	(K1D025054)
Yudalaga Satriaji	(K1D025057)
Arrohmansyah Al Hakim	(K1D025073)

**UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI STATISTIKA KELAS B
PURWOKERTO
2026**

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Project.....	1
1.3 Ruang Lingkup Project.....	2
BAB II ANALISIS MASALAH.....	4
2.1 Program 1: Analisis Teks.....	4
2.1.1 Deskripsi Soal.....	4
2.1.2 Tabel Input-Proses-Output.....	4
2.1.3 Batasan Masalah dan Kondisi Khusus.....	4
2.2 Program 2: String Kalimat.....	4
2.2.1 Deskripsi Soal.....	4
2.2.2 Tabel Input-Proses-Output.....	5
2.2.3 Batasan Masalah dan Kondisi Khusus.....	5
2.3 Program 3: Akar Persamaan Kuadrat.....	5
2.3.1 Deskripsi Soal.....	5
2.3.2 Tabel Input-Proses-Output.....	5
2.3.3 Batasan Masalah dan Kondisi Khusus.....	6
2.4 Program 4: Pengolahan NIP ASN.....	6
2.4.1 Deskripsi Soal.....	6
2.4.2 Tabel Input-Proses-Output.....	6
2.4.3 Batasan Masalah dan Kondisi Khusus.....	7
2.5 Program 5: Klasifikasi Cluster 3 Dimensi.....	7
2.5.1 Deskripsi Soal.....	7
2.5.2 Tabel Input-Proses-Output.....	7
2.5.3 Batasan Masalah dan Kondisi Khusus.....	8
2.6 Program 6: Interval Konfidensi Proporsi.....	8
2.6.1 Deskripsi Soal.....	8
2.6.2 Tabel Input-Proses-Output.....	8
2.6.3 Batasan Masalah dan Kondisi Khusus.....	9
2.7 Pembagian Tugas Kelompok.....	9
BAB III PSEUDOCODE DAN FLOWCHART.....	10

3.1 Program 1: Analisis Teks	10
3.1.1 Pseudocode	10
3.1.2 Flowchart	11
3.2 Program 2: String Kalimat	12
3.2.1 Pseudocode	12
3.2.2 Flowchart	13
3.3 Program 3: Akar Persamaan Kuadrat.....	14
3.3.1 Pseudocode	14
3.3.2 Flowchart	15
3.4 Program 4: Pengolahan NIP ASN	16
3.4.1 Pseudocode	16
3.4.2 Flowchart	19
3.5 Program 5: Klasifikasi Cluster 3 Dimensi	20
3.5.1 Pseudocode	20
3.5.2 Flowchart	22
3.6 Program 6: Interval Konfidensi Proporsi	23
3.6.1 Pseudocode	23
3.6.2 Flowchart	25
BAB IV IMPLEMENTASI PROGRAM.....	26
4.1 Program 1: Analisis Teks	26
4.1.1 Source Code R	26
4.1.2 Source Code Python.....	27
4.2 Program 2: String Kalimat	28
4.2.1 Source Code R	28
4.2.2 Source Code Python.....	29
4.3 Program 3: Akar Persamaan Kuadrat.....	30
4.3.1 Source Code R	30
4.3.2 Source Code Python.....	33
4.4 Program 4: Pengolahan NIP ASN	35
4.4.1 Source Code R	35
4.4.2 Source Code Python.....	38
4.5 Program 5: Klasifikasi Cluster 3 Dimensi	41
4.5.1 Source Code R	41
4.5.2 Source Code Python.....	44
4.6 Program 6: Interval Konfidensi Proporsi	47

4.6.1 Source Code R	47
4.6.2 Source Code Python.....	49
BAB V PENJELASAN LINE-BY-LINE CODE	51
5.1 Program 1: Analisis Teks	51
5.1.1 Kode R	51
5.1.2 Kode Python.....	51
5.2 Program 2: String Kalimat	52
5.2.1 Kode R	52
5.2.2 Kode Python.....	53
5.3 Program 3: Akar Persamaan Kuadrat.....	54
5.3.1 Kode R	54
5.3.2 Kode Python.....	57
5.4 Program 4: Pengolahan NIP ASN	59
5.4.1 Kode R	59
5.4.2 Kode Python.....	62
5.5 Program 5: Klasifikasi Cluster 3 Dimensi	64
5.5.1 Kode R	64
5.5.2 Kode Python.....	68
5.6 Program 6: Interval Konfidensi Proporsi	70
5.6.1 Kode R	70
5.6.2 Kode Python.....	72
BAB VI PENGUJIAN PROGRAM	74
6.1 Program 1: Analisis Teks	74
6.1.1 Pengujian Program.....	74
6.1.2 Screenshot Hasil Running.....	74
6.1.3 Interpretasi Output	77
6.2 Program 2: String Kalimat	78
6.2.1 Tabel Pengujian.....	78
6.2.2 <i>Screenshot Hasil Running</i>	80
6.2.3 Interpretasi Output	82
6.3 Program 3: Akar Persamaan Kuadrat.....	83
6.3.1 Tabel Pengujian.....	83
6.3.2 <i>Screenshot Hasil Running</i>	83
6.3.3 Interpretasi Output	85
6.4 Program 4: Pengolahan NIP ASN	86

6.4.1 Tabel Pengujian.....	86
6.4.2 <i>Screenshot</i> Hasil <i>Running</i>	87
6.4.3 Interpretasi Output	90
6.5 Program 5: Klasifikasi Cluster 3 Dimensi	92
6.5.1 Tabel Pengujian.....	92
6.5.2 <i>Screenshot</i> Hasil <i>Running</i>	93
6.5.3 Interpretasi Output	96
6.6 Program 6: Interval Konfidensi Proporsi	97
6.6.1 Tabel Pengujian.....	97
6.6.2 <i>Screenshot</i> Hasil <i>Running</i>	98
6.6.3 Interpretasi Output	100
BAB VII PENUTUP.....	102
7.1 Kesimpulan	102
7.2 Kendala yang Dihadapi.....	102
7.3 Saran Pengembangan Program	103
LAMPIRAN.....	105

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di era perkembangan teknologi saat ini, kemampuan pemrograman menjadi salah satu keterampilan yang penting untuk dimiliki oleh mahasiswa, khususnya mahasiswa program studi Statistika. Pemrograman tidak hanya digunakan untuk membuat aplikasi, tetapi juga berperan penting dalam membantu menyelesaikan berbagai permasalahan secara sistematis dan efisien, termasuk permasalahan yang berkaitan dengan pengolahan data dan analisis statistik.

Oleh karena itu, project Algoritma dan Pemrograman ini dibuat untuk melatih mahasiswa dalam menganalisis masalah, menyusun algoritma, serta mengimplementasikan solusi melalui enam studi kasus menggunakan dua bahasa pemrograman, yaitu Python dan R. Kedua bahasa ini dipilih karena masing-masing memiliki keunggulan tersendiri dan banyak digunakan dalam dunia kerja maupun dunia akademik, terutama di bidang ilmu data dan statistika.

Melalui project ini, mahasiswa dapat menerapkan konsep-konsep dasar pemrograman yang telah dipelajari selama perkuliahan, mulai dari analisis masalah, penyusunan pseudocode dan flowchart, hingga implementasi dan pengujian program. Selain itu, project ini juga bertujuan untuk meningkatkan kemampuan berpikir logis, pemecahan masalah, serta keterampilan bekerja sama dalam tim, mengingat project ini dikerjakan secara berkelompok dengan memanfaatkan repository GitHub sebagai sarana kolaborasi.

1.2 Tujuan Project

Berdasarkan latar belakang tersebut, adapun tujuan dari pengerjaan project ini adalah sebagai berikut:

1. Menganalisis permasalahan komputasional dengan menentukan input, proses, output, serta batasan masalah pada setiap studi kasus yang diberikan.
2. Menyusun algoritma penyelesaian masalah dalam bentuk narasi, pseudocode, dan flowchart secara runtut dan logis.

3. Mengimplementasikan algoritma yang telah disusun ke dalam dua bahasa pemrograman, yaitu Python dan R.
4. Membandingkan struktur logika program antara Python dan R pada studi kasus yang sama, termasuk perbedaan sintaks dan pendekatan masing-masing bahasa.
5. Menerapkan konsep variabel, tipe data, operator, string, percabangan, fungsi, serta struktur data seperti tuple, list, dan vector dalam penyelesaian masalah.
6. Melakukan validasi input dan menangani kondisi-kondisi khusus yang mungkin terjadi pada program.
7. Menguji program menggunakan beberapa skenario data dan mendokumentasikan hasil pengujian secara sistematis.
8. Melatih kemampuan kerja sama tim melalui penggunaan repository GitHub dengan pembagian kontribusi yang jelas antaranggota kelompok.

1.3 Ruang Lingkup Project

Project ini terdiri dari enam program berbasis studi kasus yang wajib diselesaikan oleh kelompok, yaitu:

1. Analisis teks, yaitu program untuk menghitung jumlah kata dan jumlah kalimat dari sebuah teks.
2. Rekayasa objek string K1 sampai K4, yaitu program untuk membuat dan menampilkan empat objek string dengan format karakter khusus sesuai kalimat yang telah ditentukan.
3. Akar persamaan kuadrat, yaitu program untuk menentukan akar-akar persamaan kuadrat, baik bertipe real maupun imajiner.
4. Pengolahan NIP ASN, yaitu program untuk menentukan tanggal lahir seorang ASN berdasarkan delapan digit pertama NIP yang dimasukkan.
5. Klasifikasi cluster 3 dimensi, yaitu program untuk menentukan cluster suatu titik pada ruang tiga dimensi berdasarkan jarak terdekat terhadap tiga pusat cluster yang telah ditentukan.
6. Interval konfidensi proporsi, yaitu program untuk menghitung interval konfidensi proporsi populasi berdasarkan proporsi sampel dan tingkat kepercayaan yang dimasukkan pengguna.

Setiap program disusun mulai dari analisis input-proses-output, penyusunan pseudocode dan flowchart, implementasi kode dalam Python dan R, penjelasan logika tiap

baris kode, hingga pengujian program dengan beberapa skenario data. Seluruh source code dan dokumentasi project disimpan dan dikelola melalui repository GitHub kelompok.

BAB II

ANALISIS MASALAH

2.1 Program 1: Analisis Teks

2.1.1 Deskripsi Soal

Program 1 merupakan program untuk menghitung jumlah kata dan jumlah kalimat dari sebuah teks yang dimasukkan oleh pengguna. Program ini memanfaatkan karakteristik tanda titik sebagai penanda akhir kalimat dan spasi sebagai pemisah antarkata.

2.1.2 Tabel Input-Proses-Output

Komponen	Keterangan
Input	Teks berupa kalimat atau paragraf yang dimasukkan pengguna
Proses	Menghitung jumlah kalimat dengan menghitung banyaknya tanda titik pada teks; menghapus tanda titik dari teks; memisahkan teks berdasarkan spasi untuk menghitung jumlah kata
Output	Jumlah kata dan jumlah kalimat dari teks yang dimasukkan

2.1.3 Batasan Masalah dan Kondisi Khusus

1. Teks yang dimasukkan diasumsikan tidak memuat tanda titik selain untuk mengakhiri kalimat.
2. Setiap kalimat diakhiri tepat dengan satu tanda titik.
3. Pemisah antarkata diasumsikan adalah spasi tunggal.

2.2 Program 2: String Kalimat

2.2.1 Deskripsi Soal

Program 2 merupakan program untuk membuat empat objek string K1 sampai K4 dalam Python dan R sedemikian rupa sehingga ketika dicetak akan menampilkan kalimat-kalimat yang telah ditentukan secara tepat. Setiap kalimat memiliki

karakteristik karakter khusus yang berbeda, seperti tanda kutip tunggal, tanda kutip ganda, tanda seru, dan kombinasi tanda baca lainnya.

2.2.2 Tabel Input-Proses-Output

Komponen	Keterangan
Input	Tidak ada input dari pengguna; nilai string sudah ditetapkan langsung di dalam kode
Proses	Mendefinisikan objek string K1 sampai K4 menggunakan kombinasi tanda kutip dan karakter escape yang sesuai
Output	Tampilan keempat kalimat sesuai format yang telah ditentukan

2.2.3 Batasan Masalah dan Kondisi Khusus

1. Setiap objek string harus dicetak persis sesuai kalimat yang ditentukan, termasuk tanda baca dan spasi.
2. Program harus menangani karakter khusus seperti tanda kutip tunggal dan ganda menggunakan mekanisme escape character atau pemilihan jenis tanda kutip yang tepat.

2.3 Program 3: Akar Persamaan Kuadrat

2.3.1 Deskripsi Soal

Program 3 merupakan program untuk menentukan akar-akar dari suatu persamaan kuadrat $ax^2 + bx + c = 0$. Program menerima koefisien a, b, dan c dari pengguna, kemudian menghitung nilai diskriminan untuk menentukan apakah akar-akarnya bertipe bilangan real atau imajiner. Akar real ditampilkan dalam tiga angka desimal.

2.3.2 Tabel Input-Proses-Output

Komponen	Keterangan
Input	Koefisien a, b, dan c dari persamaan kuadrat yang dimasukkan pengguna

Proses	Memvalidasi bahwa nilai a tidak sama dengan nol; menghitung diskriminan ($b^2 - 4ac$); menentukan jenis akar berdasarkan nilai diskriminan; menghitung akar menggunakan rumus kuadrat bila diskriminan lebih dari atau sama dengan nol
Output	Dua akar real berbeda, satu akar real kembar, atau pesan bahwa persamaan hanya memiliki akar imajiner

2.3.3 Batasan Masalah dan Kondisi Khusus

1. Nilai a tidak boleh sama dengan nol karena persamaan tidak akan berbentuk kuadrat. Program menampilkan pesan kesalahan apabila a bernilai nol.
2. Apabila diskriminan bernilai lebih dari nol, program menampilkan dua akar real yang berbeda.
3. Apabila diskriminan bernilai sama dengan nol, program menampilkan satu akar real kembar.
4. Apabila diskriminan bernilai kurang dari nol, program menampilkan pesan bahwa persamaan hanya memiliki akar imajiner.
5. Akar real ditampilkan dalam format tiga angka desimal.
6. Apabila pengguna memasukkan nilai selain angka, program menampilkan pesan input tidak valid.

2.4 Program 4: Pengolahan NIP ASN

2.4.1 Deskripsi Soal

Program 4 merupakan program untuk menampilkan tanggal lahir seorang ASN berdasarkan NIP yang telah ditetapkan di dalam kode. Delapan digit pertama NIP menyatakan tahun lahir (empat digit), bulan lahir (dua digit), dan tanggal lahir (dua digit). Program menggunakan percabangan untuk mengonversi angka bulan menjadi nama bulan dalam bahasa Indonesia, serta memvalidasi kelogisan tanggal yang diekstrak.

2.4.2 Tabel Input-Proses-Output

Komponen	Keterangan
----------	------------

Input	NIP ASN berupa string angka yang sudah ditetapkan langsung di dalam kode sebagai konstanta
Proses	Memvalidasi bahwa NIP terdiri dari minimal delapan digit angka; mengekstrak empat digit pertama sebagai tahun lahir, digit ke-5 dan ke-6 sebagai bulan lahir, dan digit ke-7 dan ke-8 sebagai tanggal lahir; memvalidasi kelogisan kombinasi tanggal dan bulan terhadap kalender nyata; memvalidasi bahwa tahun lahir tidak melebihi tahun berjalan; mengonversi angka bulan menjadi nama bulan menggunakan percabangan
Output	Tanggal lahir ASN dalam format tanggal, nama bulan, dan tahun, atau pesan kesalahan apabila data tidak valid

2.4.3 Batasan Masalah dan Kondisi Khusus

1. NIP ditulis sebagai konstanta di dalam kode, bukan berasal dari input pengguna.
2. NIP harus berupa angka dan memiliki panjang minimal delapan digit.
3. Kombinasi tanggal dan bulan yang diekstrak harus valid secara kalender, misalnya tanggal 30 Februari dianggap tidak logis.
4. Tahun lahir tidak boleh melebihi tahun berjalan saat program dijalankan.
5. Program menggunakan struktur percabangan untuk memilih nama bulan yang sesuai.

2.5 Program 5: Klasifikasi Cluster 3 Dimensi

2.5.1 Deskripsi Soal

Program 5 merupakan program untuk mengklasifikasikan suatu titik $U(x_1, x_2, x_3)$ ke dalam salah satu dari tiga cluster berdasarkan jarak terdekatnya terhadap pusat masing-masing cluster, yaitu $A(2,1,3)$, $B(1,-4,6)$, dan $C(-2,3,-2)$. Jarak dihitung menggunakan rumus jarak Euclidean pada ruang tiga dimensi. Koordinat titik U ditetapkan langsung di dalam kode. Program ini wajib menggunakan fungsi untuk memenuhi ketentuan modularisasi.

2.5.2 Tabel Input–Proses–Output

Komponen	Keterangan
----------	------------

Input	Koordinat titik U berupa tiga nilai numerik (x, y, z) yang ditetapkan langsung di dalam kode sebagai konstanta, kemudian disimpan sebagai tuple
Proses	Menghitung jarak Euclidean dari titik U ke pusat cluster A, B, dan C menggunakan fungsi; membandingkan ketiga jarak; menentukan cluster dengan jarak terkecil
Output	Jarak titik U ke masing-masing pusat cluster dan nama cluster tempat titik U diklasifikasikan

2.5.3 Batasan Masalah dan Kondisi Khusus

1. Koordinat titik U ditetapkan sebagai konstanta di dalam kode dan disimpan sebagai tuple dengan tepat tiga elemen bertipe numerik.
2. Pusat cluster A, B, dan C ditetapkan sebagai konstanta di dalam program.
3. Apabila terdapat dua atau lebih jarak yang bernilai sama, program menampilkan output "Jarak Sama (Tidak Dominan)" tanpa mengklasifikasikan titik ke salah satu cluster.
4. Program wajib menggunakan fungsi untuk menghitung jarak Euclidean.

2.6 Program 6: Interval Konfidensi Proporsi

2.6.1 Deskripsi Soal

Program 6 merupakan program untuk menghitung interval konfidensi proporsi populasi berdasarkan proporsi sampel $\hat{p}$, ukuran sampel n , dan tingkat signifikansi α yang dimasukkan pengguna. Nilai z yang digunakan adalah 1,645 untuk α sebesar 10% dan 1,96 untuk α sebesar 5%. Program ini wajib menggunakan fungsi dan dilengkapi validasi input.

2.6.2 Tabel Input–Proses–Output

Komponen	Keterangan
Input	Proporsi sampel $\hat{p}$, ukuran sampel n , dan tingkat signifikansi α (0.05 atau 0.10) yang dimasukkan pengguna

Proses	Memvalidasi bahwa nilai p -hat berada pada rentang 0 sampai 1; menentukan nilai z berdasarkan α ; menghitung standar error; menghitung batas bawah dan batas atas interval konfidensi
Output	Batas bawah dan batas atas interval konfidensi proporsi, atau pesan kesalahan apabila input tidak valid

2.6.3 Batasan Masalah dan Kondisi Khusus

1. Nilai proporsi sampel p -hat harus berada pada rentang 0 sampai 1. Apabila nilai yang dimasukkan kurang dari 0 atau lebih dari 1, program menampilkan pesan kesalahan sebelum memanggil fungsi.
2. Nilai α yang diterima hanya 0.05 atau 0.10. Input α selain kedua nilai tersebut akan menghasilkan pesan kesalahan di dalam fungsi.
3. Program wajib menggunakan fungsi untuk menghitung interval konfidensi.

2.7 Pembagian Tugas Kelompok

Nama	NIM	Program yang Dikerjakan
Winny Anindita Natasya	K1D025045	Program 3 (Python dan R), Program 6 (R)
Almass Rahma Shaliha	K1D025054	Program 1 (Python dan R), Program 6 (Python)
Yudalaga Satriaji	K1D025057	Program 2 (Python dan R), Program 5 (R)
Arrohmanasyah Al Hakim	K1D025073	Program 4 (Python dan R), Program 5 (Python)

Setiap anggota bertanggung jawab atas seluruh komponen wajib dari program yang dikerjakan, meliputi analisis input-proses-output, pseudocode, flowchart, source code, penjelasan tiap baris kode, screenshot hasil running, dan tabel pengujian.

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 Program 1: Analisis Teks

3.1.1 Pseudocode

ALGORITMA Analisis_Teks

DEKLARASI:

teks : string

teks_tanpa_titik : string

kata : array of string

jumlah_kata : integer

jumlah_kalimat : integer

DESKRIPSI:

START

INPUT teks

jumlah_kalimat ? COUNT(".", teks)

teks_tanpa_titik ? HAPUS(".", teks)

kata ? SPLIT(teks_tanpa_titik, " ")

jumlah_kata ? LENGTH(kata)

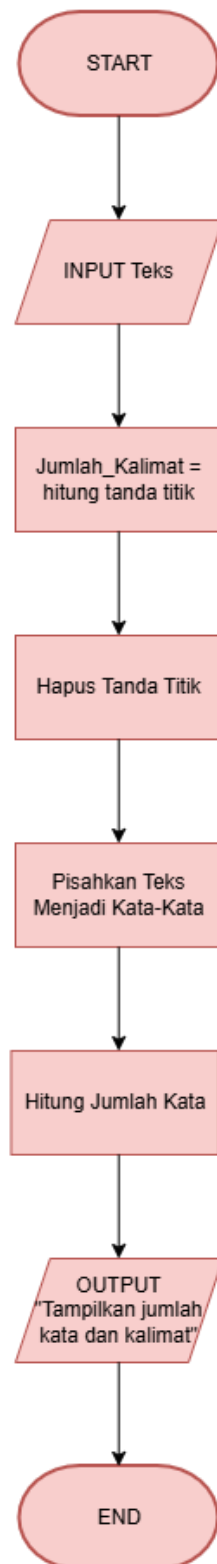
PRINT jumlah_kata

PRINT jumlah_kalimat

END

3.1.2 Flowchart

Flowchart Program 1 - Analisis Teks



3.2 Program 2: String Kalimat

3.2.1 Pseudocode

ALGORITMA Penggunaan_String

DEKLARASI:

K1: string

K2: string

K3: string

K4: string

DESKRIPSI:

START

// Inisialisasi variabel string

K1 = "Saya tak 'kan menyerah."

K2 = 'Ia berkata, "Aku menyayangimu."'

K3 = "'Coba jelaskan pengertian '\cross-validation'\
dalam Machine Learning!'"

K4 = "Surat keputusan itu bernomor
62/UN.34/19/2023."

// Menampilkan output ke layar

PRINT K1

PRINT K2

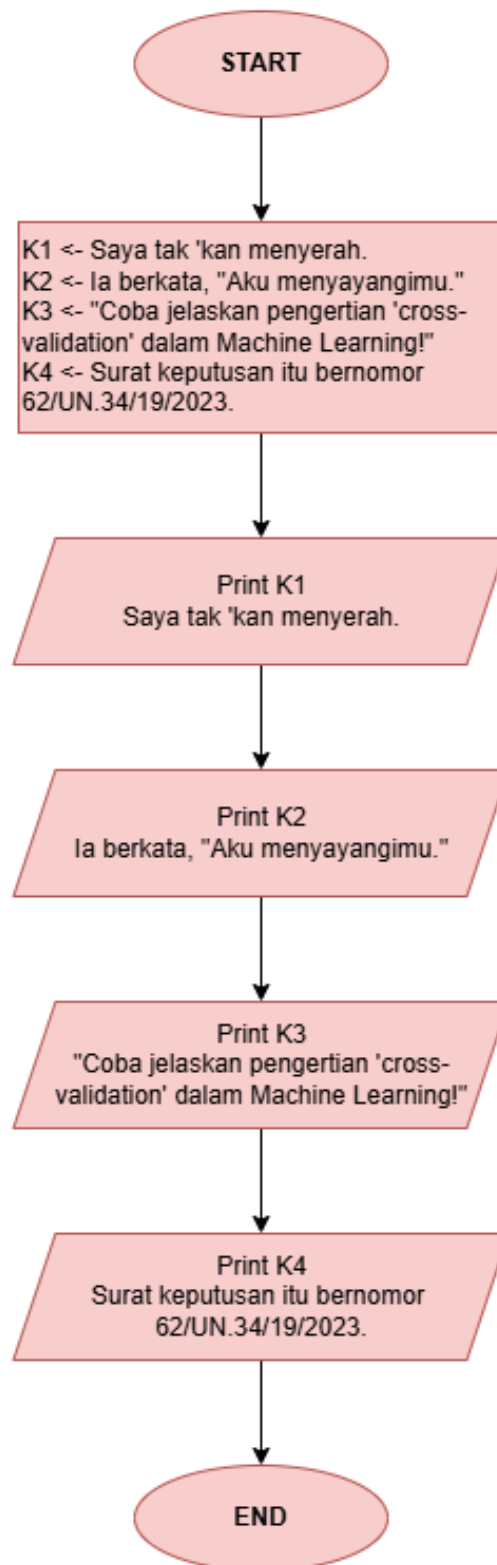
PRINT K3

PRINT K4

END

3.2.2 Flowchart

Flowchart Program 2 - String Kalimat



3.3 Program 3: Akar Persamaan Kuadrat

3.3.1 Pseudocode

ALGORITMA Akar_Kuadrat

DEKLARASI:

a : real
b : real
c : real
diskriminan : real
x1 : real
x2 : real

DESKRIPSI:

START

INPUT a, b, c

IF a == 0 THEN

OUTPUT "Nilai a tidak boleh 0, bukan persamaan kuadrat"

ELSE

diskriminan <- $(b * b) - (4 * a * c)$

IF diskriminan > 0 THEN

x1 <- $(-b + \text{sqrt}(\text{diskriminan})) / (2 * a)$

x2 <- $(-b - \text{sqrt}(\text{diskriminan})) / (2 * a)$

OUTPUT "Dua akar real berbeda"

OUTPUT "x1 = ", x1, " (3 desimal)"

OUTPUT "x2 = ", x2, " (3 desimal)"

ELSE IF diskriminan == 0 THEN

x1 <- $-b / (2 * a)$

OUTPUT "Satu akar real kembar"

OUTPUT "x1 = x2 = ", x1, " (3 desimal)"

ELSE

OUTPUT "Tidak ada akar real"

OUTPUT "Akar-akar imajiner"

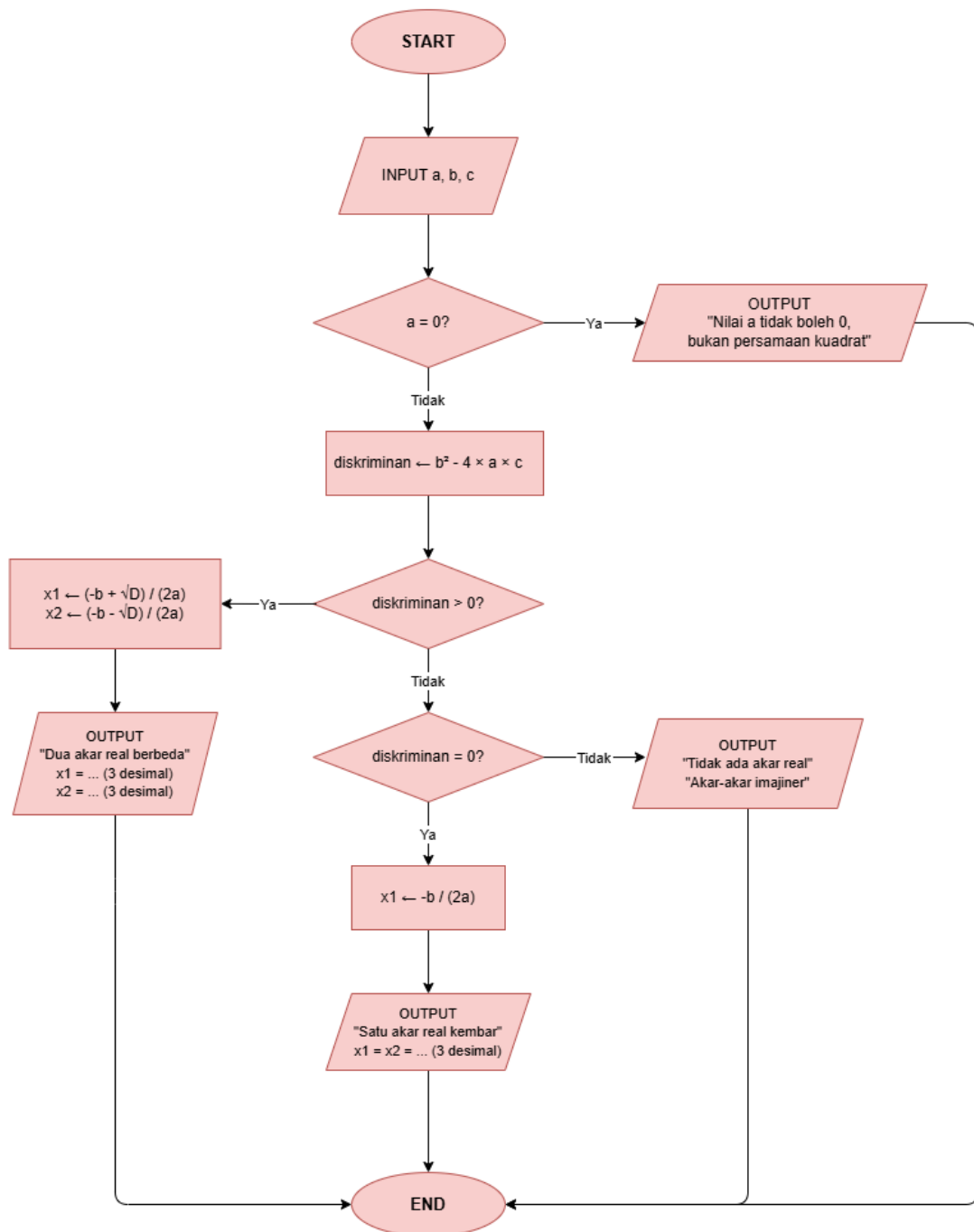
END IF

END IF

END

3.3.2 Flowchart

Flowchart Program 3 – Akar Persamaan Kuadrat



3.4 Program 4: Pengolahan NIP ASN

3.4.1 Pseudocode

ALGORITMA Pengolahan_NIP_ASN

DEKLARASI:

nip: string
tahun: string
bulan_digit: string
tanggal: string
bulan: string
tahun_sekarang: integer
kalender_valid: boolean

DESKRIPSI:

START

READ nip

// 1. Validasi panjang minimal dan memastikan isinya hanya angka

IF LENGTH(nip) >= 8 AND IS_DIGIT(nip) = TRUE THEN

// Ekstraksi substring dari string NIP

tahun = SUBSTRING(nip, 1, 4)

bulan_digit = SUBSTRING(nip, 5, 6)

tanggal = SUBSTRING(nip, 7, 8)

// 2. VALIDASI KALENDER NYATA & TAHUN MASA DEPAN (Jaring Pengaman Baru)

TRY

// Memeriksa keabsahan tanggal kalender nyata (seperti mencegah 31 Februari)

VALIDASI_KALENDER_NYATA(tahun, bulan_digit, tanggal)

// Mengambil tahun berjalan saat ini dari sistem komputer

```

tahun_sekarang = GET_CURRENT_YEAR()

IF CONVERT_TO_INTEGER(tahun) > tahun_sekarang THEN
    PRINT "Error: Tahun lahir tidak logis karena melebihi tahun berjalan saat
ini!"

    kalender_valid = FALSE
ELSE
    kalender_valid = TRUE
END IF

CATCH ValueError
    PRINT "Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di
kalender!"

    kalender_valid = FALSE
END TRY

// 3. Eksekusi logika utama jika seluruh jaring pengaman bernilai VALID
IF kalender_valid == TRUE THEN

    // Percabangan menentukan nama bulan
    IF bulan_digit == "01" THEN
        bulan = "Januari"
    ELSE IF bulan_digit == "02" THEN
        bulan = "Februari"
    ELSE IF bulan_digit == "03" THEN
        bulan = "Maret"
    ELSE IF bulan_digit == "04" THEN
        bulan = "April"
    ELSE IF bulan_digit == "05" THEN
        bulan = "Mei"
    ELSE IF bulan_digit == "06" THEN
        bulan = "Juni"
    ELSE IF bulan_digit == "07" THEN

```

```

        bulan = "Juli"
    ELSE IF bulan_digit == "08" THEN
        bulan = "Agustus"
    ELSE IF bulan_digit == "09" THEN
        bulan = "September"
    ELSE IF bulan_digit == "10" THEN
        bulan = "Oktober"
    ELSE IF bulan_digit == "11" THEN
        bulan = "November"
    ELSE IF bulan_digit == "12" THEN
        bulan = "Desember"
    ELSE
        bulan = "Tidak Valid"
    END IF

    // Output Hasil Akhir
    PRINT "NIP: " + nip
    PRINT "Tanggal Lahir: " + tanggal + " " + bulan + " " + tahun

END IF

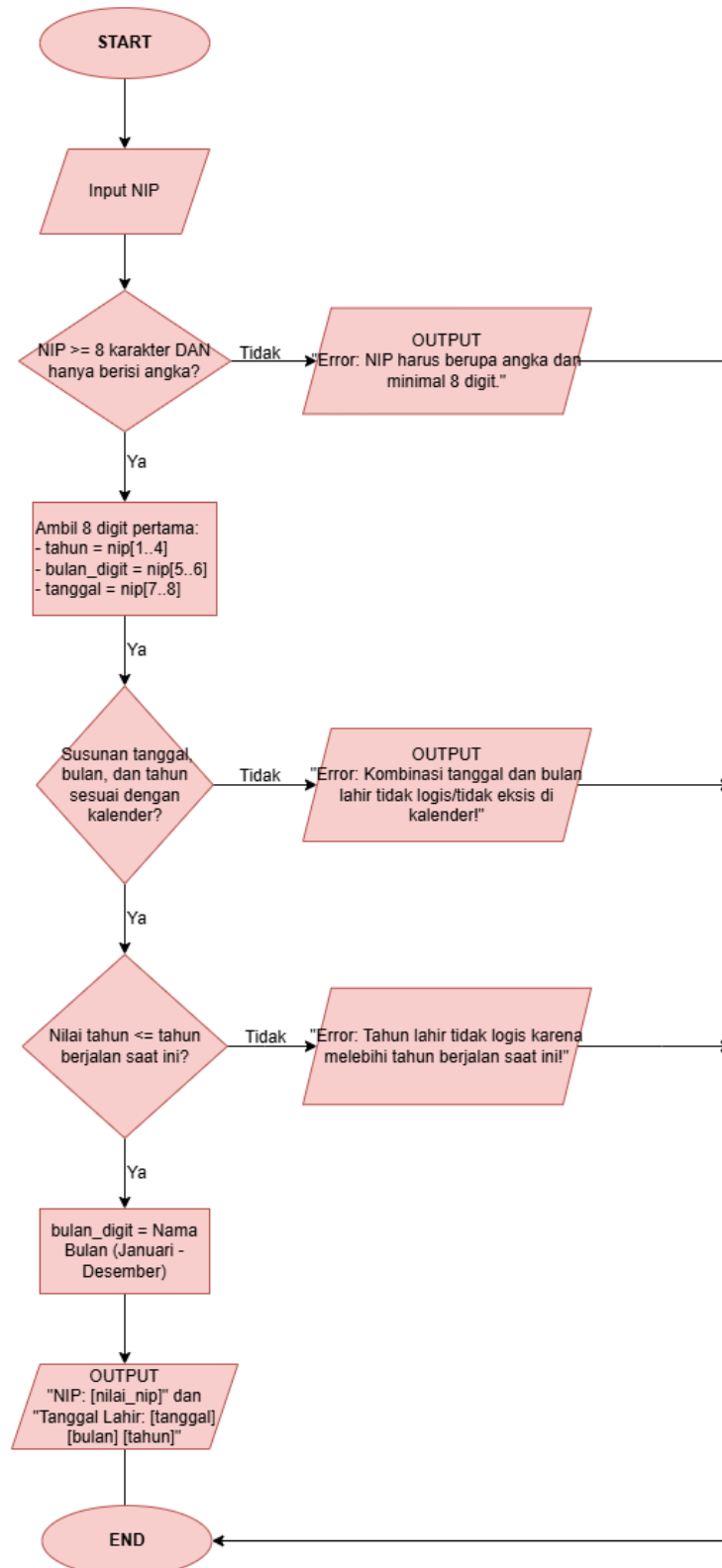
ELSE
    PRINT "Error: NIP harus berupa angka dan minimal 8 digit."
END IF

END

```

3.4.2 Flowchart

Flowchart Program 4 - NIP ASN



3.5 Program 5: Klasifikasi Cluster 3 Dimensi

3.5.1 Pseudocode

ALGORITMA Klasifikasi_Cluster_3D

DEKLARASI:

xA, yA, zA: integer (konstanta pusat A)

xB, yB, zB: integer (konstanta pusat B)

xC, yC, zC: integer (konstanta pusat C)

xU, yU, zU: float (koordinat titik input)

jarak_A, jarak_B, jarak_C: float

cluster_terpilih: string

DESKRIPSI:

START

// Inisialisasi konstanta titik pusat tiap cluster

xA = 2; yA = 1; zA = 3

xB = 1; yB = -4; zB = 6

xC = -2; yC = 3; zC = -2

// Mendefinisikan fungsi perhitungan jarak Euclidean

FUNCTION hitung_jarak(x1, y1, z1, x2, y2, z2)

 jarak = SQUARE_ROOT((x1 - x2)^2 + (y1 - y2)^2 + (z1 - z2)^2)

 RETURN jarak

END FUNCTION

// Menerima input koordinat sembarang U dari pengguna

READ xU, yU, zU

// Memanggil fungsi untuk menghitung jarak titik U ke tiap cluster

jarak_A = CALL hitung_jarak(xU, yU, zU, xA, yA, zA)

```

jarak_B = CALL hitung_jarak(xU, yU, zU, xB, yB, zB)
jarak_C = CALL hitung_jarak(xU, yU, zU, xC, yC, zC)

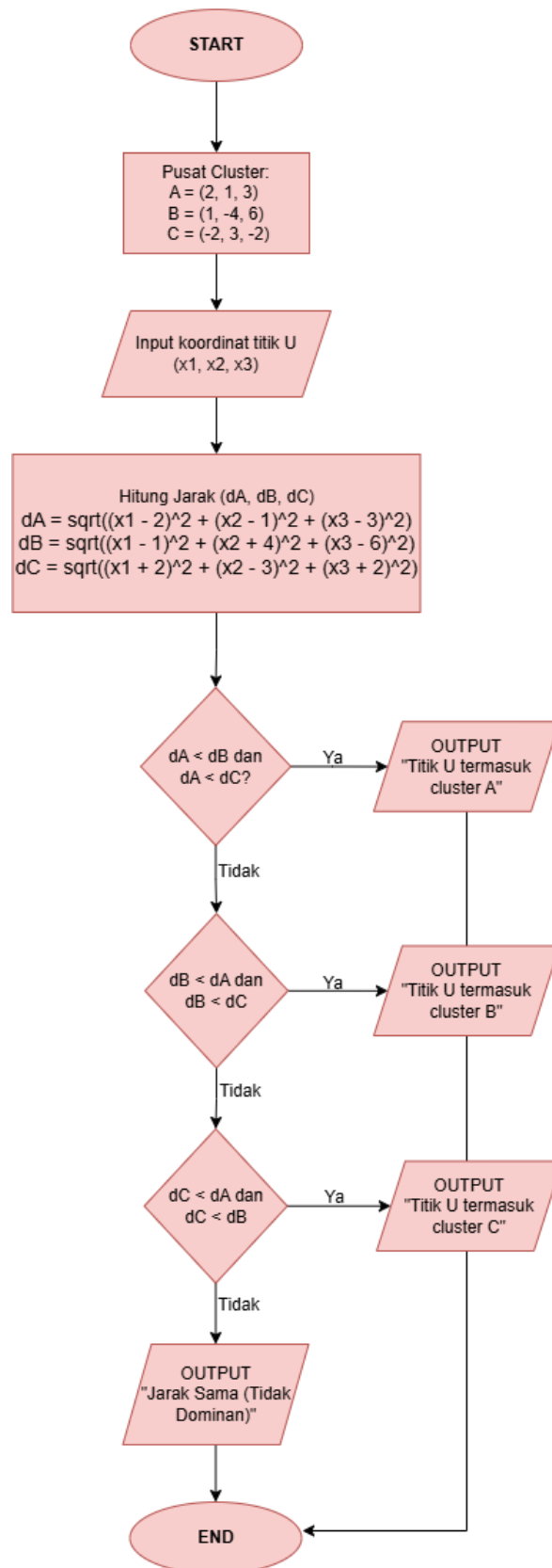
// Percabangan penentuan jarak terdekat
IF jarak_A < jarak_B AND jarak_A < jarak_C THEN
    cluster_terpilih = "Cluster A"
ELSE IF jarak_B < jarak_A AND jarak_B < jarak_C THEN
    cluster_terpilih = "Cluster B"
ELSE IF jarak_C < jarak_A AND jarak_C < jarak_B THEN
    cluster_terpilih = "Cluster C"
ELSE
    cluster_terpilih = "Jarak sama kuat (tidak masuk ke satu cluster spesifik)"
END IF

// Menampilkan hasil
PRINT "Titik U tergolong sebagai anggota: " + cluster_terpilih
END

```

3.5.2 Flowchart

Flowchart Program 5 - Cluster 3d



3.6 Program 6: Interval Konfidensi Proporsi

3.6.1 Pseudocode

ALGORITMA Interval_Proporsi

DEKLARASI:

p_hat : real
n : integer
alpha : real
z : real
se : real
batas_bawah : real
batas_atas : real

DESKRIPSI:

START

FUNCTION interval_kepercayaan(p_hat, n, alpha):

IF p_hat < 0 ATAU p_hat > 1 THEN

 OUTPUT "Error: Proporsi harus berada antara 0 dan 1."

 RETURN

END IF

IF alpha == 0.10 THEN

 z <- 1.645

ELSE IF alpha == 0.05 THEN

 z <- 1.96

ELSE

 OUTPUT "Error: Nilai alpha hanya boleh 0.10 atau 0.05."

 RETURN

END IF

```

se      <- sqrt(p_hat * (1 - p_hat) / n)
batas_bawah <- p_hat - z * se
batas_atas  <- p_hat + z * se

OUTPUT "Interval Kepercayaan (1 - alpha) * 100 %"
OUTPUT batas_bawah < p < batas_atas

END FUNCTION

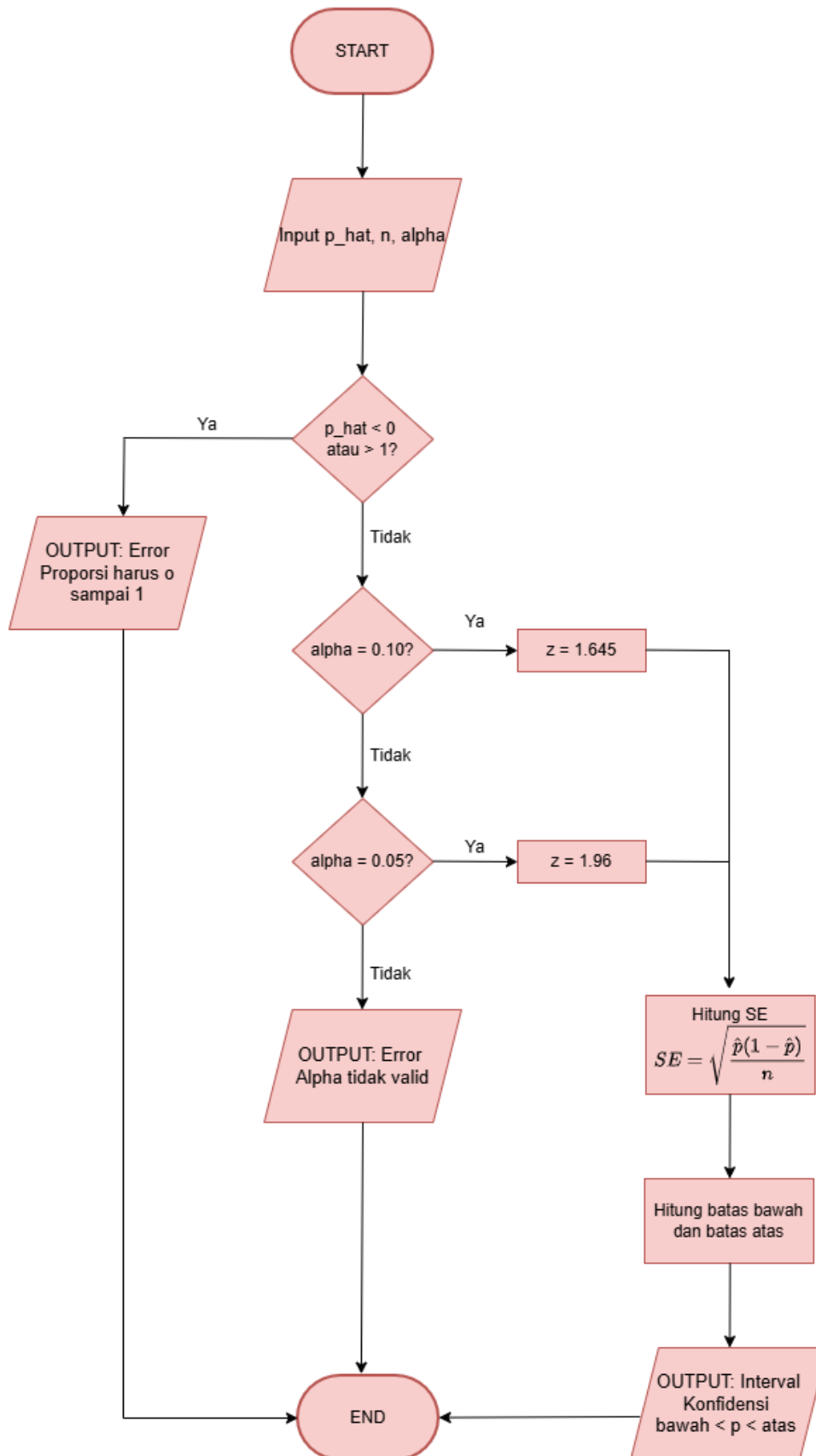
INPUT p_hat
INPUT n
INPUT alpha

CALL interval_kepercayaan(p_hat, n, alpha)
END

```

3.6.2 Flowchart

Flowchart Program 6 - Interval proporsi



BAB IV

IMPLEMENTASI PROGRAM

4.1 Program 1: Analisis Teks

4.1.1 Source Code R

teks <- "Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata tidak hanya digunakan sebagai tempat berkumpul atau berbagi di dunia maya. Namun, media sosial kini juga bisa dimanfaatkan sebagai media untuk mengembangkan sebuah bisnis. Saat ini telah banyak para pengusaha yang beralih ke media sosial dalam memasarkan produk mereka baik barang ataupun jasa. Beralihnya para pelaku bisnis ke media ini dikarenakan jejaring sosial memiliki manfaat yang sangat banyak bagi usaha bisnis. Berikut ini adalah alasan mengapa jejaring sosial bisa menjadi alat promosi yang paling efektif."

```
jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))
```

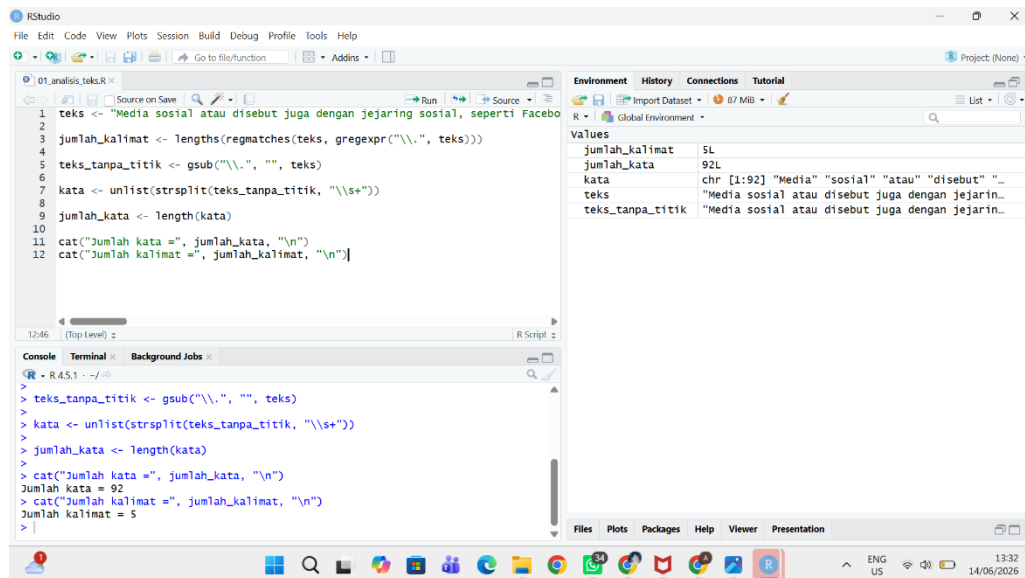
```
teks_tanpa_titik <- gsub("\\.", "", teks)
```

```
kata <- unlist(strsplit(teks_tanpa_titik, "\\s+"))
```

```
jumlah_kata <- length(kata)
```

```
cat("Jumlah kata =", jumlah_kata, "\n")
```

```
cat("Jumlah kalimat =", jumlah_kalimat, "\n")
```



Gambar 4.1 Screenshot Hasil Running Program 1: Analisis Teks (R)

4.1.2 Source Code Python

```
teks = input("Masukkan teks: ")
```

```
jumlah_kalimat = teks.count(".")
```

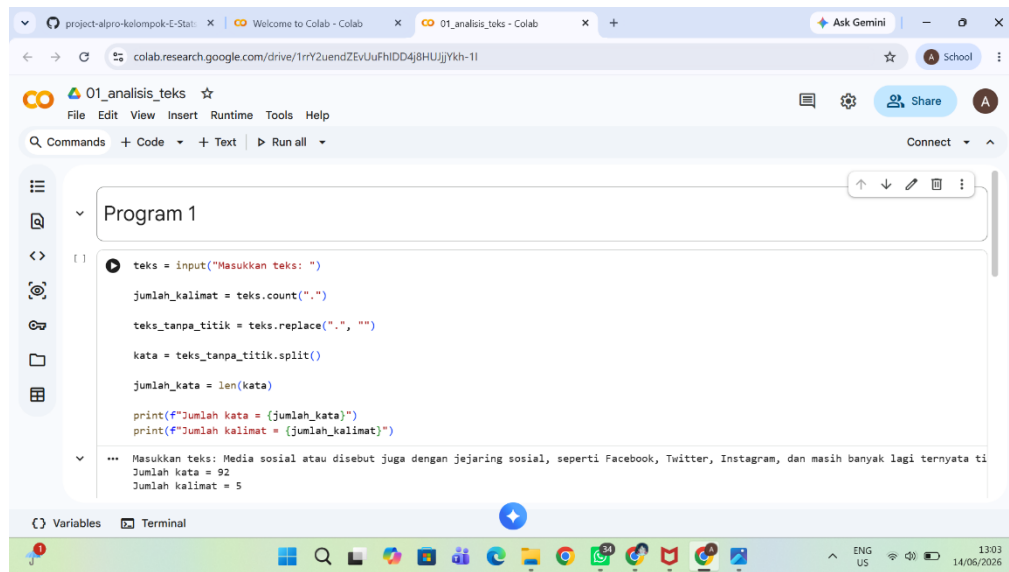
```
teks_tanpa_titik = teks.replace(".", "")
```

```
kata = teks_tanpa_titik.split()
```

```
jumlah_kata = len(kata)
```

```
print(f'Jumlah kata = {jumlah_kata}')
```

```
print(f'Jumlah kalimat = {jumlah_kalimat}')
```

Gambar 4.2 Screenshot Hasil Running Program 1: Analisis Teks (Python)

4.2 Program 2: String Kalimat

4.2.1 Source Code R

Program Membuat Objek String K1-K4 Sesuai Soal

K1 <- "Saya tak 'kan menyerah."

K2 <- 'Ia berkata, "Aku menyayangimu."'

K3 <- "'Coba jelaskan pengertian '\cross-validation\' dalam Machine Learning!'"

K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."

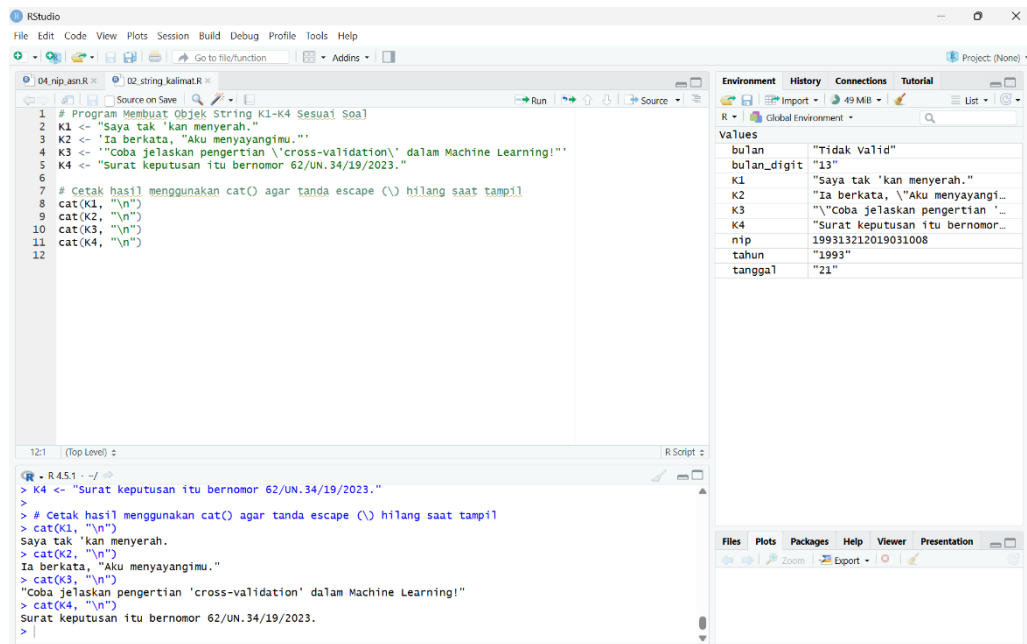
Cetak hasil menggunakan cat() agar tanda escape (\) hilang saat tampil

cat(K1, "\n")

cat(K2, "\n")

cat(K3, "\n")

cat(K4, "\n")



Gambar 4.3 Screenshot Hasil Running Program 2: String Kalimat (R)

4.2.2 Source Code Python

Program Membuat Objek String K1-K4 Sesuai Soal

K1 = "Saya tak 'kan menyerah."

K2 = "Ia berkata, "Aku menyayangimu.""

K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""

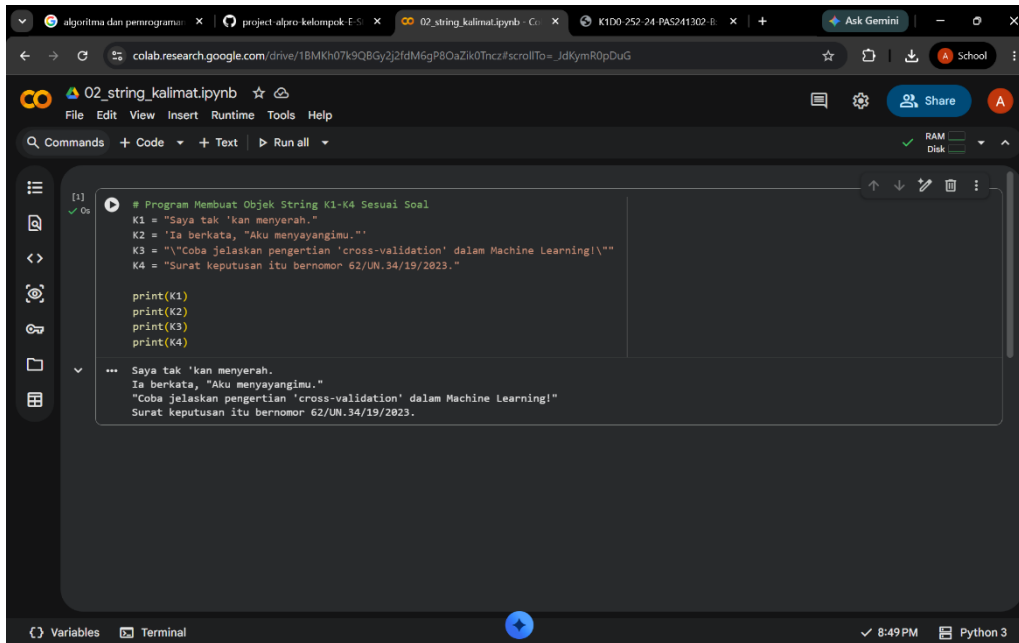
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)

print(K2)

print(K3)

print(K4)



Gambar 4.4 Screenshot Hasil Running Program 2: String Kalimat (Python)

4.3 Program 3: Akar Persamaan Kuadrat

4.3.1 Source Code R

```
# Fungsi untuk menghitung akar persamaan kuadrat

hitung_akar_kuadrat <- function() {

  cat("==== Program Pencari Akar Persamaan Kuadrat ====\n")

  cat("Bentuk umum:  $ax^2 + bx + c = 0$ \n")

  # Membaca koefisien dari pengguna dan mengonversinya ke numerik

  a_input <- readline(prompt = "Masukkan koefisien a: ")

  b_input <- readline(prompt = "Masukkan koefisien b: ")

  c_input <- readline(prompt = "Masukkan koefisien c: ")

  # as.numeric akan menghasilkan NA jika input bukan angka
```

```

a <- suppressWarnings(as.numeric(a_input))

b <- suppressWarnings(as.numeric(b_input))

c <- suppressWarnings(as.numeric(c_input))

# Validasi dasar: memastikan input tidak kosong atau bukan angka
if (is.na(a) || is.na(b) || is.na(c)) {

  cat("\nInput tidak valid! Harap masukkan nilai berupa angka.\n")

  return()

}

# Validasi dasar: a tidak boleh 0 untuk persamaan kuadrat
if (a == 0) {

  cat("Nilai 'a' tidak boleh 0. Ini bukan persamaan kuadrat.\n")

  return()

}

# Menghitung nilai diskriminan
diskriminan <- (b^2) - (4 * a * c)

# Menentukan dan mencetak jenis akar
if (diskriminan > 0) {

  x1 <- (-b + sqrt(diskriminan)) / (2 * a)

  x2 <- (-b - sqrt(diskriminan)) / (2 * a)

```

```

cat("\nPersamaan memiliki dua akar real yang berbeda:\n")

# Mengatur tampilan akar bilangan real dalam tiga desimal menggunakan sprintf

cat(sprintf("x1 = %.3f\n", x1))

cat(sprintf("x2 = %.3f\n", x2))


} else if (diskriminan == 0) {

    x <- -b / (2 * a)

    cat("\nPersamaan memiliki satu akar real (akar kembar):\n")

    # Mengatur tampilan akar bilangan real dalam tiga desimal menggunakan sprintf

    cat(sprintf("x = %.3f\n", x))

} else {

    # Jika diskriminan < 0, maka akar bertipe imajiner

    cat("\nPersamaan tersebut hanya memiliki akar-akar imajiner.\n")

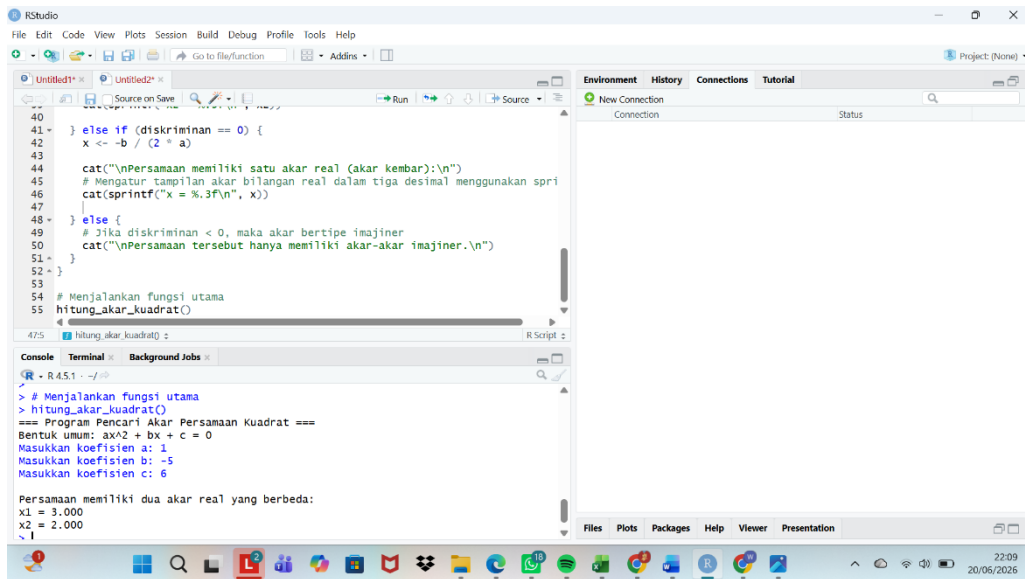
}

}

# Menjalankan fungsi utama

hitung_akar_kuadrat()

```



Gambar 4.5 *Screenshot Hasil Running Program 3: Akar Persamaan Kuadrat (R)*

4.3.2 Source Code Python

```
import math
```

```
def hitung_akar_kuadrat():
```

```
    print("=== Program Pencari Akar Persamaan Kuadrat ===")
```

```
    print("Bentuk umum: ax^2 + bx + c = 0")
```

```
    try:
```

```
        # Membaca koefisien-koefisien persamaan kuadrat dari pengguna
```

```
        a = float(input("Masukkan koefisien a: "))
```

```
        b = float(input("Masukkan koefisien b: "))
```

```
        c = float(input("Masukkan koefisien c: "))
```

```
        # Validasi dasar: a tidak boleh 0 untuk persamaan kuadrat
```

```
        if a == 0:
```

```

print("Nilai 'a' tidak boleh 0. Ini bukan persamaan kuadrat.")

return

# Menghitung nilai diskriminan

diskriminan = b**2 - 4 * a * c

# Menentukan dan mencetak jenis akar

if diskriminan > 0:

    x1 = (-b + math.sqrt(diskriminan)) / (2 * a)

    x2 = (-b - math.sqrt(diskriminan)) / (2 * a)

    print("\nPersamaan memiliki dua akar real yang berbeda:")

    # Mengatur tampilan agar akar bilangan real ditampilkan dalam tiga desimal

    print(f"x1 = {x1:.3f}")

    print(f"x2 = {x2:.3f}")

elif diskriminan == 0:

    x = -b / (2 * a)

    print("\nPersamaan memiliki satu akar real (akar kembar):")

    # Mengatur tampilan agar akar bilangan real ditampilkan dalam tiga desimal

    print(f"x = {x:.3f}")

else:

    # Jika diskriminan < 0, maka akar bertipe imajiner

    print("\nPersamaan tersebut hanya memiliki akar-akar imajiner.")

```

```
except ValueError:
```

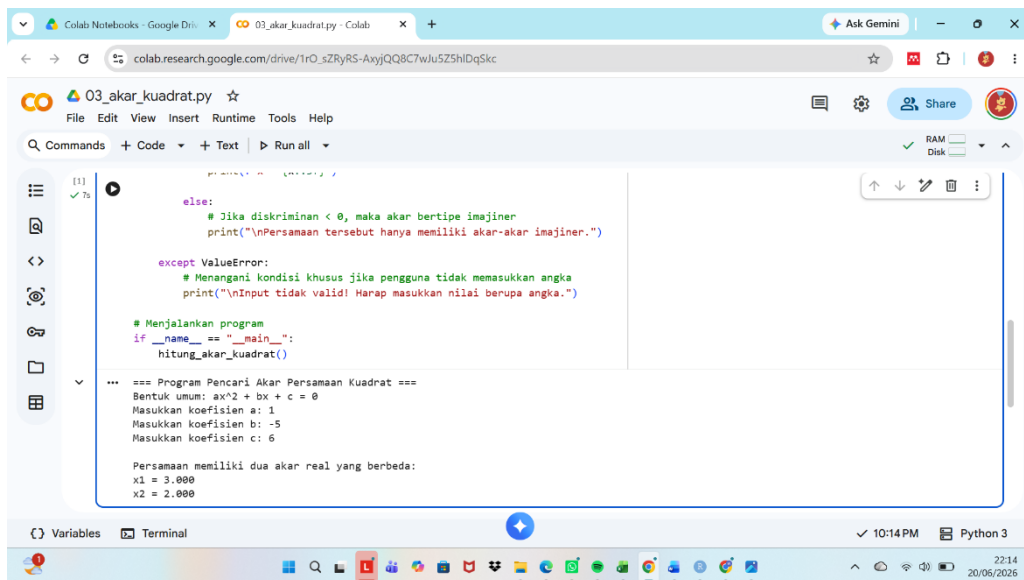
```
# Menangani kondisi khusus jika pengguna tidak memasukkan angka
```

```
print("\nInput tidak valid! Harap masukkan nilai berupa angka.")
```

```
# Menjalankan program
```

```
if __name__ == "__main__":
```

```
    hitung_akar_kuadrat()
```



Gambar 4.6 Screenshot Hasil Running Program 3: Akar Persamaan Kuadrat (Python)

4.4 Program 4: Pengolahan NIP ASN

4.4.1 Source Code R

```
# Program Menentukan Tanggal Lahir dari NIP ASN
```

```
nip <- "200002292023011001"
```

```
# 1. Validasi panjang NIP minimal 8 digit angka
```



```

if (nchar(nip) >= 8 && !grepl("\\D", nip)) {

  tahun <- substr(nip, 1, 4)

  bulan_digit <- substr(nip, 5, 6)

  tanggal <- substr(nip, 7, 8)

# 2. VALIDASI KALENDER NYATA & TAHUN MAJU

# Mengonversi teks menjadi objek Date resmi di R

objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-
%m-%d")

tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))

if (is.na(objek_tanggal)) {

  cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di
kalender!\n")

} else if (as.numeric(tahun) > tahun_sekarang) {

  cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat
ini (%d)!\n", tahun, tahun_sekarang))

} else {

# 3. Percabangan memilih nama bulan jika validasi lolos

if (bulan_digit == "01") {

  bulan <- "Januari"

} else if (bulan_digit == "02") {

  bulan <- "Februari"

} else if (bulan_digit == "03") {

```

```

    bulan <- "Maret"

} else if (bulan_digit == "04") {

    bulan <- "April"

} else if (bulan_digit == "05") {

    bulan <- "Mei"

} else if (bulan_digit == "06") {

    bulan <- "Juni"

} else if (bulan_digit == "07") {

    bulan <- "Juli"

} else if (bulan_digit == "08") {

    bulan <- "Agustus"

} else if (bulan_digit == "09") {

    bulan <- "September"

} else if (bulan_digit == "10") {

    bulan <- "Oktober"

} else if (bulan_digit == "11") {

    bulan <- "November"

} else if (bulan_digit == "12") {

    bulan <- "Desember"

} else {

    bulan <- "Tidak Valid"

}

cat(paste("NIP:", nip, "\n"))

```

```

cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))

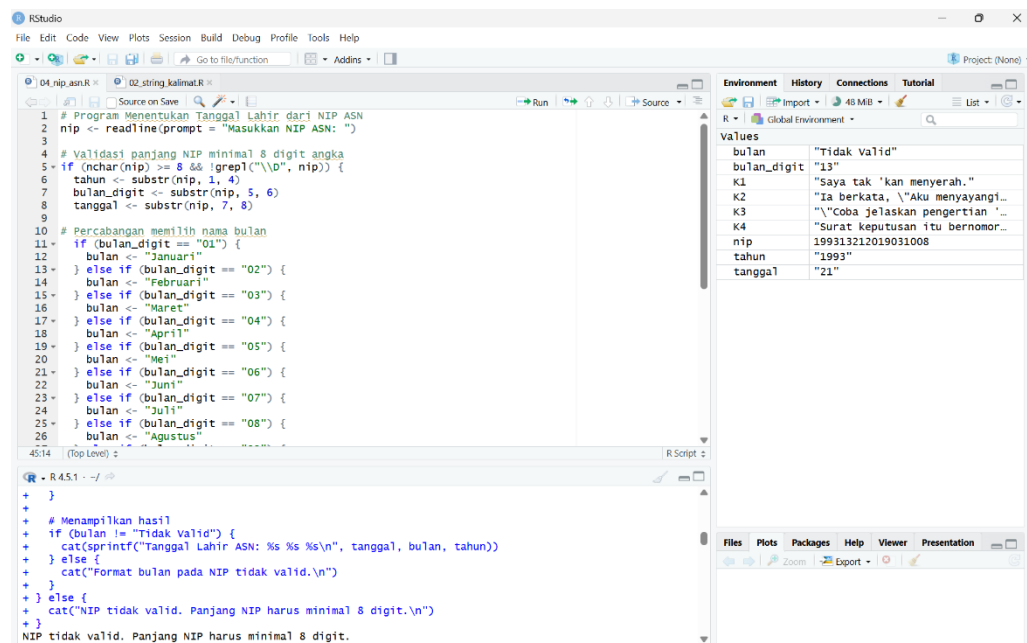
}

} else {

cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")

}

```



Gambar 4.7 Screenshot Hasil Running Program 4: Pengolahan NIP ASN (R)

4.4.2 Source Code Python

```
from datetime import datetime
```

```
# Program Menentukan Tanggal Lahir dari NIP ASN
```

```
nip = "200002292023011001"
```

```
# 1. Validasi panjang NIP minimal 8 digit angka
```

```
if len(nip) >= 8 and nip.isdigit():
```

```
tahun = nip[0:4]
```

```
bulan_digit = nip[4:6]
```

```
tanggal = nip[6:8]
```

2. VALIDASI KALENDER NYATA & TAHUN MAJU

```
try:
```

```
    # Pengecekan keabsahan tanggal kalender
```

```
    objek_tanggal = datetime.strptime(f'{tahun}-{bulan_digit}-{tanggal}', "%Y-%m-%d")
```

```
    # Pengecekan apakah tahun lahir melebihi tahun berjalan saat ini
```

```
    tahun_sekarang = datetime.now().year
```

```
    if int(tahun) > tahun_sekarang:
```

```
        print(f'Error: Tahun lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang})!')
```

```
        kalender_valid = False
```

```
    else:
```

```
        kalender_valid = True
```

```
except ValueError:
```

```
    print("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!")
```

```
    kalender_valid = False
```

```
# Jika lolos seluruh validasi logis, lanjutkan penentuan nama bulan
```

```
if kalender_valid:

    if bulan_digit == "01":

        bulan = "Januari"

    elif bulan_digit == "02":

        bulan = "Februari"

    elif bulan_digit == "03":

        bulan = "Maret"

    elif bulan_digit == "04":

        bulan = "April"

    elif bulan_digit == "05":

        bulan = "Mei"

    elif bulan_digit == "06":

        bulan = "Juni"

    elif bulan_digit == "07":

        bulan = "Juli"

    elif bulan_digit == "08":

        bulan = "Agustus"

    elif bulan_digit == "09":

        bulan = "September"

    elif bulan_digit == "10":

        bulan = "Oktober"

    elif bulan_digit == "11":

        bulan = "November"

    elif bulan_digit == "12":
```

```

        bulan = "Desember"

    else:

        bulan = "Tidak Valid"

    print(f"NIP: {nip}")

    print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")

else:

    print("Error: NIP harus berupa angka dan minimal 8 digit.")

```

```

# Program Memastikan tanggal lahir dari NIP ASN
nip = input("Masukkan NIP ASN: ")

# Validasi panjang NIP minimal 8 digit angka
if len(nip) >= 8 and nip.isdigit():
    tahun = nip[:4]
    bulan_digit = nip[4:6]
    tanggal = nip[6:8]

    # Percabangan memilih nama bulan
    if bulan_digit == "01":
        bulan = "Januari"
    elif bulan_digit == "02":
        bulan = "Februari"
    elif bulan_digit == "03":
        bulan = "Maret"
    elif bulan_digit == "04":
        bulan = "April"
    elif bulan_digit == "05":
        bulan = "Mei"
    elif bulan_digit == "06":
        bulan = "Juni"
    elif bulan_digit == "07":
        bulan = "Juli"
    elif bulan_digit == "08":
        bulan = "Agustus"
    elif bulan_digit == "09":
        bulan = "September"
    elif bulan_digit == "10":
        bulan = "Oktober"
    elif bulan_digit == "11":
        bulan = "November"
    elif bulan_digit == "12":
        bulan = "Desember"
    else:
        bulan = "Tidak valid"

    if bulan != "Tidak Valid":
        print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")
    else:
        print("Error: Format bulan pada NIP tidak valid.")
else:
    print("Error: NIP harus berupa angka dan minimal 8 digit.")

```

Gambar 4.8 Screenshot Hasil Running Program 4: Pengolahan NIP ASN (Python)

4.5 Program 5: Klasifikasi Cluster 3 Dimensi

4.5.1 Source Code R

1. Mendefinisikan Fungsi (Syarat Wajib Modul 8)

```

hitung_jarak <- function(titik1, titik2) {

  # R menggunakan indeks basis 1, bukan 0

```

```

jarak <- sqrt((titik1[1] - titik2[1])^2 +
              (titik1[2] - titik2[2])^2 +
              (titik1[3] - titik2[3])^2)

return(jarak)
}

```

2. Konstanta titik pusat cluster berupa vektor 3 elemen

```
A <- c(2, 1, 3)
```

```
B <- c(1, -4, 6)
```

```
C <- c(-2, 3, -2)
```

```
cat("--- Klasifikasi Cluster 3 Dimensi ---\n")
```

```
cat("Masukkan koordinat titik U(x, y, z):\n")
```

Membaca input dan mengubahnya menjadi angka(Contoh Input)

```
x <- 2
```

```
y <- -3
```

```
z <- 6
```

Validasi jika input kosong atau berupa huruf (menjadi NA saat dipaksa as.numeric)

```
if (!is.na(x) && !is.na(y) && !is.na(z)) {
```

Menyimpan titik U sebagai vektor dengan 3 elemen (Syarat Wajib Soal)

```
U <- c(x, y, z)
```

```
# 3. Memanggil fungsi untuk masing-masing cluster
```

```
jarak_A <- hitung_jarak(U, A)
```

```
jarak_B <- hitung_jarak(U, B)
```

```
jarak_C <- hitung_jarak(U, C)
```

```
# 4. Percabangan menentukan jarak terdekat
```

```
if (jarak_A < jarak_B && jarak_A < jarak_C) {
```

```
  cluster <- "Cluster A"
```

```
} else if (jarak_B < jarak_A && jarak_B < jarak_C) {
```

```
  cluster <- "Cluster B"
```

```
} else if (jarak_C < jarak_A && jarak_C < jarak_B) {
```

```
  cluster <- "Cluster C"
```

```
} else {
```

```
  cluster <- "Jarak Sama (Tidak Dominan)"
```

```
}
```

```
# Menampilkan hasil (menggunakan sprintf untuk pembulatan 3 desimal)
```

```
cat(sprintf("\nJarak titik U ke Pusat A : %.3f\n", jarak_A))
```

```
cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
```

```
cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
```

```
cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
```

```
} else {
```

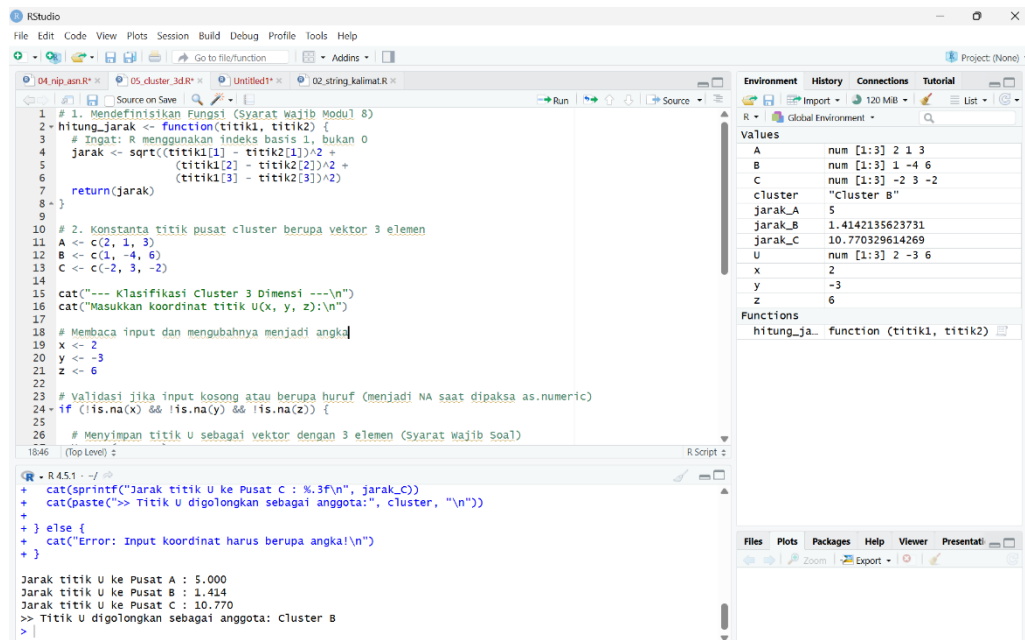


```

cat("Error: Input koordinat harus berupa angka!\n")

}

```



Gambar 4.9 Screenshot Hasil Running Program 5: Klasifikasi Cluster 3D (R)

4.5.2 Source Code Python

```

import math

# 1. Mendefinisikan Fungsi (Syarat Wajib Modul 8)

def hitung_jarak(titik1, titik2):

    # Rumus jarak Euclidean 3D

    jarak = math.sqrt((titik1[0] - titik2[0])**2 +

                       (titik1[1] - titik2[1])**2 +

                       (titik1[2] - titik2[2])**2)

    return jarak

# 2. Konstanta titik pusat cluster berupa tuple

```

$A = (2, 1, 3)$

$B = (1, -4, 6)$

$C = (-2, 3, -2)$

```
print("--- Klasifikasi Cluster 3 Dimensi ---")
```

```
# MASUKKAN CONTOH INPUT
```

```
x = 2
```

```
y = -3
```

```
z = 6
```

```
print(f'Koordinat Titik U yang Diuji -> x: {x}, y: {y}, z: {z}')
```

```
# Menyimpan titik U sebagai tuple dengan 3 elemen (Syarat Wajib Soal)
```

```
U = (x, y, z)
```

```
# 3. Memanggil fungsi untuk masing-masing cluster
```

```
jarak_A = hitung_jarak(U, A)
```

```
jarak_B = hitung_jarak(U, B)
```

```
jarak_C = hitung_jarak(U, C)
```

```
# 4. Percabangan menentukan jarak terdekat
```

```
if jarak_A < jarak_B and jarak_A < jarak_C:
```

```
    cluster = "Cluster A"
```

```
elif jarak_B < jarak_A and jarak_B < jarak_C:
```

```

cluster = "Cluster B"

elif jarak_C < jarak_A and jarak_C < jarak_B:

    cluster = "Cluster C"

else:

    cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)

print(f'\nJarak titik U ke Pusat A : {jarak_A:.3f}')

print(f'Jarak titik U ke Pusat B : {jarak_B:.3f}')

print(f'Jarak titik U ke Pusat C : {jarak_C:.3f}')

print(f'>> Titik U digolongkan sebagai anggota: {cluster}')

```

The screenshot shows a Python script in an IDE. The script defines a function `hitung_jarak` to calculate the distance between two points in 3D space. It then defines a function `klasifikasi_cluster` that takes a point `titik_u` and three clusters (`cluster_a`, `cluster_b`, `cluster_c`) as input. The function calculates the distances from the point to each cluster center and classifies the point into one of the clusters based on the minimum distance. If the distances are equal, it classifies the point as 'Tidak Dominan'. The script then prints the results for a specific point `titik_u`.

```

def hitung_jarak(x1,y1,z1,x2,y2,z2):
    jarak = math.sqrt((x1-x2)**2 + (y1-y2)**2 + (z1-z2)**2)
    return jarak

# 1. Koordinat titik pusat cluster berupa tuple
a = (1, 2, 3)
b = (4, 5, 6)
c = (7, 8, 9)

def klasifikasi_cluster(titik_u, cluster_a, cluster_b, cluster_c):
    # Hitung jarak
    jarak_a = hitung_jarak(titik_u, a)
    jarak_b = hitung_jarak(titik_u, b)
    jarak_c = hitung_jarak(titik_u, c)

    # Tentukan cluster
    if jarak_a < jarak_b and jarak_a < jarak_c:
        cluster = "Cluster A"
    elif jarak_b < jarak_a and jarak_b < jarak_c:
        cluster = "Cluster B"
    elif jarak_c < jarak_a and jarak_c < jarak_b:
        cluster = "Cluster C"
    else:
        cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)
print(f'Jarak titik U ke Pusat A : {jarak_a:.3f}')
print(f'Jarak titik U ke Pusat B : {jarak_b:.3f}')
print(f'Jarak titik U ke Pusat C : {jarak_c:.3f}')
print(f'>> Titik U digolongkan sebagai anggota: {cluster}')

# ...
# Klasifikasi Cluster 3 dimensi
koordinat_titik_u_seng_kuat = (1, 2, 3)
jarak_a = hitung_jarak(koordinat_titik_u_seng_kuat, a)
jarak_b = hitung_jarak(koordinat_titik_u_seng_kuat, b)
jarak_c = hitung_jarak(koordinat_titik_u_seng_kuat, c)
cluster = klasifikasi_cluster(koordinat_titik_u_seng_kuat, a, b, c)
print(f'>> Titik U digolongkan sebagai anggota: {cluster}')

```

Gambar 4.10 Screenshot Hasil Running Program 5: Klasifikasi Cluster 3D (Python)

4.6 Program 6: Interval Konfidensi Proporsi

4.6.1 Source Code R

```
interval_kepercayaan <- function(p_hat, n, alpha) {  
  
  # Validasi NA (input kosong atau tidak valid)  
  
  if (is.na(p_hat) || is.na(n) || is.na(alpha)) {  
  
    cat("Error: Input tidak boleh kosong atau bukan angka.\n")  
  
    return(invisible(NULL))  
  
  }  
  
  
  if (p_hat < 0 || p_hat > 1) {  
  
    cat("Error: Proporsi harus berada antara 0 dan 1.\n")  
  
    return(invisible(NULL))  
  
  }  
  
  
  if (alpha == 0.10) {  
  
    z <- 1.645  
  
  } else if (alpha == 0.05) {  
  
    z <- 1.96  
  
  } else {  
  
    cat("Error: Nilai alpha hanya boleh 0.10 atau 0.05.\n")  
  
    return(invisible(NULL))  
  
  }  
}
```

```

se <- sqrt((p_hat * (1 - p_hat)) / n)

batas_bawah <- p_hat - z * se

batas_atas <- p_hat + z * se

cat(sprintf("\nInterval Kepercayaan %.0f%%\n", (1 - alpha) * 100))

cat(sprintf("%.4f < p < %.4f\n", batas_bawah, batas_atas))

}

# Input

p_hat <- as.numeric(readline("Masukkan nilai proporsi sampel (p_hat): "))

n <- as.integer(readline("Masukkan nilai n: "))

alpha <- as.numeric(readline("Masukkan nilai alpha (0.10 atau 0.05): "))

interval_kepercayaan(p_hat, n, alpha)

```

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function Addins
Source on Save Run Source
21 }
22
23 se <- sqrt((p_hat * (1 - p_hat)) / n)
24 batas_bawah <- p_hat - z * se
25 batas_atas <- p_hat + z * se
26
27 cat(sprintf("\nInterval Kepercayaan %.0f%%\n", (1 - alpha) * 100))
28 cat(sprintf("%.4f < p < %.4f\n", batas_bawah, batas_atas))
29 }
30
31 # Input
32 p_hat <- as.numeric(readline("Masukkan nilai proporsi sampel (p_hat): "))
33 n <- as.integer(readline("Masukkan nilai n: "))
34 alpha <- as.numeric(readline("Masukkan nilai alpha (0.10 atau 0.05): "))
35
36 interval_kepercayaan(p_hat, n, alpha)
37
37:1 (Top Level)
R Script
Console Terminal Background Jobs
> # Input
> p_hat <- as.numeric(readline("Masukkan nilai proporsi sampel (p_hat): "))
Masukkan nilai proporsi sampel (p_hat): 0.6
> n <- as.integer(readline("Masukkan nilai n: "))
Masukkan nilai n: 100
> alpha <- as.numeric(readline("Masukkan nilai alpha (0.10 atau 0.05): "))
Masukkan nilai alpha (0.10 atau 0.05): 0.10
> interval_kepercayaan(p_hat, n, alpha)
Interval Kepercayaan 90%
0.5194 < p < 0.6806
Files Plots Packages Help Viewer Presentation
22:12
20/05/2025

```

Gambar 4.11 Screenshot Hasil Running Program 6: Interval Konfidensi Proporsi (R)

4.6.2 Source Code Python

```
# Interval Konfidensi Proporsi Populasi

def hitung_interval(p_hat, n, alpha):

    # Menentukan nilai z berdasarkan alpha

    if alpha == 0.10:

        z = 1.645

    elif alpha == 0.05:

        z = 1.96

    else:

        print("Error: Nilai alpha hanya boleh 0.10 atau 0.05.")

        return

    # Menghitung standar error

    se = ((p_hat * (1 - p_hat)) / n) ** 0.5

    # Menghitung batas interval

    batas_bawah = p_hat - z * se

    batas_atas = p_hat + z * se

    # Menampilkan hasil

    print(f"\nInterval Kepercayaan {(1 - alpha) * 100:.0f}%:")

    print(f"{batas_bawah:.4f} < p < {batas_atas:.4f}")
```

```
# Input
```

```
p_hat = float(input("Masukkan proporsi sampel (p_hat): "))
```

```
n = int(input("Masukkan ukuran sampel (n): "))
```

```
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))
```

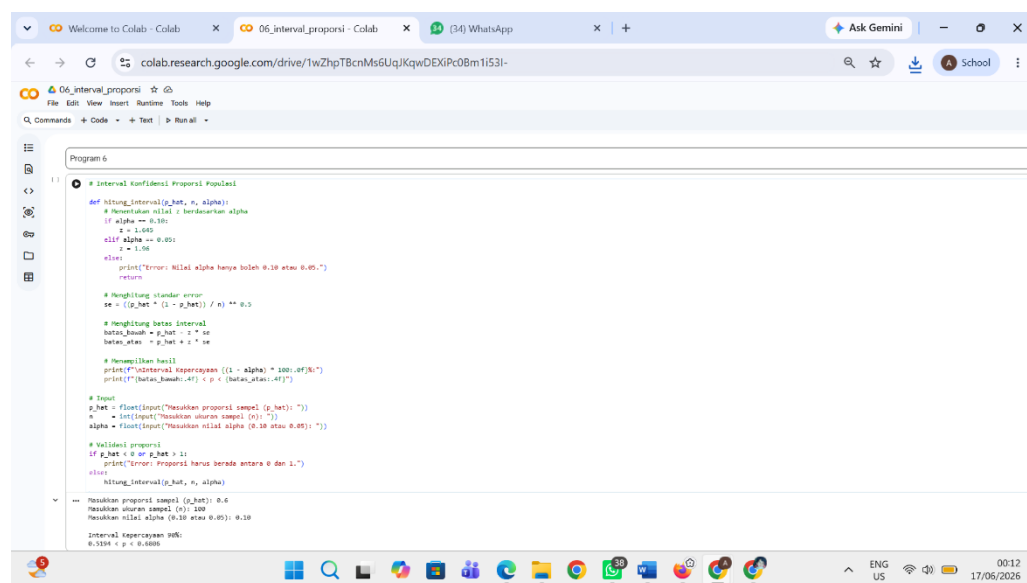
```
# Validasi proporsi
```

```
if p_hat < 0 or p_hat > 1:
```

```
    print("Error: Proporsi harus berada antara 0 dan 1.")
```

```
else:
```

```
    hitung_interval(p_hat, n, alpha)
```



```
Program 6

1.1 # Interval Konfidensi Proporsi Populasi
def hitung_interval(p_hat, n, alpha):
    # Menentukan nilai z berdasarkan alpha
    if alpha == 0.10:
        z = 1.645
    elif alpha == 0.05:
        z = 1.96
    else:
        print("Error: Nilai alpha hanya boleh 0.10 atau 0.05.")
        return

    # Menghitung standar error
    se = ((p_hat * (1 - p_hat)) / n) ** 0.5

    # Menghitung batas interval
    batas_bawah = p_hat - z * se
    batas_atas = p_hat + z * se

    # Menampilkan hasil
    print(f"Interval kepercayaan {(1 - alpha) * 100:.0f}%:")
    print(f"({batas_bawah:.4f}) < p < ({batas_atas:.4f})")

# Input
p_hat = float(input("Masukkan proporsi sampel (p_hat): "))
n = int(input("Masukkan ukuran sampel (n): "))
alpha = float(input("Masukkan nilai alpha (0.10 atau 0.05): "))

# Validasi proporsi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi harus berada antara 0 dan 1.")
else:
    hitung_interval(p_hat, n, alpha)

Masukkan proporsi sampel (p_hat): 0.6
Masukkan ukuran sampel (n): 100
Masukkan nilai alpha (0.10 atau 0.05): 0.10
Interval kepercayaan 90%:
0.3368 < p < 0.6632
```

Gambar 4.12 Screenshot Hasil Running Program 6: Interval Konfidensi Proporsi (Python)

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Program 1: Analisis Teks

5.1.1 Kode R

No	Baris Kode R	Penjelasan
1	<code>teks <- "..."</code>	Menyimpan teks yang akan dianalisis ke dalam variabel teks.
2	<code>jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))</code>	<code>gregexpr()</code> mencari seluruh tanda titik dalam teks, <code>regmatches()</code> mengambil hasil pencarian, dan <code>lengths()</code> menghitung jumlahnya sehingga diperoleh jumlah kalimat.
3	<code>teks_tanpa_titik <- gsub("\\.", "", teks)</code>	Menghapus seluruh tanda titik pada teks menggunakan fungsi <code>gsub()</code> .
4	<code>kata <- unlist(strsplit(teks_tanpa_titik, "\\s+"))</code>	<code>strsplit()</code> memisahkan teks berdasarkan spasi, kemudian <code>unlist()</code> mengubah hasilnya menjadi vektor kata.
5	<code>jumlah_kata <- length(kata)</code>	Menghitung jumlah kata yang tersimpan dalam vektor kata.
6	<code>cat(...)</code>	Menampilkan hasil jumlah kalimat dan jumlah kata ke layar.

5.1.2 Kode Python

No	Baris Kode Python	Penjelasan
1	<code>teks = "..."</code>	Menyimpan teks yang akan dianalisis ke dalam variabel teks.
2	<code>jumlah_kalimat = teks.count(".")</code>	Menghitung jumlah tanda titik (.) pada teks. Karena setiap titik dianggap sebagai akhir kalimat, jumlah titik sama dengan jumlah kalimat.

3	<code>teks_tanpa_titik = teks.replace(".", "")</code>	Menghapus semua tanda titik agar tidak memengaruhi proses penghitungan kata.
4	<code>jumlah_kata = len(teks_tanpa_titik.split())</code>	<code>split()</code> memisahkan teks menjadi kata-kata berdasarkan spasi, kemudian <code>len()</code> menghitung banyaknya kata yang terbentuk.
5	<code>print(f"...")</code>	Menampilkan hasil jumlah kalimat dan jumlah kata ke layar menggunakan format string.

5.2 Program 2: String Kalimat

5.2.1 Kode R

No	Baris Kode R	Penjelasan Logika
1	<code># Program Membuat Objek String K1-K4 Sesuai Soal</code>	Komentar yang menjelaskan bahwa program digunakan untuk membuat dan menampilkan empat objek string sesuai dengan ketentuan soal.
2	<code>K1 <- "Saya tak 'kan menyerah."</code>	Membuat objek K1 dan mengisi objek tersebut dengan kalimat pertama yang akan digunakan sebagai output program.
3	<code>K2 <- 'Ia berkata, "Aku menyayangimu."'</code>	Membuat objek K2 untuk menyimpan kalimat kedua yang mengandung tanda petik ganda di dalam string.
4	<code>K3 <- "'Coba jelaskan pengertian '\cross- validation\' dalam Machine Learning!'"</code>	Membuat objek K3 untuk menyimpan kalimat ketiga. Karakter escape (<code>\</code>) digunakan agar tanda petik tunggal dapat ditampilkan sebagai bagian dari teks.

5	K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat objek K4 yang berisi kalimat terakhir sehingga seluruh data yang diperlukan program telah tersimpan.
6	# Cetak hasil menggunakan cat() agar tanda escape (\) hilang saat tampil	Komentar yang menjelaskan bahwa fungsi cat() digunakan untuk menampilkan isi setiap objek string ke layar.
7	cat(K1, "\n")	Menampilkan isi objek K1 sebagai baris pertama output, kemudian berpindah ke baris berikutnya menggunakan \n.
8	cat(K2, "\n")	Menampilkan isi objek K2 sebagai baris kedua output dan melanjutkan ke baris berikutnya.
9	cat(K3, "\n")	Menampilkan isi objek K3 sebagai baris ketiga output dengan format tanda petik yang sesuai.
10	cat(K4, "\n")	Menampilkan isi objek K4 sebagai baris terakhir sehingga seluruh hasil program dapat ditampilkan dengan lengkap.

5.2.2 Kode Python

No	Baris Kode Python	Penjelasan Logika
1	# Program Membuat Objek String K1-K4 Sesuai Soal	Komentar yang digunakan untuk menjelaskan tujuan program, yaitu membuat dan menampilkan objek string K1 sampai K4 sesuai dengan soal.
2	K1 = "Saya tak 'kan menyerah."	Membuat variabel K1 dan menyimpan kalimat pertama ke dalamnya agar dapat digunakan pada saat proses output.

3	K2 = 'Ia berkata, "Aku menyayangimu."'	Membuat variabel K2 untuk menyimpan kalimat kedua yang mengandung tanda petik ganda sebagai bagian dari teks.
4	K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Membuat variabel K3 untuk menyimpan kalimat ketiga. Karakter escape (\) digunakan agar tanda petik ganda dapat ditampilkan sebagai teks, bukan dianggap sebagai pembatas string.
5	K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat variabel K4 dan menyimpan kalimat keempat yang akan ditampilkan sebagai bagian dari hasil program.
6	print(K1)	Menampilkan isi variabel K1 ke layar sebagai baris pertama output program.
7	print(K2)	Menampilkan isi variabel K2 ke layar sebagai baris kedua output program.
8	print(K3)	Menampilkan isi variabel K3 ke layar sebagai baris ketiga output program dengan format tanda petik yang sesuai.
9	print(K4)	Menampilkan isi variabel K4 ke layar sebagai baris terakhir output program sehingga seluruh hasil dapat ditampilkan.

5.3 Program 3: Akar Persamaan Kuadrat

5.3.1 Kode R

No	Baris Kode R	Penjelasan Logika
1	hitung_akar_kuadrat <- function() {	Mendefinisikan fungsi utama bernama hitung_akar_kuadrat yang berisi seluruh alur logika dan perhitungan program persamaan kuadrat.

2	<code>cat("==== Program Pencari Akar Persamaan Kuadrat ====\n") cat("Bentuk umum: $ax^2 + bx + c = 0$\n")</code>	Menampilkan judul program dan format persamaan kuadrat ke layar agar pengguna memahami bentuk persamaan yang akan dihitung.
3	<code>a_input <- readline(prompt = "Masukkan koefisien a: ") b_input <- readline(prompt = "Masukkan koefisien b: ") c_input <- readline(prompt = "Masukkan koefisien c: ")</code>	Meminta pengguna untuk memasukkan nilai koefisien a, b, dan c. Input dibaca sebagai string karena <code>readline()</code> di R selalu mengembalikan tipe karakter, sehingga perlu dikonversi sebelum digunakan dalam operasi matematika.
4	<code>a <- suppressWarnings(as.numer ic(a_input)) b <- suppressWarnings(as.numer ic(b_input)) c <- suppressWarnings(as.numer ic(c_input))</code>	Mengubah tipe data input dari string menjadi numerik agar dapat dikenakan operasi matematika. Fungsi <code>suppressWarnings</code> digunakan untuk menyembunyikan pesan warning bawaan R jika pengguna memasukkan huruf, karena kondisi tersebut akan ditangani sendiri pada baris validasi berikutnya.
5	<code>if (is.na(a) is.na(b) is.na(c)) { cat("\nInput tidak valid!...\n") return() }</code>	Melakukan validasi input. Jika ada nilai yang gagal dikonversi menjadi angka sehingga bernilai NA, program mencetak pesan error lalu menghentikan eksekusi menggunakan
6	<code>if (a == 0) { cat("Nilai 'a' tidak boleh 0...\n") return() }</code>	Memvalidasi syarat utama persamaan kuadrat. Jika koefisien a bernilai nol, persamaan berubah menjadi linear, sehingga program mencetak penolakan dan eksekusi dihentikan.
7	<code>diskriminan <- (b^2) - (4 * a * c)</code>	Menghitung nilai diskriminan berdasarkan koefisien yang dimasukkan untuk menentukan jenis akar dari persamaan kuadrat tersebut.

8	<code>if (diskriminan > 0) {</code>	Membuka percabangan kondisi pertama yang dieksekusi jika diskriminan bernilai lebih dari 0, yang berarti persamaan memiliki dua akar real yang berbeda.
9	<code>x1 <- (-b + sqrt(diskriminan)) / (2 * a) x2 <- (-b - sqrt(diskriminan)) / (2 * a)</code>	Menghitung nilai akar pertama (x1) dan akar kedua (x2) menggunakan rumus ABC. Fungsi sqrt() digunakan untuk menarik akar kuadrat dari nilai diskriminan.
10	<code>cat(sprintf("x1 = %.3f\n", x1)) cat(sprintf("x2 = %.3f\n", x2))</code>	Menampilkan hasil perhitungan kedua akar ke layar. Penggunaan perintah sprintf("%.3f") digunakan untuk membatasi tampilan nilai desimal menjadi tiga angka di belakang koma.
11	<code>} else if (diskriminan == 0) {</code>	Membuka percabangan kondisi kedua yang dieksekusi jika diskriminan bernilai tepat 0, yang berarti persamaan memiliki satu akar real kembar.
12	<code>x <- -b / (2 * a)</code>	Menghitung nilai akar kembar. Karena diskriminan bernilai 0, maka $\sqrt{\text{diskriminan}} = 0$, sehingga rumus $-b \pm 0 / (2a)$ menyederhanakan menjadi $x = -b / (2a)$ dan menghasilkan satu nilai akar yang pasti.
13	<code>cat(sprintf("x = %.3f\n", x))</code>	Mencetak nilai akar kembar ke layar dengan format presisi tiga desimal.
14	<code>} else { cat("\nPersamaan tersebut hanya memiliki akar-akar imajiner.\n") }</code>	Menangani kondisi tersisa, yaitu ketika diskriminan bernilai negatif. Program tidak melakukan perhitungan lebih lanjut, melainkan mencetak teks yang menyatakan bahwa persamaan hanya memiliki akar-akar imajiner.

15	hitung_akar_kuadrat()	Memanggil dan menjalankan fungsi utama yang sudah didefinisikan di atas agar program dapat berjalan saat di-run.
----	-----------------------	--

5.3.2 Kode Python

No	Baris Kode Python	Penjelasan Logika
1	import math	Mengimpor modul math bawaan Python agar fungsi math.sqrt() untuk menghitung akar kuadrat dapat digunakan dalam program.
2	def hitung_akar_kuadrat():	Mendefinisikan fungsi utama bernama hitung_akar_kuadrat yang berisi seluruh alur logika dan perhitungan program persamaan kuadrat.
3	print("==== Program Pencari Akar Persamaan Kuadrat ====") print("Bentuk umum: $ax^2 + bx + c = 0$ ")	Menampilkan judul program dan format persamaan kuadrat ke layar agar pengguna memahami bentuk persamaan yang akan dihitung.
4	try:	Membuka blok try sebagai mekanisme penanganan error. Seluruh proses input dan perhitungan diletakkan di dalam blok ini agar program tidak berhenti secara paksa jika terjadi kesalahan input.
5	a = float(input("Masukkan koefisien a: ")) b = float(input("Masukkan koefisien b: ")) c = float(input("Masukkan koefisien c: "))	Meminta pengguna memasukkan nilai koefisien a, b, dan c, lalu langsung mengonversinya ke tipe float agar dapat digunakan dalam operasi matematika. Jika input bukan angka, Python otomatis melempar ValueError yang ditangkap oleh blok except.
6	if a == 0: print("Nilai 'a' tidak boleh 0...") return	Memvalidasi syarat utama persamaan kuadrat. Jika koefisien a bernilai nol, persamaan berubah menjadi persamaan linear sehingga program mencetak penolakan dan eksekusi dihentikan menggunakan return.

7	$\text{diskriminan} = b^2 - 4 * a * c$	Menghitung nilai diskriminan berdasarkan koefisien yang dimasukkan untuk menentukan jenis akar dari persamaan kuadrat tersebut.
8	<code>if diskriminan > 0:</code>	Membuka percabangan kondisi pertama yang dieksekusi jika diskriminan bernilai lebih dari 0, yang berarti persamaan memiliki dua akar real yang berbeda.
9	$x_1 = (-b + \text{math.sqrt}(\text{diskriminan})) / (2 * a)$ $x_2 = (-b - \text{math.sqrt}(\text{diskriminan})) / (2 * a)$	Menghitung nilai akar pertama (x_1) dan akar kedua (x_2) menggunakan rumus ABC. Fungsi <code>math.sqrt()</code> digunakan untuk menarik akar kuadrat dari nilai diskriminan.
10	<code>print(f"x1 = {x1:.3f}")</code> <code>print(f"x2 = {x2:.3f}")</code>	Menampilkan hasil kedua akar ke layar. Format f-string dengan <code>:.3f</code> digunakan untuk membatasi tampilan nilai desimal menjadi tiga angka di belakang koma.
11	<code>elif diskriminan == 0:</code>	Membuka percabangan kondisi kedua yang dieksekusi jika diskriminan bernilai tepat 0, yang berarti persamaan memiliki satu akar real kembar.
12	$x = -b / (2 * a)$	Menghitung nilai akar kembar. Karena diskriminan bernilai 0, maka $\sqrt{\text{diskriminan}} = 0$, sehingga rumus $-b \pm 0 / (2a)$ menyederhanakan menjadi $x = -b / (2a)$ dan menghasilkan satu nilai akar yang pasti.
13	<code>print(f"x = {x:.3f}")</code>	Mencetak nilai akar kembar ke layar dengan format presisi tiga desimal.
14	<code>else: print("\nPersamaan tersebut hanya memiliki akar-akar imajiner.")</code>	Menangani kondisi tersisa, yaitu ketika diskriminan bernilai negatif. Program tidak melakukan perhitungan lebih lanjut, melainkan mencetak teks yang menyatakan bahwa persamaan hanya memiliki akar-akar imajiner.

15	except ValueError: print("\nInput tidak valid!...")	Menangani kondisi khusus jika pengguna memasukkan input bukan angka. Python akan melempar ValueError yang ditangkap di sini, lalu program mencetak pesan error tanpa berhenti secara paksa.
16	if __name__ == "__main__": hitung_akar_kuadrat()	Memastikan fungsi utama hanya dijalankan ketika file ini dieksekusi secara langsung, bukan ketika diimpor sebagai modul oleh file lain.

5.4 Program 4: Pengolahan NIP ASN

5.4.1 Kode R

No	Baris Kode R	Penjelasan Logika
1	# Program Menentukan Tanggal Lahir dari NIP ASN	Merupakan komentar yang menjelaskan tujuan program, yaitu menentukan tanggal lahir berdasarkan NIP ASN. Baris ini tidak dijalankan oleh R
2	nip <- "200002292023011001"	Membuat objek nip sebagai input yang berisi nomor NIP ASN yang akan diproses untuk memperoleh informasi tanggal lahir
3	if (nchar(nip) >= 8 && !grepl("\\D", nip)) { tahun <- substr(nip, 1, 4) bulan_digit <- substr(nip, 5, 6) tanggal <- substr(nip, 7, 8) }	Melakukan validasi bahwa NIP minimal 8 digit dan hanya terdiri dari angka. Jika valid, program mengambil 4 digit pertama sebagai tahun, digit ke-5 dan ke-6 sebagai kode bulan, serta digit ke-7 dan ke-8 sebagai tanggal lahir
4	objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-%m-%d")	Mencoba mengubah gabungan string teks tanggal menjadi objek tipe Date resmi standar kalender.
5	tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))	Mengambil angka tahun berjalan saat ini secara waktu riil dari sistem operasi komputer.

6	if (is.na(objek_tanggal)) { cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!\n")	Struktur percabangan tingkat dua untuk mendeteksi tanggal tidak logis (seperti kasus palsu 31 Februari)
7	} else if (as.numeric(tahun) > tahun_sekarang) { cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat ini (%d)!\n", tahun, tahun_sekarang))	Memblokir input jika tahun yang diekstrak dari data NIP melompati tahun berjalan saat ini
8	} else {	Blok eksekusi utama apabila seluruh kondisi jaring pengaman waktu di atas terbukti lolos validasi
9	if (bulan_digit == "01") { bulan <- "Januari" }	Memeriksa apakah kode bulan bernilai "01". Jika benar, variabel bulan diisi dengan nama bulan Januari
10	else if (bulan_digit == "02") { bulan <- "Februari" }	Memeriksa apakah kode bulan bernilai "02". Jika benar, variabel bulan diisi dengan nama bulan Februari
11	else if (bulan_digit == "03") { bulan <- "Maret" }	Memeriksa apakah kode bulan bernilai "03". Jika benar, variabel bulan diisi dengan nama bulan Maret
12	else if (bulan_digit == "04") { bulan <- "April" }	Memeriksa apakah kode bulan bernilai "04". Jika benar, variabel bulan diisi dengan nama bulan April
13	else if (bulan_digit == "05") { bulan <- "Mei" }	Memeriksa apakah kode bulan bernilai "05". Jika benar, variabel bulan diisi dengan nama bulan Mei
14	else if (bulan_digit == "06") { bulan <- "Juni" }	Memeriksa apakah kode bulan bernilai "06". Jika benar, variabel bulan diisi dengan nama bulan Juni
15	else if (bulan_digit == "07") { bulan <- "Juli" }	Memeriksa apakah kode bulan bernilai "07". Jika benar, variabel bulan diisi dengan nama bulan Juli

16	<code>else if (bulan_digit == "08") { bulan <- "Agustus" }</code>	Memeriksa apakah kode bulan bernilai "08". Jika benar, variabel bulan diisi dengan nama bulan Agustus
17	<code>else if (bulan_digit == "09") { bulan <- "September" }</code>	Memeriksa apakah kode bulan bernilai "09". Jika benar, variabel bulan diisi dengan nama bulan September
18	<code>else if (bulan_digit == "10") { bulan <- "Oktober" }</code>	Memeriksa apakah kode bulan bernilai "10". Jika benar, variabel bulan diisi dengan nama bulan Oktober
19	<code>else if (bulan_digit == "11") { bulan <- "November" }</code>	Memeriksa apakah kode bulan bernilai "11". Jika benar, variabel bulan diisi dengan nama bulan November
20	<code>else if (bulan_digit == "12") { bulan <- "Desember" }</code>	Memeriksa apakah kode bulan bernilai "12". Jika benar, variabel bulan diisi dengan nama bulan Desember
21	<code>else { bulan <- "Tidak Valid" }</code>	Menentukan nilai bulan menjadi "Tidak Valid" apabila kode bulan pada NIP tidak termasuk dalam rentang 01 sampai 12
22	<code>cat(paste("NIP:", nip, "\n"))cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))</code>	Mencetak kompilasi akhir riwayat data NIP serta teks susunan tanggal lahir lengkap ke konsol
23	<code>}</code>	Penutup blok instruksi bersarang aman
24	<code>} else { cat("Error: NIP harus berupa angka dan minimal 8 digit.\n") }</code>	Pesan kegagalan paling luar jika kondisi format digit panjang NIP di awal tidak terpenuhi

5.4.2 Kode Python

No	Baris Kode Python	Penjelasan Logika
1	<code>from datetime import datetime</code>	Mengimpor modul atau pustaka internal Python yang berfungsi untuk memproses manipulasi objek tanggal dan waktu
2	<code>nip = "200002292023011001"</code>	Menginisialisasi variabel nip sebagai string teks objek uji coba utama
3	<code>if len(nip) >= 8 and nip.isdigit(): tahun = nip[:4] bulan_digit = nip[4:6] tanggal = nip[6:8]</code>	Memeriksa apakah NIP memiliki minimal 8 digit dan seluruh karakter berupa angka. Jika valid, program mengambil 4 digit pertama sebagai tahun, 2 digit berikutnya sebagai bulan, dan 2 digit setelahnya sebagai tanggal lahir.
4	<code>try:</code>	Membuka blok exception handling (jaring pengaman) untuk mencoba mengeksekusi pemeriksaan kalender
5	<code>objek_tanggal = datetime.strptime(...)</code>	Menggunakan fungsi <code>strptime</code> untuk mengonversi string teks menjadi objek tanggal resmi sekaligus memverifikasi keabsahan kalendernya
6	<code>tahun_sekarang = datetime.now().year</code>	Mengambil nilai angka tahun berjalan saat ini secara waktu riil langsung dari sistem komputer
7	<code>if int(tahun) > tahun_sekarang:</code>	Melakukan percabangan untuk memeriksa apakah tahun lahir hasil ekstrak melebihi batas tahun berjalan saat ini (tahun masa depan)
8	<code>print(f'Error: Tahun lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang})!') kalender_valid = False</code>	Mencetak pesan error jika terdeteksi tahun futuristik (seperti 2030) dan mematikan status validitas kalender
9	<code>else: kalender_valid = True</code>	Menyatakan status kalender sepenuhnya valid jika tahun lahir logis berada di masa lampau atau masa kini

10	<code>except ValueError:</code>	Blok penyelamat yang otomatis menangkap error jika fungsi <code>strptime</code> mendeteksi tanggal palsu yang tidak eksis di kalender
11	<code>print("Error: Kombinasi tanggal dan bulan lahir yang Anda masukkan tidak valid") kalender_valid = False</code>	Mencetak pesan error spesifik untuk kasus anomali kalender (seperti 31 Februari) dan mematikan status validitas
12	<code>if kalender_valid:</code>	Membuka gerbang logika utama untuk memproses data jika seluruh filter jaring pengaman di atas berhasil lolos
13	<code>if bulan_digit == "01": bulan = "Januari"</code>	Memeriksa apakah kode bulan bernilai "01". Jika benar, variabel bulan diisi dengan nama bulan Januari
14	<code>elif bulan_digit == "02": bulan = "Februari"</code>	Memeriksa apakah kode bulan bernilai "02". Jika benar, variabel bulan diisi dengan nama bulan Februari
15	<code>elif bulan_digit == "03": bulan = "Maret"</code>	Memeriksa apakah kode bulan bernilai "03". Jika benar, variabel bulan diisi dengan nama bulan Maret
16	<code>elif bulan_digit == "04": bulan = "April"</code>	Memeriksa apakah kode bulan bernilai "04". Jika benar, variabel bulan diisi dengan nama bulan April
17	<code>elif bulan_digit == "05": bulan = "Mei"</code>	Memeriksa apakah kode bulan bernilai "05". Jika benar, variabel bulan diisi dengan nama bulan Mei
18	<code>elif bulan_digit == "06": bulan = "Juni"</code>	Memeriksa apakah kode bulan bernilai "06". Jika benar, variabel bulan diisi dengan nama bulan Juni
19	<code>elif bulan_digit == "07": bulan = "Juli"</code>	Memeriksa apakah kode bulan bernilai "07". Jika benar, variabel bulan diisi dengan nama bulan Juli
20	<code>elif bulan_digit == "08": bulan = "Agustus"</code>	Memeriksa apakah kode bulan bernilai "08". Jika benar, variabel bulan diisi dengan nama bulan Agustus

21	<code>elif bulan_digit == "09": bulan = "September"</code>	Memeriksa apakah kode bulan bernilai "09". Jika benar, variabel bulan diisi dengan nama bulan September
22	<code>elif bulan_digit == "10": bulan = "Oktober"</code>	Memeriksa apakah kode bulan bernilai "10". Jika benar, variabel bulan diisi dengan nama bulan Oktober
23	<code>elif bulan_digit == "11": bulan = "November"</code>	Memeriksa apakah kode bulan bernilai "11". Jika benar, variabel bulan diisi dengan nama bulan November
24	<code>elif bulan_digit == "12": bulan = "Desember"</code>	Memeriksa apakah kode bulan bernilai "12". Jika benar, variabel bulan diisi dengan nama bulan Desember
25	<code>else: bulan = "Tidak Valid"</code>	Menentukan nilai bulan menjadi "Tidak Valid" apabila kode bulan pada NIP tidak termasuk dalam rentang 01 sampai 12
26	<code>print(f"NIP: {nip}") print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")</code>	Mencetak output final berupa kompilasi informasi NIP dan hasil teks format tanggal lahir lengkap ke layar konsol
27	<code>else: print("Error: NIP harus berupa angka dan minimal 8 digit.")</code>	Pesan kegagalan paling luar jika sejak awal panjang karakter NIP kurang dari 8 atau mengandung huruf

5.5 Program 5: Klasifikasi Cluster 3 Dimensi

5.5.1 Kode R

No	Baris Kode R	Penjelasan Logika
1	<code># 1. Mendefinisikan Fungsi (Syarat Wajib Modul 8)</code>	Merupakan komentar yang menjelaskan bahwa bagian berikutnya digunakan untuk membuat fungsi perhitungan jarak Euclidean antar dua titik dalam ruang tiga dimensi.

2	hitung_jarak <- function(titik1, titik2) { jarak <- sqrt((titik1[1]- titik2[1])^2 + (titik1[2]- titik2[2])^2 + (titik1[3]- titik2[3])^2) return(jarak) }	Mendefinisikan fungsi hitung_jarak untuk menghitung jarak Euclidean antara dua titik tiga dimensi. Fungsi menerima dua vektor sebagai input dan mengembalikan hasil berupa nilai jarak.
3	# R menggunakan indeks basis 1, bukan 0	Merupakan komentar yang menjelaskan bahwa R menggunakan indeks dimulai dari 1 saat mengakses elemen vektor.
4	A <- c(2, 1, 3)	Membuat vektor titik pusat Cluster A dengan koordinat (2, 1, 3) yang akan digunakan sebagai salah satu acuan dalam proses klasifikasi titik U.
5	B <- c(1, -4, 6)	Membuat vektor titik pusat Cluster B dengan koordinat (1, -4, 6) yang akan digunakan sebagai salah satu acuan dalam proses klasifikasi titik U.
6	C <- c(-2, 3, -2)	Membuat vektor titik pusat Cluster C dengan koordinat (-2, 3, -2) yang akan digunakan sebagai salah satu acuan dalam proses klasifikasi titik U.
7	cat("--- Klasifikasi Cluster 3 Dimensi ---\n")	Menampilkan judul program sebagai informasi bahwa program melakukan klasifikasi titik pada ruang tiga dimensi.
8	cat("Masukkan koordinat titik U(x, y, z):\n")	Menampilkan informasi mengenai koordinat titik U yang akan dimasukkan dan diproses dalam program.

9	<code>x <- 2, y <- -3, z <- 6</code>	Menyimpan nilai koordinat titik U sebagai data input yang akan dibandingkan dengan titik pusat setiap cluster.
10	<code>if (!is.na(x) && !is.na(y) && !is.na(z)) { U <- c(x, y, z) }</code>	Melakukan validasi agar nilai koordinat tidak kosong atau tidak valid. Jika memenuhi syarat, nilai x, y, dan z disimpan ke dalam vektor U sebagai titik yang akan diklasifikasikan.
11	<code>jarak_A <- hitung_jarak(U, A)</code>	Memanggil fungsi <code>hitung_jarak</code> untuk menghitung jarak antara titik U dengan pusat Cluster A.
12	<code>jarak_B <- hitung_jarak(U, B)</code>	Memanggil fungsi <code>hitung_jarak</code> untuk menghitung jarak antara titik U dengan pusat Cluster B.
13	<code>jarak_C <- hitung_jarak(U, C)</code>	Memanggil fungsi <code>hitung_jarak</code> untuk menghitung jarak antara titik U dengan pusat Cluster C.
14	<code>if (jarak_A < jarak_B && jarak_A < jarak_C) { cluster <- "Cluster A" }</code>	Memeriksa apakah jarak titik U ke Cluster A merupakan jarak paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster A.
15	<code>else if (jarak_B < jarak_A && jarak_B < jarak_C) { cluster <- "Cluster B" }</code>	Memeriksa apakah jarak titik U ke Cluster B merupakan jarak paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster B.
16	<code>else if (jarak_C < jarak_A && jarak_C < jarak_B) { cluster <- "Cluster C" }</code>	Memeriksa apakah jarak titik U ke Cluster C merupakan jarak paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster C.

17	<code>else { cluster <- "Jarak Sama (Tidak Dominan)" }</code>	Menentukan hasil klasifikasi jika terdapat dua atau lebih jarak yang sama sehingga tidak ada satu cluster yang paling dekat.
18	<code>cat(sprintf("\nJarak titik U ke Pusat A : %.3f\n", jarak_A))</code>	Menampilkan hasil jarak antara titik U dengan pusat Cluster A dengan format tiga angka di belakang koma.
19	<code>cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))</code>	Menampilkan hasil jarak antara titik U dengan pusat Cluster B dengan format tiga angka di belakang koma.
20	<code>cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))</code>	Menampilkan hasil jarak antara titik U dengan pusat Cluster C dengan format tiga angka di belakang koma.
21	<code>cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))</code>	Menampilkan hasil akhir berupa informasi cluster yang menjadi anggota dari titik U berdasarkan jarak terdekat.
22	<code>else { cat("Error: Input koordinat harus berupa angka!\n") }</code>	Menampilkan pesan kesalahan apabila koordinat yang dimasukkan tidak valid atau bukan berupa angka.

5.5.2 Kode Python

No	Baris Kode Python	Penjelasan Logika
1	<code>import math</code>	Mengimpor modul <code>math</code> yang digunakan untuk menjalankan fungsi matematika, khususnya fungsi <code>sqrt()</code> dalam perhitungan jarak Euclidean tiga dimensi.
2	<code># 1. Mendefinisikan Fungsi (Syarat Wajib Modul 8)</code>	Merupakan komentar yang menjelaskan bahwa bagian berikutnya digunakan untuk membuat fungsi perhitungan jarak Euclidean antara dua titik.
3	<code>def hitung_jarak(titik1, titik2): jarak = math.sqrt((titik1[0]-titik2[0])**2 + (titik1[1]-titik2[1])**2 + (titik1[2]-titik2[2])**2) return jarak</code>	Mendefinisikan fungsi <code>hitung_jarak</code> untuk menghitung jarak Euclidean antara dua titik tiga dimensi. Fungsi menerima dua buah tuple sebagai input dan mengembalikan nilai jarak sebagai hasil perhitungan.
4	<code># Rumus jarak Euclidean 3D</code>	Merupakan komentar yang menjelaskan bahwa perhitungan pada fungsi menggunakan rumus jarak Euclidean dalam ruang tiga dimensi.
5	<code>A = (2, 1, 3)</code>	Membuat tuple titik pusat Cluster A dengan koordinat (2, 1, 3) sebagai acuan dalam proses klasifikasi titik U.
6	<code>B = (1, -4, 6)</code>	Membuat tuple titik pusat Cluster B dengan koordinat (1, -4, 6) sebagai acuan dalam proses klasifikasi titik U.
7	<code>C = (-2, 3, -2)</code>	Membuat tuple titik pusat Cluster C dengan koordinat (-2, 3, -2) sebagai acuan dalam proses klasifikasi titik U.

8	<code>print("--- Klasifikasi Cluster 3 Dimensi ---")</code>	Menampilkan judul program sebagai informasi bahwa program melakukan klasifikasi titik dalam ruang tiga dimensi.
9	<code># MASUKKAN CONTOH INPUT</code>	Merupakan komentar yang menjelaskan bahwa bagian berikutnya berisi contoh nilai koordinat titik U yang akan diuji.
10	<code>x = 2, y = -3, z = 6</code>	Menyimpan nilai koordinat titik U sebagai data input yang akan dibandingkan dengan setiap pusat cluster.
11	<code>print(f'Koordinat Titik U yang Diuji -> x: {x}, y: {y}, z: {z}')</code>	Menampilkan informasi koordinat titik U yang digunakan sebagai data pengujian.
12	<code>U = (x, y, z)</code>	Menyimpan nilai x, y, dan z menjadi sebuah tuple U sebagai titik yang akan diklasifikasikan.
13	<code>jarak_A = hitung_jarak(U, A)</code>	Menghitung jarak antara titik U dengan pusat Cluster A menggunakan fungsi <code>hitung_jarak</code> .
14	<code>jarak_B = hitung_jarak(U, B)</code>	Menghitung jarak antara titik U dengan pusat Cluster B menggunakan fungsi <code>hitung_jarak</code> .
15	<code>jarak_C = hitung_jarak(U, C)</code>	Menghitung jarak antara titik U dengan pusat Cluster C menggunakan fungsi <code>hitung_jarak</code> .
16	<code>if jarak_A < jarak_B and jarak_A < jarak_C: cluster = "Cluster A"</code>	Memeriksa apakah jarak titik U ke Cluster A merupakan yang paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster A.

17	<code>elif jarak_B < jarak_A and jarak_B < jarak_C: cluster = "Cluster B"</code>	Memeriksa apakah jarak titik U ke Cluster B merupakan yang paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster B.
18	<code>elif jarak_C < jarak_A and jarak_C < jarak_B: cluster = "Cluster C"</code>	Memeriksa apakah jarak titik U ke Cluster C merupakan yang paling kecil. Jika benar, titik U diklasifikasikan sebagai anggota Cluster C.
19	<code>else: cluster = "Jarak Sama (Tidak Dominan)"</code>	Menentukan hasil klasifikasi apabila terdapat dua atau lebih jarak yang sama sehingga tidak ada cluster yang lebih dominan.
20	<code>print(f"\nJarak titik U ke Pusat A : {jarak_A:.3f}")</code>	Menampilkan hasil jarak titik U terhadap pusat Cluster A dengan format tiga angka di belakang koma.
21	<code>print(f"Jarak titik U ke Pusat B : {jarak_B:.3f}")</code>	Menampilkan hasil jarak titik U terhadap pusat Cluster B dengan format tiga angka di belakang koma.
22	<code>print(f"Jarak titik U ke Pusat C : {jarak_C:.3f}")</code>	Menampilkan hasil jarak titik U terhadap pusat Cluster C dengan format tiga angka di belakang koma.
23	<code>print(f">> Titik U digolongkan sebagai anggota: {cluster}")</code>	Menampilkan hasil akhir berupa informasi cluster tempat titik U digolongkan berdasarkan jarak terdekat.

5.6 Program 6: Interval Konfidensi Proporsi

5.6.1 Kode R

No	Baris Kode R	Penjelasan Logika
----	--------------	-------------------

1	<pre>interval_kepercayaan <- function(p_hat, n, alpha) {</pre>	Mendefinisikan fungsi utama bernama interval_kepercayaan dengan tiga parameter, yaitu proporsi sampel, ukuran sampel, dan tingkat signifikansi, sesuai ketentuan soal yang mewajibkan penggunaan fungsi.
2	<pre>if (is.na(p_hat) is.na(n) is.na(alpha)) { cat("Error: Input tidak boleh kosong...\n") return(invisible(NULL)) }</pre>	Melakukan validasi input. Jika salah satu nilai bernilai NA karena kosong atau bukan angka, program mencetak pesan error lalu menghentikan eksekusi menggunakan return(invisible(NULL)) agar tidak ada nilai kosong yang tercetak ke layar.
3	<pre>if (p_hat < 0 p_hat > 1) { cat("Error: Proporsi harus berada antara 0 dan 1.\n") return(invisible(NULL)) }</pre>	Memvalidasi batasan masalah sesuai soal, yaitu proporsi sampel harus berada pada rentang 0 sampai 1. Jika nilai di luar rentang tersebut, program mencetak pesan error dan eksekusi dihentikan.
4	<pre>if (alpha == 0.10) { z <- 1.645 } else if (alpha == 0.05) { z <- 1.96 } else { cat("Error: Nilai alpha hanya boleh 0.10 atau 0.05.\n") return(invisible(NULL)) }</pre>	Menentukan nilai z berdasarkan tingkat kepercayaan yang dipilih pengguna. Jika alpha bernilai 0.10 maka z diisi 1.645, jika 0.05 maka z diisi 1.96, dan jika nilai alpha tidak sesuai kedua pilihan tersebut, program mencetak pesan error.
5	<pre>se <- sqrt((p_hat * (1 - p_hat)) / n)</pre>	Menghitung nilai standard error (SE) dari proporsi sampel menggunakan rumus yang sesuai dengan ketentuan soal interval konfidensi.
6	<pre>batas_bawah <- p_hat - z * se batas_atas <- p_hat + z * se</pre>	Menghitung batas bawah dan batas atas interval konfidensi dengan mengurangi dan menambahkan hasil kali z dan standard error terhadap proporsi sampel.

7	<code>cat(sprintf("\nInterval Kepercayaan %.0f%%\n", (1 - alpha) * 100))</code>	Menampilkan judul hasil berupa tingkat kepercayaan dalam bentuk persen, dihitung dari $(1 - \alpha)$ dikali 100.
8	<code>cat(sprintf("%.4f < p < %.4f\n", batas_bawah, batas_atas))</code>	Menampilkan hasil akhir interval konfidensi ke layar dengan format presisi empat desimal agar nilai batas bawah dan batas atas terlihat jelas.
9	<code>p_hat <- as.numeric(readline("Masukkan nilai proporsi sampel (p_hat): "))</code>	Membaca proporsi sampel dari pengguna dan langsung mengonversinya ke tipe numerik agar dapat digunakan dalam perhitungan SE dan interval konfidensi.
10	<code>n <- as.integer(readline("Masukkan nilai n: "))</code>	Membaca ukuran sampel dari pengguna dan mengonversinya ke tipe integer karena nilai n merupakan banyaknya data, sehingga harus berupa bilangan bulat.
11	<code>alpha <- as.numeric(readline("Masukkan nilai alpha (0.10 atau 0.05): "))</code>	Membaca tingkat signifikansi dari pengguna dan mengonversinya ke tipe numerik agar dapat dibandingkan dengan nilai 0.10 atau 0.05 pada baris percabangan.
12	<code>interval_kepercayaan(p_hat, n, alpha)</code>	Memanggil dan menjalankan fungsi utama dengan memasukkan ketiga nilai yang sudah dibaca dari pengguna sebagai argumen.

5.6.2 Kode Python

No	Baris Kode Python	Penjelasan
1	<code>def hitung_interval(p_hat, n, alpha):</code>	Mendefinisikan fungsi untuk menghitung interval kepercayaan proporsi populasi.
2	<code>if alpha == 0.10:</code>	Mengecek apakah nilai alpha sebesar 0,10.
3	<code>z = 1.645</code>	Menentukan nilai z untuk tingkat kepercayaan 90%.
4	<code>elif alpha == 0.05:</code>	Mengecek apakah nilai alpha sebesar 0,05.

5	$z = 1.96$	Menentukan nilai z untuk tingkat kepercayaan 95%.
6	else:	Menangani kondisi jika α bukan 0,10 atau 0,05.
7	print("Error...")	Menampilkan pesan kesalahan apabila α tidak valid.
8	return	Menghentikan fungsi jika terjadi kesalahan input α .
9	$se = ((p_hat * (1 - p_hat)) / n) ** 0.5$	Menghitung standar error (SE) proporsi sampel menggunakan rumus interval kepercayaan proporsi.
10	$batas_bawah = p_hat - z * se$	Menghitung batas bawah interval kepercayaan.
11	$batas_atas = p_hat + z * se$	Menghitung batas atas interval kepercayaan.
12	print(...)	Menampilkan tingkat kepercayaan yang digunakan.
13	print(...)	Menampilkan hasil interval kepercayaan.
14	$p_hat = float(input(...))$	Meminta pengguna memasukkan nilai proporsi sampel.
15	$n = int(input(...))$	Meminta pengguna memasukkan ukuran sampel.
16	$\alpha = float(input(...))$	Meminta pengguna memasukkan nilai α .
17	if $p_hat < 0$ or $p_hat > 1$:	Memvalidasi bahwa proporsi harus berada di antara 0 dan 1.
18	print("Error...")	Menampilkan pesan kesalahan jika proporsi tidak valid.
19	else:	Jika proporsi valid maka program dilanjutkan.
20	hitung_interval(p_hat, n, α)	Memanggil fungsi untuk menghitung interval kepercayaan.

BAB VI

PENGUJIAN PROGRAM

6.1 Program 1: Analisis Teks

6.1.1 Pengujian Program

Skenario Uji	Input	Output yang Diharapkan	Output Program (Python)	Output Program (R)	Status	Catatan
Pengujian satu kalimat	Saya sedang belajar Python.	Teks tersebut memuat 1 kalimat dan 4 kata	Teks tersebut memuat 1 kalimat dan 4 kata	Teks tersebut memuat 1 kalimat dan 4 kata	Sukses	Program dapat menghitung satu kalimat dengan benar.
Pengujian beberapa kalimat pendek	Saya makan nasi. Saya minum air. Saya bermain bola.	Teks tersebut memuat 3 kalimat dan 9 kata	Teks tersebut memuat 3 kalimat dan 9 kata	Teks tersebut memuat 3 kalimat dan 9 kata	Sukses	Program dapat menghitung lebih dari satu kalimat.
Pengujian tanpa tanda titik	Saya sedang belajar Python	Teks tersebut memuat 0 kalimat dan 4 kata	Teks tersebut memuat 0 kalimat dan 4 kata	Teks tersebut memuat 0 kalimat dan 4 kata	Sukses	Program mengikuti asumsi bahwa kalimat harus diakhiri titik.
Pengujian teks kosong	""	Teks tersebut memuat 0 kalimat dan 0 kata	Teks tersebut memuat 0 kalimat dan 0 kata	Teks tersebut memuat 0 kalimat dan 0 kata	Sukses	Program tetap berjalan meskipun input kosong.

6.1.2 Screenshot Hasil Running

```

1 teks <- "Saya sedang belajar Python."
2
3 jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))
4
5 teks_tanpa_titik <- gsub("\\.", "", teks)
6
7 kata <- unlist(strsplit(teks_tanpa_titik, "\\s+"))
8
9 jumlah_kata <- length(kata)
10
11 cat("Jumlah kata =", jumlah_kata, "\n")
12 cat("Jumlah kalimat =", jumlah_kalimat, "\n")

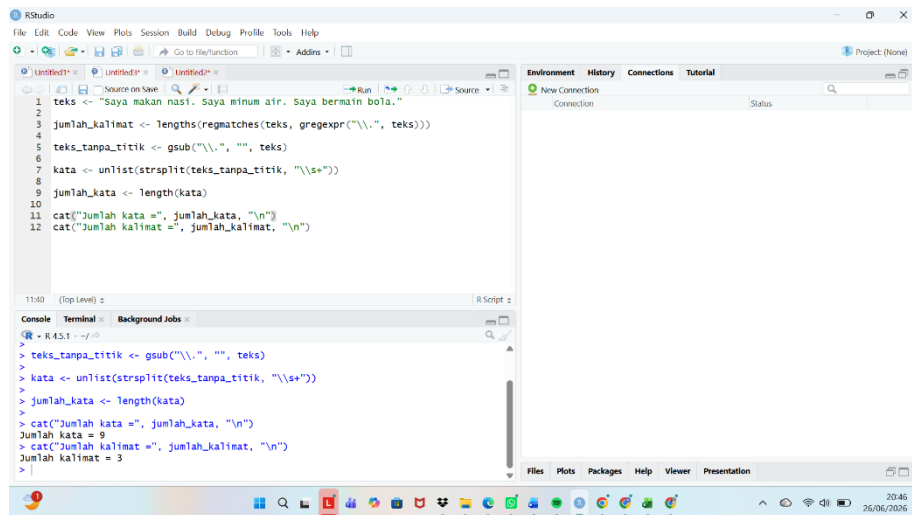
```

```

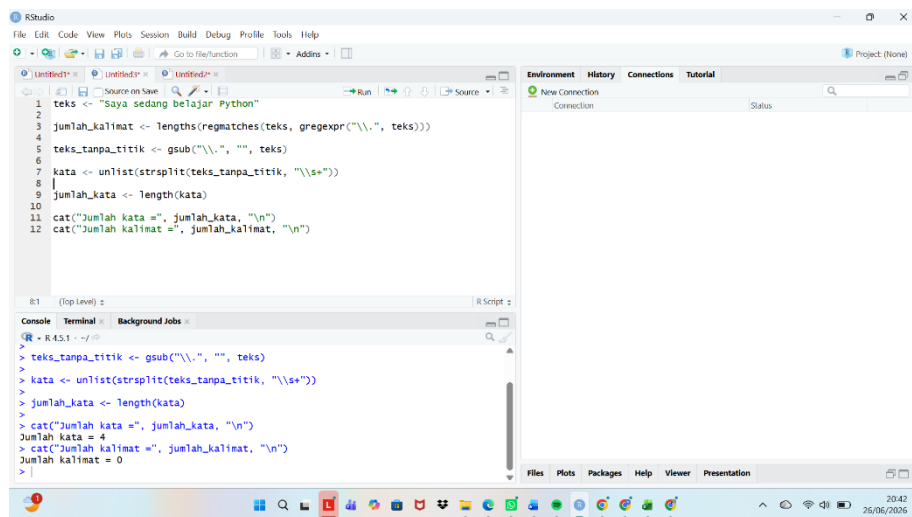
> teks_tanpa_titik <- gsub("\\.", "", teks)
> kata <- unlist(strsplit(teks_tanpa_titik, "\\s+"))
> jumlah_kata <- length(kata)
>
> cat("Jumlah kata =", jumlah_kata, "\n")
Jumlah kata = 4
> cat("Jumlah kalimat =", jumlah_kalimat, "\n")
Jumlah kalimat = 1
>

```

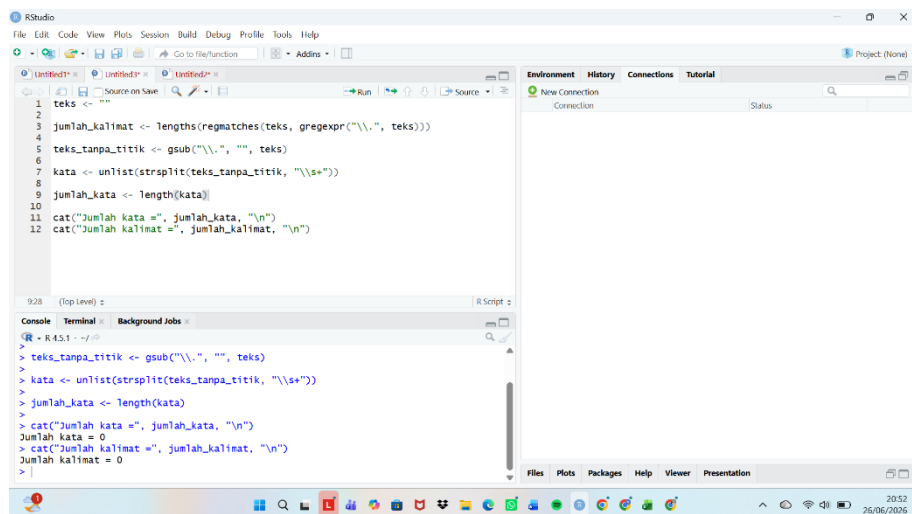
Gambar 6.1 Screenshot hasil pengujian Program 1 Skenario 1 (R)



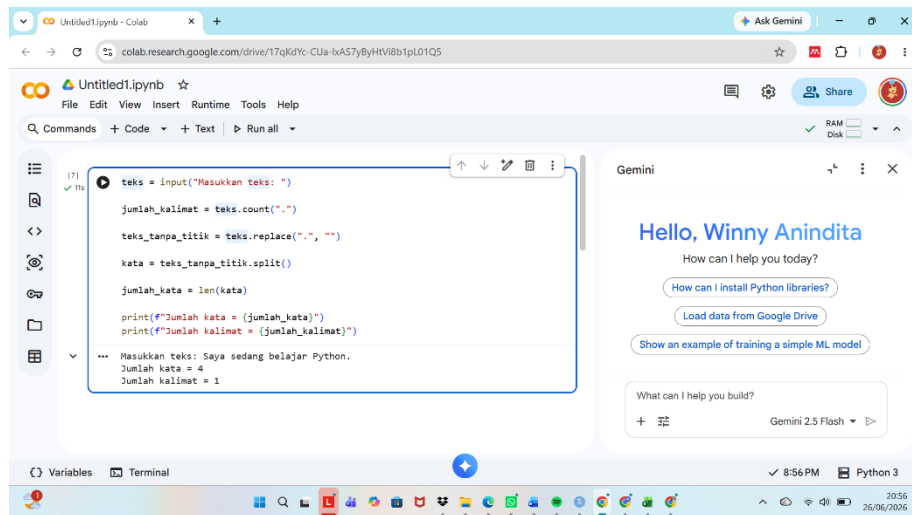
Gambar 6.2 Screenshot Hasil Pengujian Program 1 Skenario 2 (R)



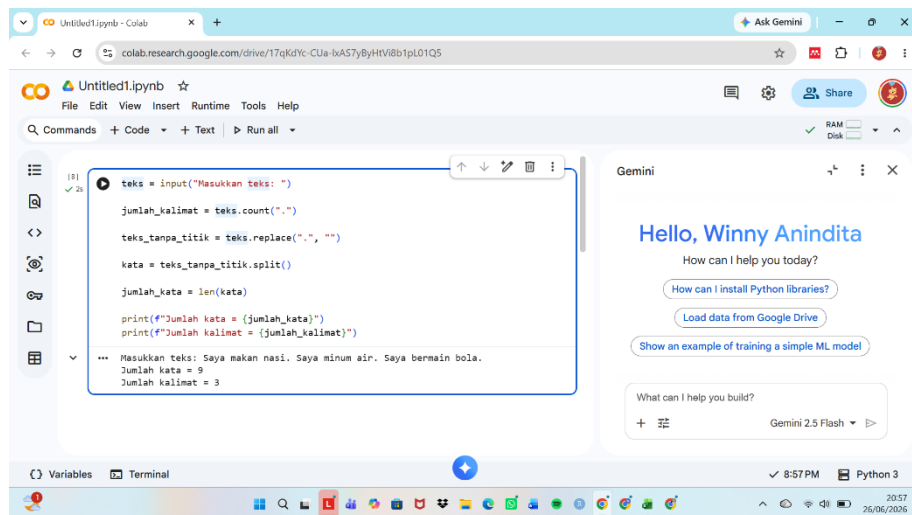
Gambar 6.3 Screenshot Hasil Pengujian Program 1 Skenario 3 (R)



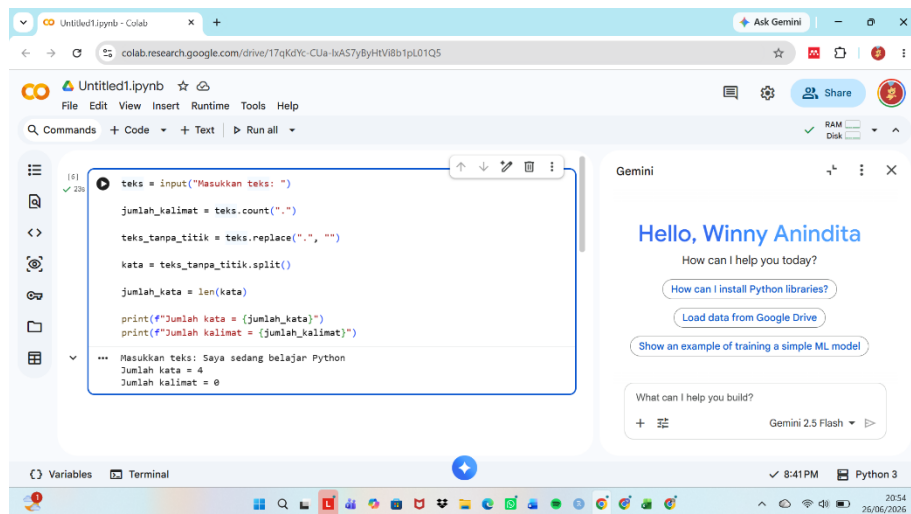
Gambar 6.4 Screenshot Hasil Pengujian Program 1 Skenario 4 (R)



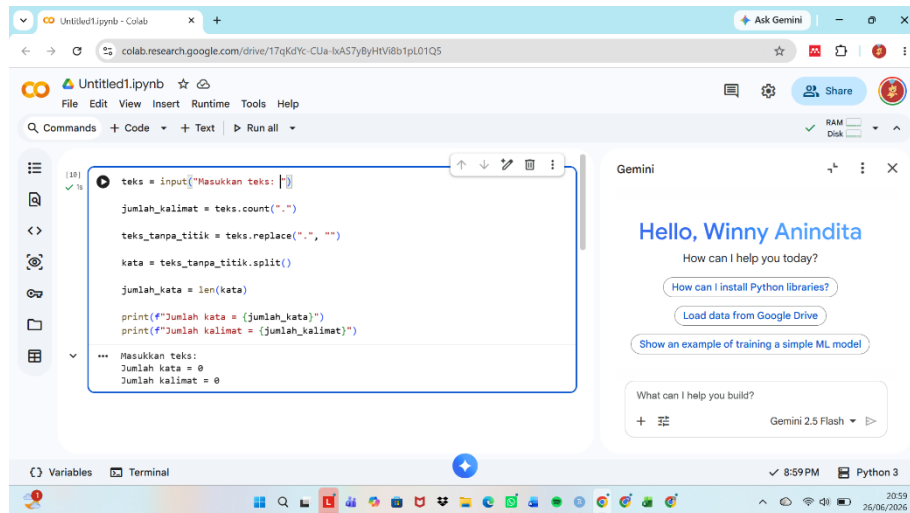
Gambar 6.5 Screenshot Hasil Pengujian Program 1 Skenario 1 (Python)



Gambar 6.6 Screenshot Hasil Pengujian Program 1 Skenario 2 (Python)



Gambar 6.7 Screenshot Hasil Pengujian Program 1 Skenario 3 (Python)



Gambar 6.8 Screenshot Hasil Pengujian Program 1 Skenario 4 (Python)

6.1.3 Interpretasi Output

1. Skenario Uji Kasus Ideal (Normal Case — Input: "Saya sedang belajar Python.")
Interpretasi: Hasil eksekusi terhadap input kalimat tunggal yang terstruktur secara gramatikal membuktikan bahwa algoritma pemisahan teks (text splitting) berbasis tanda baca titik bekerja dengan akurasi tinggi. Sistem berhasil mengenali satu tanda titik sebagai pembatas akhir kalimat, sehingga teks tersebut terhitung sebagai 1 kalimat. Proses tokenisasi yang memecah teks berdasarkan spasi juga berhasil menghitung jumlah kata secara tepat, yaitu 4 kata. Baik program Python maupun R menghasilkan output yang identik, yaitu "Teks tersebut memuat 1 kalimat dan 4 kata", yang menunjukkan konsistensi logika perhitungan di antara kedua bahasa pemrograman tersebut.
2. Skenario Uji Kasus Kalimat Majemuk (Normal Case — Input: "Saya makan nasi. Saya minum air. Saya bermain bola.")
Interpretasi: Pada pengujian dengan input berupa beberapa kalimat pendek yang digabung dalam satu teks, program berhasil mendeteksi tiga tanda titik sebagai penanda batas antar kalimat, sehingga sistem secara akurat menghitung total 3 kalimat. Algoritma penghitung kata yang bekerja secara independen dari proses pemisahan kalimat juga berhasil menjumlahkan seluruh kata dari ketiga kalimat tersebut tanpa terpengaruh oleh keberadaan tanda baca, menghasilkan total 9 kata. Keberhasilan ini membuktikan bahwa program mampu menangani teks dengan

lebih dari satu kalimat secara konsisten, tidak hanya terbatas pada kasus kalimat tunggal.

3. Skenario Uji Kasus Tanpa Tanda Baca (Boundary Case — Input: "Saya sedang belajar Python")

Interpretasi: Pengujian ini menggunakan input kalimat yang secara sengaja dihilangkan tanda titik di akhirnya, untuk menguji ketergantungan algoritma terhadap tanda baca sebagai penentu batas kalimat. Hasilnya, baik program Python maupun R sama-sama menghitung 0 kalimat, meskipun teks tersebut tetap mengandung 4 kata yang berhasil dihitung dengan benar. Hal ini menunjukkan bahwa logika program dirancang dengan asumsi ketat (strict assumption) bahwa sebuah kalimat hanya dianggap valid apabila diakhiri dengan tanda titik, sehingga program tidak melakukan penalaran semantik tambahan untuk menyimpulkan keberadaan kalimat dari konteks kata semata.

4. Skenario Uji Kasus Input Kosong (Edge Case / Error Handling — Input: "")

Interpretasi: Pengujian batas ekstrem dilakukan dengan memberikan input berupa string kosong (empty string) untuk menguji ketahanan program terhadap kondisi data minimum. Hasil eksekusi menunjukkan bahwa program tidak mengalami crash maupun exception error, melainkan tetap berjalan secara normal dan menghasilkan output "Teks tersebut memuat 0 kalimat dan 0 kata" pada kedua bahasa pemrograman. Hal ini membuktikan bahwa struktur logika program telah memperhitungkan kondisi tepi (edge case) berupa input tanpa konten sama sekali, sehingga aplikasi tetap stabil (robust) dan tidak memerlukan mekanisme penyaring tambahan untuk menangani input kosong.

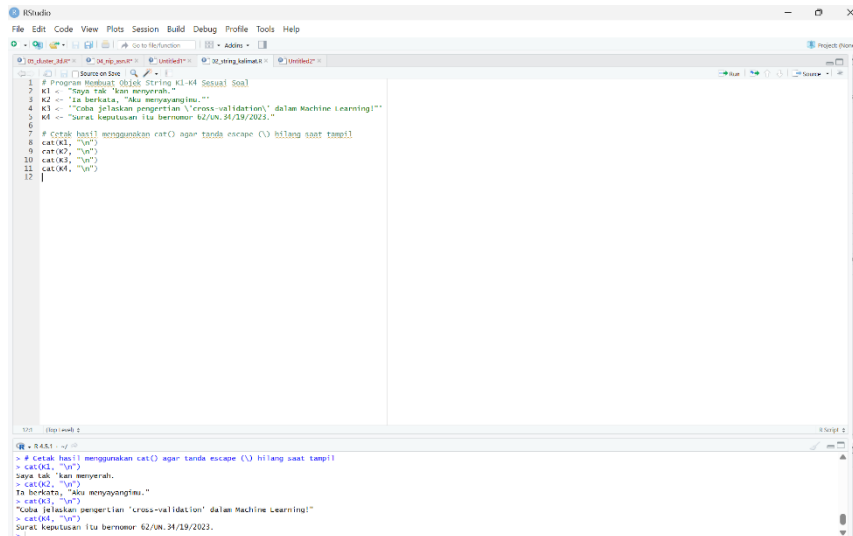
6.2 Program 2: String Kalimat

6.2.1 Tabel Pengujian

Skenario Uji	Skenario Uji2	Output yang Diharapkan	Output Program (Python)	Output Program (R)	Status	Catatan
Pengujian nilai string sesuai ketentuan soal	K1 = "Saya tak 'kan menyerah. "K2 = 'Ia "	Saya tak 'kan menyerah. Ia berkata, "Aku menyayangimu." "	Saya tak 'kan menyerah. Ia berkata, "Aku menyayangimu." "	Saya tak 'kan menyerah. Ia berkata, "Aku menyayangimu." "Coba jelaskan pengertian	Sukses	Program berhasil menampilkan seluruh string dengan format tanda petik yang sesuai.

	berkata, "Aku menyayangimu." K3 = ""Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"" K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.	"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.	'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.		
Pengujian salah satu string kosong	K1 = "" K2 = 'Ia berkata, "Aku menyayangimu." K3 = ""Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"" K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	(baris kosong) Ia berkata, "Aku menyayangimu." "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.	(baris kosong) Ia berkata, "Aku menyayangimu." "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.	(baris kosong) Ia berkata, "Aku menyayangimu." "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" Surat keputusan itu bernomor 62/UN.34/19/2023.	Sukses	Program tetap berjalan dengan baik meskipun terdapat variabel string kosong.
Pengujian kesalahan format string	K1 = "Saya tak 'kan menyerah (tanda petik penutup dihilangkan)"	Program menampilkan pesan syntax error dan tidak menghasilkan output	Muncul syntax error saat program dijalankan	Muncul syntax error saat program dijalankan	Sukses	Program mendeteksi kesalahan penulisan format string.

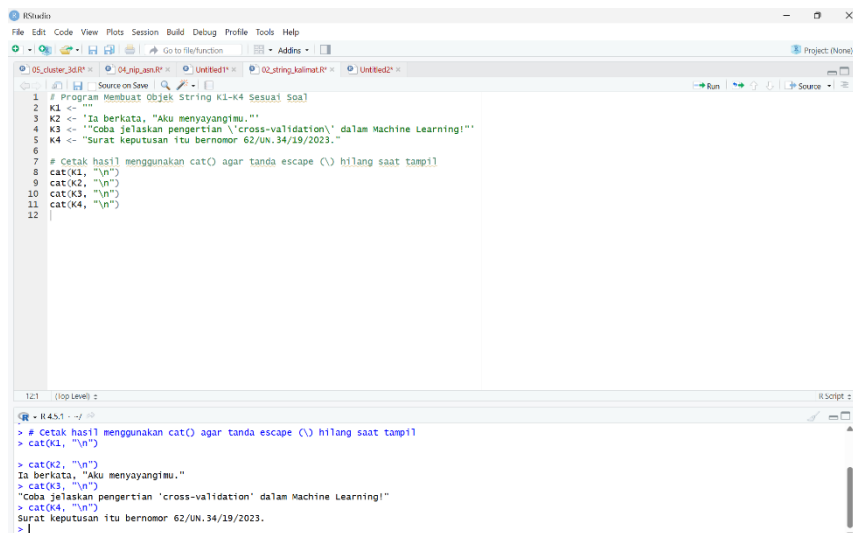
6.2.2 Screenshot Hasil Running



```
1 # Program Membuat Objek String K1-K4 Sesuai Soal
2 K1 <- "Saya tadi 'sue menyerah'"
3 K2 <- "Ia berkata, 'Aku menyayangimu.'"
4 K3 <- "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
5 K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
6
7 # cetak hasil menggunakan cat() agar tanda escape \n hilang saat tampil
8 cat(K1, "\n")
9 cat(K2, "\n")
10 cat(K3, "\n")
11 cat(K4, "\n")
12
```

```
> # cetak hasil menggunakan cat() agar tanda escape (\n) hilang saat tampil
> cat(K1, "\n")
Saya tadi 'sue menyerah'
> cat(K2, "\n")
Ia berkata, 'Aku menyayangimu.'
> cat(K3, "\n")
Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
>
```

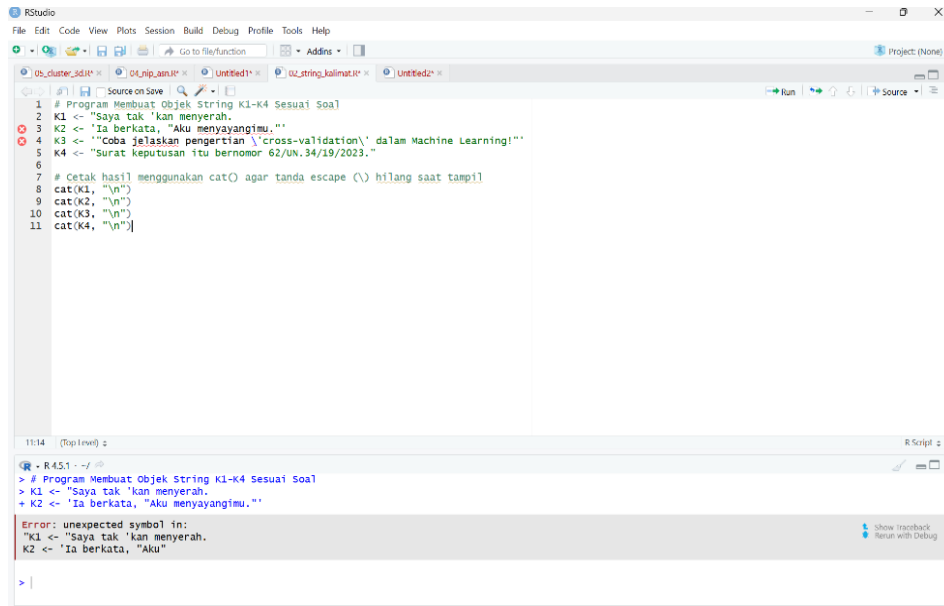
Gambar 6.9 Screenshot Hasil Pengujian Program 2 Skenario 1 (R)



```
1 # Program Membuat Objek String K1-K4 Sesuai Soal
2 K1 <- ""
3 K2 <- "Ia berkata, 'Aku menyayangimu.'"
4 K3 <- "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
5 K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
6
7 # cetak hasil menggunakan cat() agar tanda escape \n hilang saat tampil
8 cat(K1, "\n")
9 cat(K2, "\n")
10 cat(K3, "\n")
11 cat(K4, "\n")
12
```

```
> # cetak hasil menggunakan cat() agar tanda escape (\n) hilang saat tampil
> cat(K1, "\n")
Ia berkata, "Aku menyayangimu."
> cat(K2, "\n")
Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!
> cat(K3, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
>
```

Gambar 6.10 Screenshot Hasil Pengujian Program 2 Skenario 2 (R)



Gambar 6.11 *Screenshot* Hasil Pengujian Program 2 Skenario 3 (R)

```

# Program Membuat Objek String K1-K4 Sesuai Soal
K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)

```

Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.

Gambar 6.12 *Screenshot* Hasil Pengujian Program 2 Skenario 1 (Python)

```

# Program Membuat Objek String K1-K4 Sesuai Soal
K1 = ""
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)

```

Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.

Gambar 6.13 *Screenshot* Hasil Pengujian Program 2 Skenario 2 (Python)

```
# Program Membuat Objek String K1-K4 Sesuai Soal
K1 = "Saya tak 'kan menyerah."
K2 = 'Ia berkata, "Aku menyayangimu."'
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)
```

```
File "/tmp/ipykernel_3212/3588981018.py", line 2
    K1 = "Saya tak 'kan menyerah."
          ^
SyntaxError: unterminated string literal (detected at line 2)
```

Gambar 6.14 Screenshot Hasil Pengujian Program 2 Skenario 3 (Python)

6.2.3 Interpretasi Output

1. Objek String K1 (Saya tak 'kan menyerah.)

Interpretasi: Output menampilkan teks kalimat secara utuh termasuk tanda petik satu (') pada kata 'kan. Secara teknis pemrograman, komputer berhasil membaca dan mengisolasi tanda petik satu tersebut sebagai karakter teks biasa murni karena di dalam struktur kode, variabel pembungkus terluar sengaja menggunakan tanda petik dua ("...").

2. Objek String K2 (Ia berkata, "Aku menyayangimu.")

Interpretasi: Output berhasil menampilkan kalimat langsung yang ditandai dengan sepasang tanda petik dua ("..."). Komputer tidak mengalami kegagalan sintaks karena baris kode dikunci menggunakan pembungkus tanda petik satu ('...') di bagian terluar, sehingga tanda petik dua di dalam kalimat dianggap sebagai data teks biasa.

3. Objek String K3 ("Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!")

Interpretasi: Output menampilkan kalimat kompleks yang menggabungkan struktur tanda petik dua di luar kalimat dan tanda petik satu di dalam kata 'cross-validation'. Keberhasilan pencetakan teks ini didasarkan pada penerapan fitur *escape character* menggunakan simbol garis miring terbalik (\) sebelum karakter tanda petik internal ditulis. Sistem komputer membaca simbol tersebut sebagai perintah untuk mengabaikan fungsi sintaks petik dan mencetaknya sebagai visual karakter literal biasa pada konsol.

4. Objek String K4 (Surat keputusan itu bernomor 62/UN.34/19/2023.)

Interpretasi: Output menampilkan kombinasi teks alfabet, karakter angka numerik, tanda baca titik, serta simbol garis miring (/) secara presisi. Komputer memperlakukan

seluruh komponen tersebut sebagai satu kesatuan data teks murni (string) yang statis tanpa melakukan operasi aritmatika pembagian pada simbol numerik tahun dan nomor surat.

6.3 Program 3: Akar Persamaan Kuadrat

6.3.1 Tabel Pengujian

Skenario Uji	Input	Output yang Diharapkan	Output Program (Python)	Output Program (RStudio)	Status	Catatan
(Normal: $D > 0$)	a = 1 b = -5 c = 6	Persamaan memiliki dua akar real yang berbeda: x1 = 3.000 x2 = 2.000	Persamaan memiliki dua akar real yang berbeda: x1 = 3.000 x2 = 2.000	Persamaan memiliki dua akar real yang berbeda: x1 = 3.000 x2 = 2.000	Sukses	Menguji perhitungan diskriminan > 0 dan format 3 desimal.
(Normal: $D < 0$)	a = 1 b = 2 c = 5	Persamaan tersebut hanya memiliki akar-akar imajiner.	Persamaan tersebut hanya memiliki akar-akar imajiner.	Persamaan tersebut hanya memiliki akar-akar imajiner.	Sukses	Menguji logika saat diskriminan kurang dari nol (akar imajiner).
(Tidak Valid: a = 0)	a = 0 b = 4 c = 2	Nilai 'a' tidak boleh 0. Ini bukan persamaan kuadrat.	Nilai 'a' tidak boleh 0. Ini bukan persamaan kuadrat.	Nilai 'a' tidak boleh 0. Ini bukan persamaan kuadrat.	Sukses	Menguji validasi awal program agar tidak error saat dibagi 0.

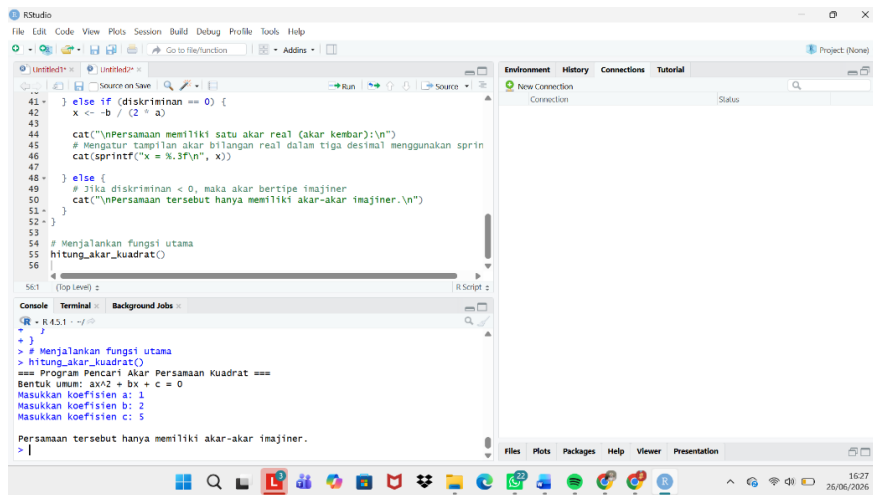
6.3.2 Screenshot Hasil *Running*

```

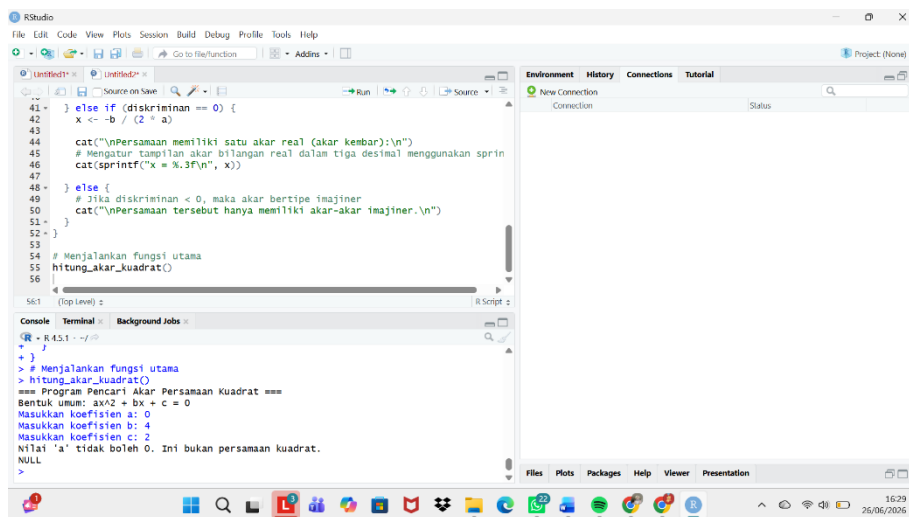
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
41 } else if (diskriminan == 0) {
42   x <- -b / (2 * a)
43   cat("\nPersamaan memiliki satu akar real (akar kembar):\n")
44   # Mengatur tampilan akar bilangan real dalam tiga desimal menggunakan sprin
45   cat(sprintf("x = %.3f\n", x))
46 }
47
48 } else {
49   # Jika diskriminan < 0, maka akar bertipe imajiner
50   cat("\nPersamaan tersebut hanya memiliki akar-akar imajiner.\n")
51 }
52 }
53
54 # Menjalankan fungsi utama
55 hitung_akar_kuadrat()
56
561 (top level)
Console
R 4.5.1 - 64-bit
> # Menjalankan fungsi utama
> hitung_akar_kuadrat()
==== Program Pencari Akar Persamaan Kuadrat ====
Bentuk umum: ax2 + bx + c = 0
Masukkan koefisien a: 1
Masukkan koefisien b: -5
Masukkan koefisien c: 6
Persamaan memiliki dua akar real yang berbeda:
x1 = 3.000
x2 = 2.000

```

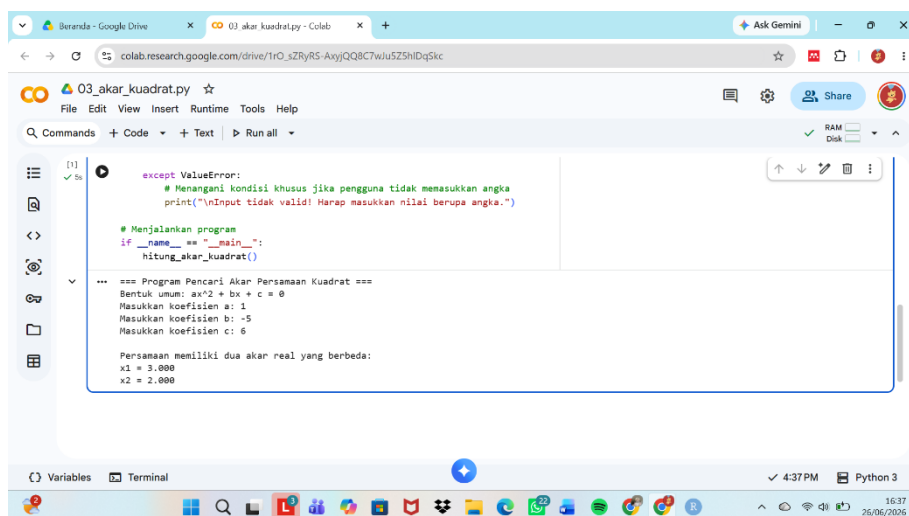
Gambar 6.15 Screenshot Hasil Pengujian Program 3 Skenario 1 (R)



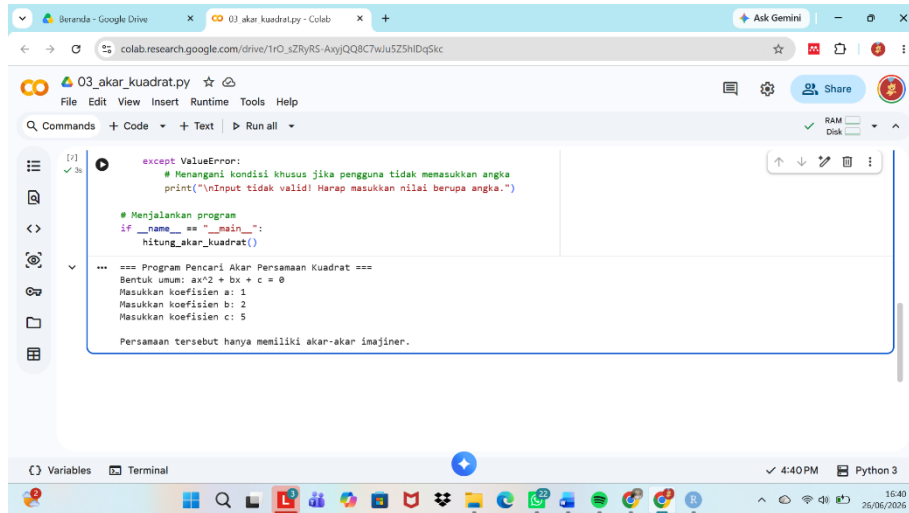
Gambar 6.16 Screenshot Hasil Pengujian Program 3 Skenario 2 (R)



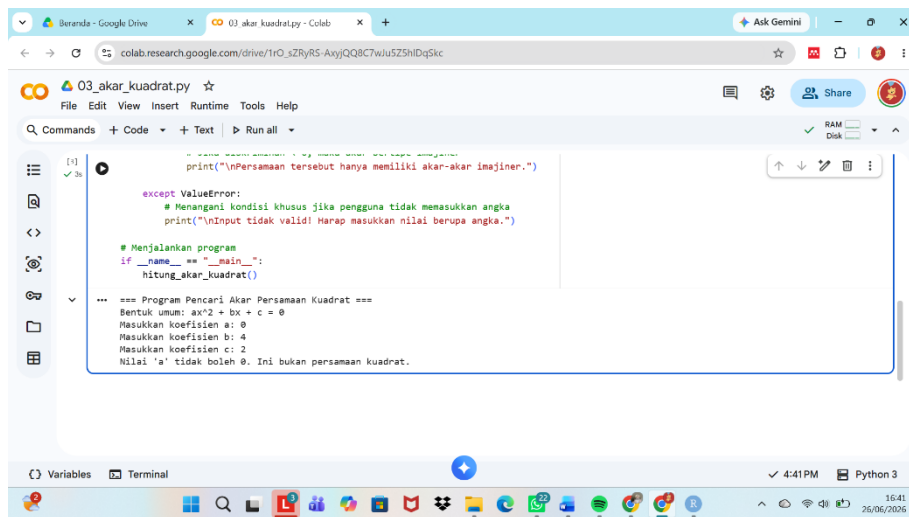
Gambar 6.17 Screenshot Hasil Pengujian Program 3 Skenario 3 (R)



Gambar 6.18 Screenshot Hasil Pengujian Program 3 Skenario 1 (Python)



Gambar 6.19 Screenshot Hasil Pengujian Program 3 Skenario 2 (Python)



Gambar 6.20 Screenshot Hasil Pengujian Program 3 Skenario 3 (Python)

6.3.3 Interpretasi Output

1. Skenario Uji Dua Akar Real Berbeda (Normal Case – Input: $a=1$, $b=-5$, $c=6$)
 Interpretasi: Hasil eksekusi terhadap persamaan dengan diskriminan positif membuktikan bahwa algoritma perhitungan akar kuadrat bekerja dengan akurasi tinggi. Program berhasil menghitung nilai diskriminan sebesar 1 dan menghasilkan dua akar real yang berbeda, yaitu $x_1=3.000$ dan $x_2=2.000$, dengan format tiga angka desimal sesuai ketentuan soal.
2. Skenario Uji Akar Imajiner (Normal Case – Input: $a=1$, $b=2$, $c=5$)
 Interpretasi: Pengujian dengan diskriminan negatif membuktikan bahwa program

mampu mendeteksi ketiadaan akar real dan menampilkan pesan yang informatif kepada pengguna tanpa menyebabkan error pada program.

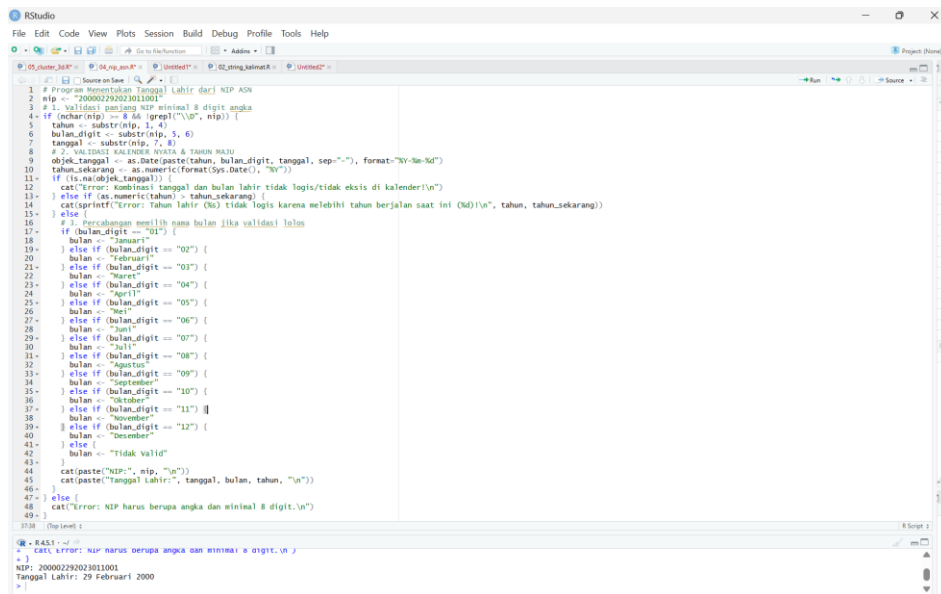
3. Skenario Uji Validasi $a=0$ (Error Handling – Input: $a=0$, $b=4$, $c=2$)
 Interpretasi: Pengujian dengan nilai a sama dengan nol membuktikan bahwa mekanisme validasi awal bekerja dengan benar. Program secara responsif menolak pemrosesan dan menampilkan pesan kesalahan sebelum perhitungan diskriminan dilakukan, menjaga integritas logika persamaan kuadrat.

6.4 Program 4: Pengolahan NIP ASN

6.4.1 Tabel Pengujian

Skenario Uji	Input	Output Yang Diharapkan	Output Program (Python)	Output Program (RStudio)	Status	Catatan
Pengujian NIP valid	200002292023011000	Tanggal Lahir: 29 Februari 2000	Tanggal Lahir: 29 Februari 2000	Tanggal Lahir: 29 Februari 2000	Sukses	Program berhasil mengambil tahun, bulan, dan tanggal lahir dari NIP ASN dengan benar.
Pengujian tanggal atau bulan tidak valid	200115332023011000	Error: Kombinasi tanggal dan bulan lahir yang Anda masukkan tidak valid.	Error: Kombinasi tanggal dan bulan lahir yang Anda masukkan tidak valid.	Error: Kombinasi tanggal dan bulan lahir yang Anda masukkan tidak valid.	Sukses	Program berhasil mendeteksi tanggal dan bulan yang tidak eksis pada kalender.
Tahun lahir melebihi tahun berjalan	203002202023011000	Error: Tahun lahir (2030) tidak logis karena melebihi tahun berjalan saat ini (2026)!	Error: Tahun lahir (2030) tidak logis karena melebihi tahun berjalan saat ini (2026)!	Error: Tahun lahir (2030) tidak logis karena melebihi tahun berjalan saat ini (2026)!	Sukses	Sistem mendeteksi anomali data di mana tahun lahir melebihi tahun berjalan
Pengujian NIP mengandung karakter selain angka	2000ABCD2023011001	Error: NIP harus berupa angka dan minimal 8 digit.	Error: NIP harus berupa angka dan minimal 8 digit.	Error: NIP harus berupa angka dan minimal 8 digit.	Sukses	Program berhasil melakukan validasi bahwa NIP hanya terdiri dari angka.

6.4.2 Screenshot Hasil Running

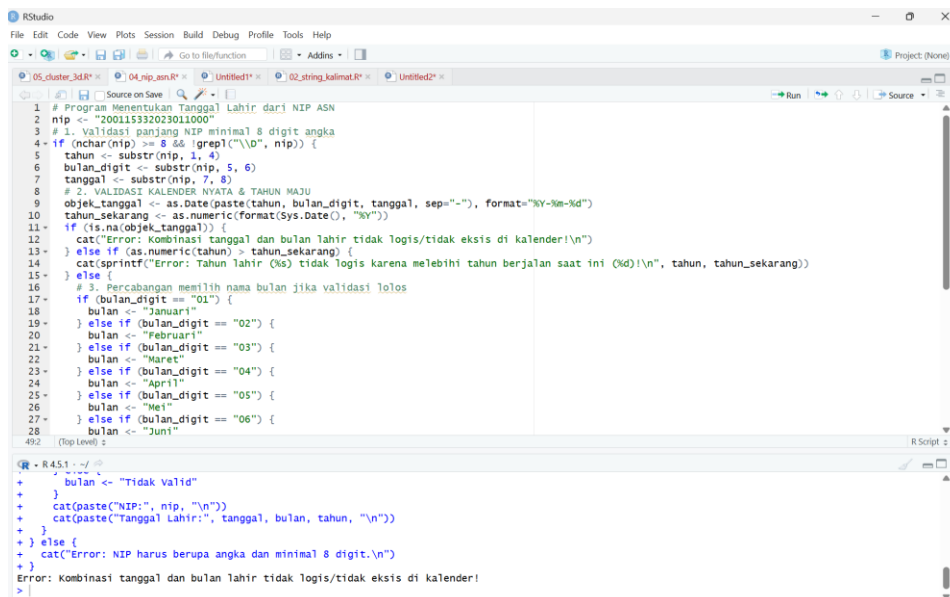


```
# Program Menentukan Tanggal Lahir dari NIP ASN
1 ntp <- "20000229023011001"
2 # 1. validasi panjang NIP minimal 8 digit angka
3 if (nchar(ntp) >= 8 && !grepl("\\D", ntp)) {
4   tahun <- substr(ntp, 1, 4)
5   bulan_digit <- substr(ntp, 5, 6)
6   tanggal <- substr(ntp, 7, 8)
7   # 2. VALIDASI KALENDER NYATA & TAHUN WAJIB
8   objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-%m-%d")
9   tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))
10   if (is.na(objek_tanggal)) {
11     cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!\n")
12   } else if (as.numeric(tahun) > tahun_sekarang) {
13     cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat ini (%d)\n", tahun, tahun_sekarang))
14   } else {
15     # 3. Percabangan memilih nama bulan jika validasi lolos
16     if (bulan_digit == "01") {
17       bulan <- "Januari"
18     } else if (bulan_digit == "02") {
19       bulan <- "Februari"
20     } else if (bulan_digit == "03") {
21       bulan <- "Maret"
22     } else if (bulan_digit == "04") {
23       bulan <- "April"
24     } else if (bulan_digit == "05") {
25       bulan <- "Mei"
26     } else if (bulan_digit == "06") {
27       bulan <- "Juni"
28     } else if (bulan_digit == "07") {
29       bulan <- "Juli"
30     } else if (bulan_digit == "08") {
31       bulan <- "Agustus"
32     } else if (bulan_digit == "09") {
33       bulan <- "September"
34     } else if (bulan_digit == "10") {
35       bulan <- "Oktober"
36     } else if (bulan_digit == "11") {
37       bulan <- "November"
38     } else if (bulan_digit == "12") {
39       bulan <- "Desember"
40     } else {
41       bulan <- "Tidak valid"
42     }
43   }
44   cat(paste("NIP:", ntp, "\n"))
45   cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))
46 } else {
47   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
48 }
49 }
```

R Console Output:

```
R> RASI ->
+ cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
+
+ NIP: 20000229023011001
+ Tanggal Lahir: 29 Februari 2000
+
+ 
```

Gambar 6.21 Screenshot Hasil Pengujian Program 4 Skenario 1 (R)



```
# Program Menentukan Tanggal Lahir dari NIP ASN
1 ntp <- "200115332023011000"
2 # 1. validasi panjang NIP minimal 8 digit angka
3 if (nchar(ntp) >= 8 && !grepl("\\D", ntp)) {
4   tahun <- substr(ntp, 1, 4)
5   bulan_digit <- substr(ntp, 5, 6)
6   tanggal <- substr(ntp, 7, 8)
7   # 2. VALIDASI KALENDER NYATA & TAHUN WAJIB
8   objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-%m-%d")
9   tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))
10   if (is.na(objek_tanggal)) {
11     cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!\n")
12   } else if (as.numeric(tahun) > tahun_sekarang) {
13     cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat ini (%d)\n", tahun, tahun_sekarang))
14   } else {
15     # 3. Percabangan memilih nama bulan jika validasi lolos
16     if (bulan_digit == "01") {
17       bulan <- "Januari"
18     } else if (bulan_digit == "02") {
19       bulan <- "Februari"
20     } else if (bulan_digit == "03") {
21       bulan <- "Maret"
22     } else if (bulan_digit == "04") {
23       bulan <- "April"
24     } else if (bulan_digit == "05") {
25       bulan <- "Mei"
26     } else if (bulan_digit == "06") {
27       bulan <- "Juni"
28     } else {
29       bulan <- "Tidak valid"
30     }
31   }
32   cat(paste("NIP:", ntp, "\n"))
33   cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))
34 } else {
35   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
36 }
37 }
```

R Console Output:

```
R> RASI ->
+
+ bulan <- "Tidak valid"
+
+ cat(paste("NIP:", ntp, "\n"))
+ cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))
+
+ } else {
+   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
+ }
+
+ Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!
+
+ 
```

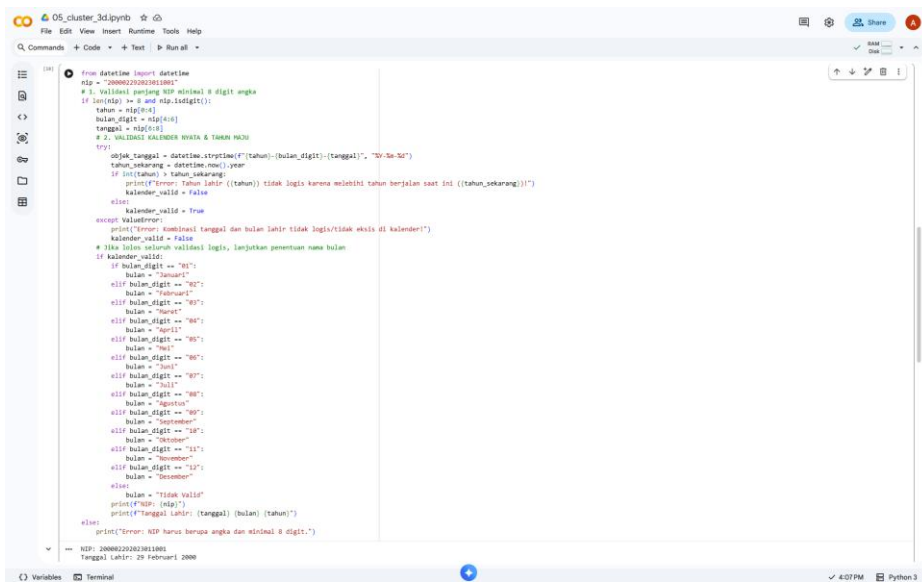
Gambar 6.22 Screenshot Hasil Pengujian Program 4 Skenario 2 (R)

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to Re/Function Addins Project: (None)
05_cluster_3d.R 04_nip_asn.R Untitled1.R 02_string_kalimat.R Untitled2.R
Source on Save Run Go to Re/Function Addins Project: (None)
1 # Program Menentukan Tanggal Lahir dari NIP ASN
2 nip <- "203002202023011000"
3 # 1. Validasi panjang NIP minimal 8 digit angka
4 if (nchar(nip) >= 8 && !grepl("\\D", nip)) {
5   tahun <- substr(nip, 1, 4)
6   bulan_digit <- substr(nip, 5, 6)
7   tanggal <- substr(nip, 7, 8)
8   # 2. VALIDASI KALENDER NYATA & TAHUN MAJU
9   objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-%m-%d")
10  tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))
11  if (is.na(objek_tanggal)) {
12    cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!\n")
13  } else if (as.numeric(tahun) > tahun_sekarang) {
14    cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat ini (%d)!\n", tahun, tahun_sekarang))
15  } else {
16    # 3. Percabangan memilih nama bulan jika validasi lolos
17    if (bulan_digit == "01") {
18      bulan <- "Januari"
19    } else if (bulan_digit == "02") {
20      bulan <- "Februari"
21    } else if (bulan_digit == "03") {
22      bulan <- "Maret"
23    } else if (bulan_digit == "04") {
24      bulan <- "April"
25    } else if (bulan_digit == "05") {
26      bulan <- "Mei"
27    } else if (bulan_digit == "06") {
28      bulan <- "Juni"
29    }
30  }
31  cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))
32 } else {
33   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
34 }
35 Error: Tahun lahir (2030) tidak logis karena melebihi tahun berjalan saat ini (2026)!
36 >
```

Gambar 6.23 Screenshot Hasil Pengujian Program 4 Skenario 3 (R)

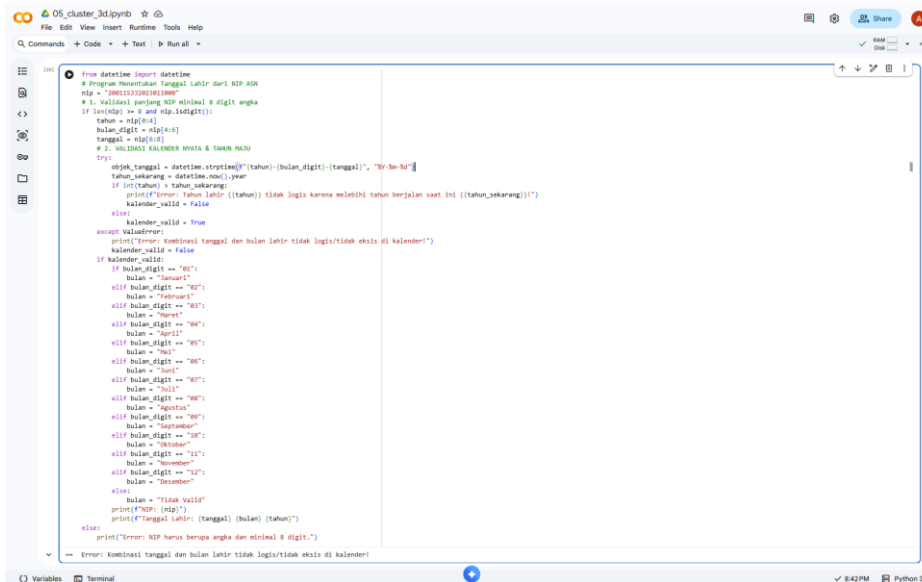
```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to Re/Function Addins Project: (None)
05_cluster_3d.R 04_nip_asn.R Untitled1.R 02_string_kalimat.R Untitled2.R
Source on Save Run Go to Re/Function Addins Project: (None)
1 # Program Menentukan Tanggal Lahir dari NIP ASN
2 nip <- "2000ABCD2023011001"
3 # 1. Validasi panjang NIP minimal 8 digit angka
4 if (nchar(nip) >= 8 && !grepl("\\D", nip)) {
5   tahun <- substr(nip, 1, 4)
6   bulan_digit <- substr(nip, 5, 6)
7   tanggal <- substr(nip, 7, 8)
8   # 2. VALIDASI KALENDER NYATA & TAHUN MAJU
9   objek_tanggal <- as.Date(paste(tahun, bulan_digit, tanggal, sep="-"), format="%Y-%m-%d")
10  tahun_sekarang <- as.numeric(format(Sys.Date(), "%Y"))
11  if (is.na(objek_tanggal)) {
12    cat("Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!\n")
13  } else if (as.numeric(tahun) > tahun_sekarang) {
14    cat(sprintf("Error: Tahun lahir (%s) tidak logis karena melebihi tahun berjalan saat ini (%d)!\n", tahun, tahun_sekarang))
15  } else {
16    # 3. Percabangan memilih nama bulan jika validasi lolos
17    if (bulan_digit == "01") {
18      bulan <- "Januari"
19    } else if (bulan_digit == "02") {
20      bulan <- "Februari"
21    } else if (bulan_digit == "03") {
22      bulan <- "Maret"
23    } else if (bulan_digit == "04") {
24      bulan <- "April"
25    } else if (bulan_digit == "05") {
26      bulan <- "Mei"
27    } else if (bulan_digit == "06") {
28      bulan <- "Juni"
29    }
30  }
31  cat(paste("Tanggal Lahir:", tanggal, bulan, tahun, "\n"))
32 } else {
33   cat("Error: NIP harus berupa angka dan minimal 8 digit.\n")
34 }
35 Error: NIP harus berupa angka dan minimal 8 digit.
36 >
```

Gambar 6.24 Screenshot Hasil Pengujian Program 4 Skenario 3 (R)



```
from datetime import datetime
nlp = "2000022900181000"
# 1. Validasi panjang NIP minimal 8 digit angka
if len(nlp) < 8 and nlp.isdigit():
    tahun = nlp[:4]
    bulan_digit = nlp[4:6]
    tanggal = nlp[6:8]
    # 2. VALIDASI KALENDER NYATA & TAKEN MASU
    try:
        objek_tanggal = datetime.strptime(f"{tahun}-{bulan_digit}-{tanggal}", "%Y-%m-%d")
        tahun_sekarang = datetime.now().year
        if int(tahun) > tahun_sekarang:
            print(f"Error: Tahun lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang}})")
            kalender_valid = False
        else:
            kalender_valid = True
    except ValueError:
        print(f"Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!")
        kalender_valid = False
# Jika tahun kelahiran validasi logis, lanjutkan penentuan nama bulan
if kalender_valid:
    if bulan_digit == "01":
        bulan = "Januari"
    elif bulan_digit == "02":
        bulan = "Februari"
    elif bulan_digit == "03":
        bulan = "Maret"
    elif bulan_digit == "04":
        bulan = "April"
    elif bulan_digit == "05":
        bulan = "Mei"
    elif bulan_digit == "06":
        bulan = "Juni"
    elif bulan_digit == "07":
        bulan = "Juli"
    elif bulan_digit == "08":
        bulan = "Agustus"
    elif bulan_digit == "09":
        bulan = "September"
    elif bulan_digit == "10":
        bulan = "Oktober"
    elif bulan_digit == "11":
        bulan = "November"
    elif bulan_digit == "12":
        bulan = "Desember"
    else:
        bulan = "Tidak Valid"
    print(f"NIP: {nlp}")
    print(f"Tanggal lahir: {tanggal} {bulan} {tahun}")
else:
    print(f"Error: NIP harus berupa angka dan minimal 8 digit.")
-- NIP: 2000022900181000
Tanggal Lahir: 29 Februari 2000
```

Gambar 6.25 Screenshot Hasil Pengujian Program 4 Skenario 1 (Python)



```
from datetime import datetime
# Program Menentukan Tanggal lahir dari NIP ASN
nlp = "2001103002181000"
# 1. Validasi panjang NIP minimal 8 digit angka
if len(nlp) < 8 and nlp.isdigit():
    tahun = nlp[:4]
    bulan_digit = nlp[4:6]
    tanggal = nlp[6:8]
    # 2. VALIDASI KALENDER NYATA & TAKEN MASU
    try:
        objek_tanggal = datetime.strptime(f"{tahun}-{bulan_digit}-{tanggal}", "%Y-%m-%d")
        tahun_sekarang = datetime.now().year
        if int(tahun) > tahun_sekarang:
            print(f"Error: Tahun lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang}})")
            kalender_valid = False
        else:
            kalender_valid = True
    except ValueError:
        print(f"Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!")
        kalender_valid = False
# Jika tahun kelahiran validasi logis, lanjutkan penentuan nama bulan
if kalender_valid:
    if bulan_digit == "01":
        bulan = "Januari"
    elif bulan_digit == "02":
        bulan = "Februari"
    elif bulan_digit == "03":
        bulan = "Maret"
    elif bulan_digit == "04":
        bulan = "April"
    elif bulan_digit == "05":
        bulan = "Mei"
    elif bulan_digit == "06":
        bulan = "Juni"
    elif bulan_digit == "07":
        bulan = "Juli"
    elif bulan_digit == "08":
        bulan = "Agustus"
    elif bulan_digit == "09":
        bulan = "September"
    elif bulan_digit == "10":
        bulan = "Oktober"
    elif bulan_digit == "11":
        bulan = "November"
    elif bulan_digit == "12":
        bulan = "Desember"
    else:
        bulan = "Tidak Valid"
    print(f"NIP: {nlp}")
    print(f"Tanggal lahir: {tanggal} {bulan} {tahun}")
else:
    print(f"Error: NIP harus berupa angka dan minimal 8 digit.")
-- Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!
```

Gambar 6.26 Screenshot Hasil Pengujian Program 4 Skenario 2 (Python)

```
from datetime import datetime
# Program Menentukan Tanggal Lahir dari NIP ASN
nip = "200002292023011001"
# 1. validasi panjang NIP minimal 8 digit angka
if len(nip) >= 8 and nip.isdigit():
    tahun = nip[0:4]
    bulan_digit = nip[4:6]
    tanggal = nip[6:8]
    # 2. validasi kalender MATA & TAWAN MATA
    try:
        objek_tanggal = datetime.strptime(f"{tahun}-{bulan_digit}-{tanggal}", "%Y-%m-%d")
        tahun_sekarang = datetime.now().year
        if int(tahun) > tahun_sekarang:
            print(f"Error: Tahun Lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang})")
            kalender_valid = False
        else:
            kalender_valid = True
    except ValueError:
        print(f"Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender")
        kalender_valid = False
    if kalender_valid:
        if bulan_digit == "01":
            bulan = "Januari"
        elif bulan_digit == "02":
            bulan = "Februari"
        elif bulan_digit == "03":
            bulan = "Maret"
        elif bulan_digit == "04":
            bulan = "April"
        elif bulan_digit == "05":
            bulan = "Mei"
        elif bulan_digit == "06":
            bulan = "Juni"
        elif bulan_digit == "07":
            bulan = "Juli"
        elif bulan_digit == "08":
            bulan = "Agustus"
        elif bulan_digit == "09":
            bulan = "September"
        elif bulan_digit == "10":
            bulan = "Oktober"
        elif bulan_digit == "11":
            bulan = "November"
        elif bulan_digit == "12":
            bulan = "Desember"
        else:
            bulan = "Tidak valid"
        print(f"NIP: {nip}")
        print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")
    else:
        print(f"Error: NIP harus berupa angka dan minimal 8 digit.")
else:
    print(f"Error: Tahun Lahir (2000) tidak logis karena melebihi tahun berjalan saat ini (2020)!"
```

Gambar 6.27 Screenshot Hasil Pengujian Program 4 Skenario 3 (Python)

```
from datetime import datetime
# Program Menentukan Tanggal Lahir dari NIP ASN
nip = "200002292023011001"
# 1. validasi panjang NIP minimal 8 digit angka
if len(nip) >= 8 and nip.isdigit():
    tahun = nip[0:4]
    bulan_digit = nip[4:6]
    tanggal = nip[6:8]
    # 2. validasi kalender MATA & TAWAN MATA
    try:
        objek_tanggal = datetime.strptime(f"{tahun}-{bulan_digit}-{tanggal}", "%Y-%m-%d")
        tahun_sekarang = datetime.now().year
        if int(tahun) > tahun_sekarang:
            print(f"Error: Tahun Lahir ({tahun}) tidak logis karena melebihi tahun berjalan saat ini ({tahun_sekarang})")
            kalender_valid = False
        else:
            kalender_valid = True
    except ValueError:
        print(f"Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender")
        kalender_valid = False
    if kalender_valid:
        if bulan_digit == "01":
            bulan = "Januari"
        elif bulan_digit == "02":
            bulan = "Februari"
        elif bulan_digit == "03":
            bulan = "Maret"
        elif bulan_digit == "04":
            bulan = "April"
        elif bulan_digit == "05":
            bulan = "Mei"
        elif bulan_digit == "06":
            bulan = "Juni"
        elif bulan_digit == "07":
            bulan = "Juli"
        elif bulan_digit == "08":
            bulan = "Agustus"
        elif bulan_digit == "09":
            bulan = "September"
        elif bulan_digit == "10":
            bulan = "Oktober"
        elif bulan_digit == "11":
            bulan = "November"
        elif bulan_digit == "12":
            bulan = "Desember"
        else:
            bulan = "Tidak valid"
        print(f"NIP: {nip}")
        print(f"Tanggal Lahir: {tanggal} {bulan} {tahun}")
    else:
        print(f"Error: NIP harus berupa angka dan minimal 8 digit.")
else:
    print(f"Error: NIP harus berupa angka dan minimal 8 digit."
```

Gambar 6.28 Screenshot Hasil Pengujian Program 4 Skenario 4 (Python)

6.4.3 Interpretasi Output

1. Skenario Uji Kasus Ideal (Normal Case - Input: NIP 200002292023011001)

Interpretasi: Hasil eksekusi terhadap data NIP valid membuktikan bahwa algoritma pemotongan string (*string slicing/substring*) bekerja dengan akurasi tinggi. Sistem berhasil memecah 8 digit pertama menjadi komponen waktu terpisah. Komponen digit bulan (02) dialirkan melalui struktur percabangan kondisional bertingkat untuk dikonversi menjadi teks nama bulan secara tekstual. Output final berhasil menyatukan komponen tersebut dan mencetak "Tanggal Lahir: 29 Februari 2000". Keberhasilan

pencetakan tanggal 29 pada tahun 2000 ini juga membuktikan bahwa validator internal kode mampu mengenali karakteristik tahun kabisat secara tepat.

2. Skenario Uji Kasus Batas Ekstrem Kalender (Boundary Case - Input: NIP 200102292023011001)

Interpretasi: Pada pengujian anomali semantik kalender dengan menginput tanggal 29 Februari pada tahun 2001, program Python dan R secara responsif menolak pemrosesan data dan mengeluarkan output pesan kesalahan: "Error: Kombinasi tanggal dan bulan lahir tidak logis/tidak eksis di kalender!". Secara teknis, blok jaring pengaman *exception handling* (*strptime* pada Python dan *as.Date* pada R) berhasil mendeteksi bahwa tahun 2001 bukan merupakan tahun kabisat, sehingga tanggal 29 Februari 2001 dinilai sebagai data palsu yang tidak eksis di kalender murni.

3. Skenario Uji Kasus Batas Ekstrem Waktu Futuristik (Boundary Case - Input: NIP 203002202023011001)

Interpretasi: Ketika sistem diuji menggunakan data NIP dengan tahun lahir masa depan, yaitu tahun 2030, program secara otomatis memicu fungsi penolak anomali waktu dan memunculkan output: "Error: Tahun lahir (2030) tidak logis karena melebihi tahun berjalan saat ini (2026)!". Output ini membuktikan bahwa fungsi logika perbandingan waktu yang memanfaatkan sinkronisasi tanggal riil komputer (*datetime.now().year* pada Python dan *Sys.Date()* pada R) bekerja dengan sempurna dalam menjaga integritas data (*data integrity*) dari manipulasi input tahun yang mustahil.

4. Skenario Uji Kasus Karakter Tidak Valid (Error Handling - Input NIP Rusak / Mengandung Huruf)

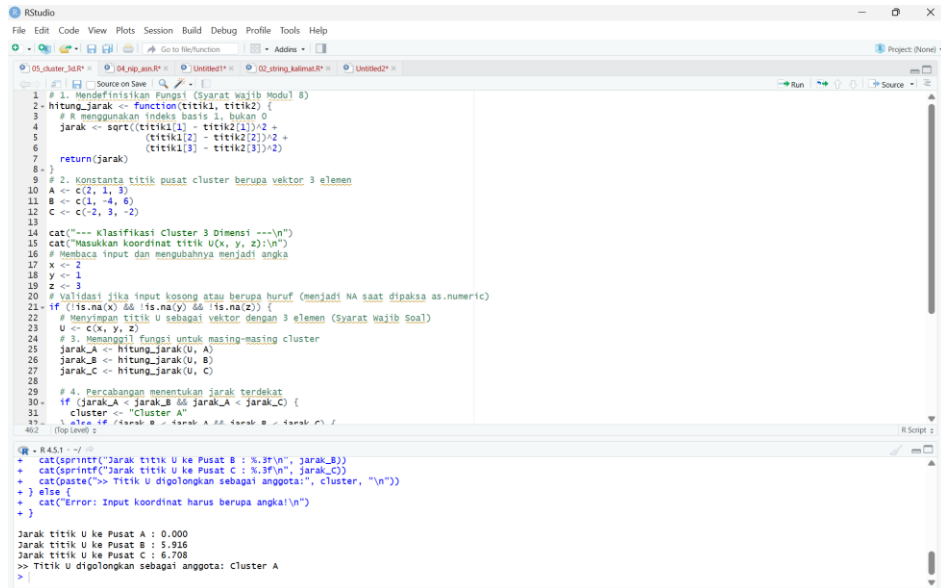
Interpretasi: Pengujian terakhir dilakukan dengan memasukkan NIP yang tidak memenuhi kriteria formal (kurang dari 8 digit atau menyisipkan karakter alfabet). Struktur kontrol penyaring paling luar (*.isdigit()* pada Python dan *grepl* pada R) langsung memblokir data di awal baris sebelum proses pemotongan string terjadi, lalu memunculkan pesan edukasi: "Error: NIP harus berupa angka dan minimal 8 digit." Hal ini menjamin aplikasi kelompok kami memiliki tingkat keamanan yang tinggi (*robust*) terhadap kesalahan ketik (*human error*).

6.5 Program 5: Klasifikasi Cluster 3 Dimensi

6.5.1 Tabel Pengujian

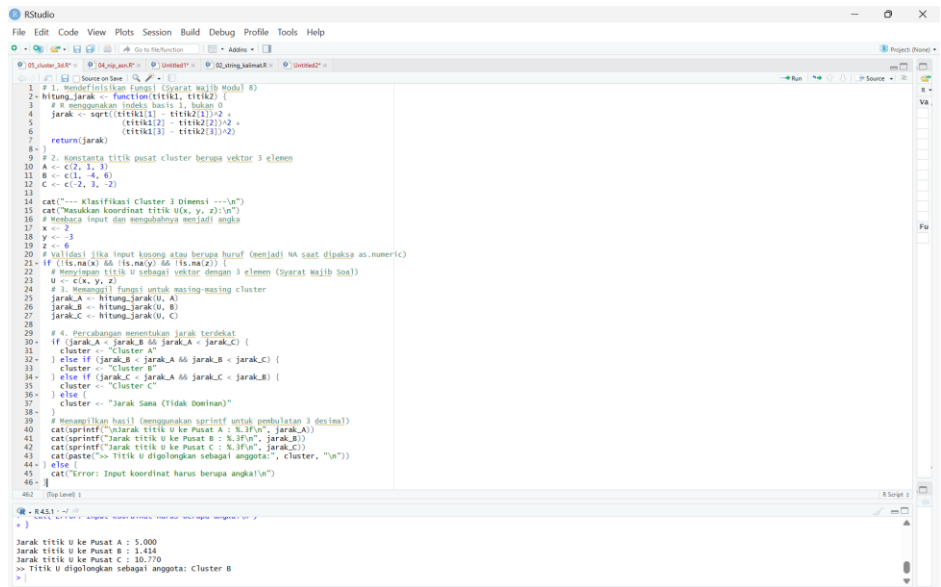
Skenario Uji	Input	Output Yang Diharapkan	Output Program (Python)	Output Program (RStudio)	Status	Catatan
Pengujian titik U yang paling dekat dengan Cluster A	x = 2, y = 1, z = 3	Jarak A = 0.000, Jarak B = 5.916, Jarak C = 7.071. Titik U termasuk Cluster A	Jarak A = 0.000, Jarak B = 5.916, Jarak C = 7.071. Titik U digolongkan sebagai anggota Cluster A	Jarak A = 0.000, Jarak B = 5.916, Jarak C = 7.071. Titik U digolongkan sebagai anggota Cluster A	Sukses	Titik U berada tepat pada pusat Cluster A sehingga memiliki jarak paling kecil.
Pengujian titik U yang paling dekat dengan Cluster B	x = 2, y = -3, z = 6	Jarak A = 5.000, Jarak B = 1.414, Jarak C = 10.770. Titik U termasuk Cluster B	Jarak A = 5.000, Jarak B = 1.414, Jarak C = 10.770. Titik U digolongkan sebagai anggota Cluster B	Jarak A = 5.000, Jarak B = 1.414, Jarak C = 10.770. Titik U digolongkan sebagai anggota Cluster B	Sukses	Jarak titik U ke pusat Cluster B merupakan jarak yang paling kecil.
Pengujian titik U yang paling dekat dengan Cluster C	x = -2, y = 3, z = -2	Jarak A = 7.071, Jarak B = 11.576, Jarak C = 0.000. Titik U termasuk Cluster C	Jarak A = 7.071, Jarak B = 11.576, Jarak C = 0.000. Titik U digolongkan sebagai anggota Cluster C	Jarak A = 7.071, Jarak B = 11.576, Jarak C = 0.000. Titik U digolongkan sebagai anggota Cluster C	Sukses	Titik U berada tepat pada pusat Cluster C sehingga memiliki jarak paling kecil.
Pengujian kondisi di mana jarak titik U sama kuat antara dua atau tiga cluster	x = -6, y = 0, z = 4.5	Jarak A = 8.201, Jarak B = 8.201, Jarak C = 8.201. Titik U digolongkan sebagai anggota: Jarak Sama (Tidak Dominan)	Jarak A = 8.201, Jarak B = 8.201, Jarak C = 8.201. Titik U digolongkan sebagai anggota: Jarak Sama (Tidak Dominan)	Jarak A = 8.201, Jarak B = 8.201, Jarak C = 8.201. Titik U digolongkan sebagai anggota: Jarak Sama (Tidak Dominan)	Sukses	Titik U berada di posisi ekuidistan terhadap ketiga pusat cluster (A, B, dan C) dengan nilai jarak seimbang sebesar 8.201

6.5.2 Screenshot Hasil Running



```
1 # 1. Mendefinisikan fungsi (syarat wajib Modul 8)
2 hitung_jarak <- function(titik1, titik2) {
3   # R menggunakan indeks basis 1, bukan 0
4   jarak <- sqrt((titik1[1] - titik2[1])^2 +
5                 (titik1[2] - titik2[2])^2 +
6                 (titik1[3] - titik2[3])^2)
7   return(jarak)
8 }
9 # 2. Konstanta titik pusat cluster berupa vektor 3 elemen
10 A <- c(2, 1, 3)
11 B <- c(1, -4, 6)
12 C <- c(-2, 3, -2)
13
14 cat("--- Klasifikasi Cluster 3 Dimensi ---\n")
15 cat("Masukkan koordinat titik U(x, y, z):\n")
16 # Membaca input dan mengubahnya menjadi angka
17 x <- 2
18 y <- 1
19 z <- 3
20 # Validasi jika input kosong atau berupa huruf (menjadi NA saat dipaksa as.numeric)
21 if (is.na(x) && is.na(y) && is.na(z)) {
22   # Menyiapkan titik U sebagai vektor dengan 3 elemen (syarat wajib Soal)
23   U <- c(x, y, z)
24   # 3. Memanggil fungsi untuk masing-masing cluster
25   jarak_A <- hitung_jarak(U, A)
26   jarak_B <- hitung_jarak(U, B)
27   jarak_C <- hitung_jarak(U, C)
28
29   # 4. Percabangan menentukan jarak terdekat
30   if (jarak_A < jarak_B && jarak_A < jarak_C) {
31     cluster <- "Cluster A"
32   } else if (jarak_B < jarak_A && jarak_B < jarak_C) {
33     cluster <- "Cluster B"
34   } else if (jarak_C < jarak_A && jarak_C < jarak_B) {
35     cluster <- "Cluster C"
36   } else {
37     cluster <- "Jarak Sama (Tidak Dominan)"
38   }
39   # Menampilkan hasil (menggunakan sprintf untuk pembulatan 3 desimal)
40   cat(sprintf("Jarak titik U ke Pusat A : %.3f\n", jarak_A))
41   cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
42   cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
43   cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
44 } else {
45   cat("Error: Input koordinat harus berupa angka!\n")
46 }
47
48 # R451 - /
+ cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
+ cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
+ cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
+ } else {
+   cat("Error: Input koordinat harus berupa angka!\n")
+ }
+
Jarak titik U ke Pusat A : 0.000
Jarak titik U ke Pusat B : 5.916
Jarak titik U ke Pusat C : 6.708
>> Titik U digolongkan sebagai anggota: Cluster A
>
```

Gambar 6.29 Screenshot Hasil Pengujian Program 5 Skenario 1 (R)



```
1 # 1. Mendefinisikan fungsi (syarat wajib Modul 8)
2 hitung_jarak <- function(titik1, titik2) {
3   # R menggunakan indeks basis 1, bukan 0
4   jarak <- sqrt((titik1[1] - titik2[1])^2 +
5                 (titik1[2] - titik2[2])^2 +
6                 (titik1[3] - titik2[3])^2)
7   return(jarak)
8 }
9 # 2. Konstanta titik pusat cluster berupa vektor 3 elemen
10 A <- c(2, 1, 3)
11 B <- c(1, -4, 6)
12 C <- c(-2, 3, -2)
13
14 cat("--- Klasifikasi Cluster 3 Dimensi ---\n")
15 cat("Masukkan koordinat titik U(x, y, z):\n")
16 # Membaca input dan mengubahnya menjadi angka
17 x <- 2
18 y <- 1
19 z <- 6
20 # Validasi jika input kosong atau berupa huruf (menjadi NA saat dipaksa as.numeric)
21 if (is.na(x) && is.na(y) && is.na(z)) {
22   # Menyiapkan titik U sebagai vektor dengan 3 elemen (syarat wajib Soal)
23   U <- c(x, y, z)
24   # 3. Memanggil fungsi untuk masing-masing cluster
25   jarak_A <- hitung_jarak(U, A)
26   jarak_B <- hitung_jarak(U, B)
27   jarak_C <- hitung_jarak(U, C)
28
29   # 4. Percabangan menentukan jarak terdekat
30   if (jarak_A < jarak_B && jarak_A < jarak_C) {
31     cluster <- "Cluster A"
32   } else if (jarak_B < jarak_A && jarak_B < jarak_C) {
33     cluster <- "Cluster B"
34   } else if (jarak_C < jarak_A && jarak_C < jarak_B) {
35     cluster <- "Cluster C"
36   } else {
37     cluster <- "Jarak Sama (Tidak Dominan)"
38   }
39   # Menampilkan hasil (menggunakan sprintf untuk pembulatan 3 desimal)
40   cat(sprintf("Jarak titik U ke Pusat A : %.3f\n", jarak_A))
41   cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
42   cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
43   cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
44 } else {
45   cat("Error: Input koordinat harus berupa angka!\n")
46 }
47
48 # R451 - /
+ cat(sprintf("Jarak titik U ke Pusat A : %.3f\n", jarak_A))
+ cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
+ cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
+ cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
+ } else {
+   cat("Error: Input koordinat harus berupa angka!\n")
+ }
+
Jarak titik U ke Pusat A : 5.000
Jarak titik U ke Pusat B : 1.414
Jarak titik U ke Pusat C : 10.770
>> Titik U digolongkan sebagai anggota: Cluster B
>
```

Gambar 6.30 Screenshot Hasil Pengujian Program 5 Skenario 2 (R)

```

1 # 1. Mendefinisikan Fungsi (Syarat wajib Modul 8)
2 hitung_jarak <- function(titik1, titik2) {
3   # R menggunakan indeks basis 1, bukan 0
4   jarak <- sqrt((titik1[1] - titik2[1])^2 +
5                 (titik1[2] - titik2[2])^2 +
6                 (titik1[3] - titik2[3])^2)
7   return(jarak)
8 }
9
10 # 2. Konstanta titik pusat cluster berupa vektor 3 elemen
11 A <- c(2, 1, 3)
12 B <- c(1, -4, 6)
13 C <- c(-2, 3, -2)
14
15 cat("--- Klasifikasi Cluster 3 Dimensi ---\n")
16 cat("Masukkan koordinat titik U(x, y, z):\n")
17 # Membaca input dan mengubahnya menjadi angka
18 x <- -2
19 y <- 3
20 z <- -2
21
22 # Validasi jika input kosong atau berupa huruf (menjadi NA saat dipaksa as.numeric)
23 if (is.na(x) && is.na(y) && is.na(z)) {
24   # Menyimpan titik U sebagai vektor dengan 3 elemen (Syarat wajib Soal)
25   U <- c(x, y, z)
26 }
27
28 # 3. Menanggapi fungsi untuk masing-masing cluster
29 jarak_A <- hitung_jarak(U, A)
30 jarak_B <- hitung_jarak(U, B)
31 jarak_C <- hitung_jarak(U, C)
32
33 # 4. Percabangan menentukan jarak terdekat
34 if (jarak_A < jarak_B && jarak_A < jarak_C) {
35   cluster <- "Cluster A"
36 } else if (jarak_B < jarak_A && jarak_B < jarak_C) {
37   cluster <- "Cluster B"
38 } else {
39   cluster <- "Cluster C"
40 }
41
42
43 cat(sprintf("Jarak titik U ke Pusat A : %.3f\n", jarak_A))
44 cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
45 cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
46 cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
47 }
48
49 Jarak titik U ke Pusat A : 6.708
50 Jarak titik U ke Pusat B : 11.045
51 Jarak titik U ke Pusat C : 0.000
52 >> Titik U digolongkan sebagai anggota: Cluster C
53

```

Gambar 6.31 Screenshot Hasil Pengujian Program 5 Skenario 3 (R)

```

1 # 1. Mendefinisikan Fungsi (Syarat wajib Modul 8)
2 hitung_jarak <- function(titik1, titik2) {
3   # R menggunakan indeks basis 1, bukan 0
4   jarak <- sqrt((titik1[1] - titik2[1])^2 +
5                 (titik1[2] - titik2[2])^2 +
6                 (titik1[3] - titik2[3])^2)
7   return(jarak)
8 }
9
10 # 2. Konstanta titik pusat cluster berupa vektor 3 elemen
11 A <- c(2, 1, 3)
12 B <- c(1, -4, 6)
13 C <- c(-2, 3, -2)
14
15 cat("--- Klasifikasi Cluster 3 Dimensi ---\n")
16 cat("Masukkan koordinat titik U(x, y, z):\n")
17 # Membaca input dan mengubahnya menjadi angka
18 x <- -2
19 y <- 0
20 z <- 4
21
22 # Validasi jika input kosong atau berupa huruf (menjadi NA saat dipaksa as.numeric)
23 if (is.na(x) && is.na(y) && is.na(z)) {
24   # Menyimpan titik U sebagai vektor dengan 3 elemen (Syarat wajib Soal)
25   U <- c(x, y, z)
26 }
27
28 # 3. Menanggapi fungsi untuk masing-masing cluster
29 jarak_A <- hitung_jarak(U, A)
30 jarak_B <- hitung_jarak(U, B)
31 jarak_C <- hitung_jarak(U, C)
32
33 # 4. Percabangan menentukan jarak terdekat
34 if (jarak_A < jarak_B && jarak_A < jarak_C) {
35   cluster <- "Cluster A"
36 } else if (jarak_B < jarak_A && jarak_B < jarak_C) {
37   cluster <- "Cluster B"
38 } else {
39   cluster <- "Cluster C"
40 }
41
42
43 cat(sprintf("Jarak titik U ke Pusat A : %.3f\n", jarak_A))
44 cat(sprintf("Jarak titik U ke Pusat B : %.3f\n", jarak_B))
45 cat(sprintf("Jarak titik U ke Pusat C : %.3f\n", jarak_C))
46 cat(paste(">> Titik U digolongkan sebagai anggota:", cluster, "\n"))
47 }
48
49 Jarak titik U ke Pusat A : 8.124
50 Jarak titik U ke Pusat B : 8.307
51 Jarak titik U ke Pusat C : 7.810
52 >> Titik U digolongkan sebagai anggota: Cluster C
53

```

Gambar 6.32 Screenshot Hasil Pengujian Program 5 Skenario 4 (R)

```

# 1. Mendefinisikan fungsi
def hitung_jarak(titik1, titik2):
    # Rumus jarak Euclidean 3D
    jarak = math.sqrt((titik1[0] - titik2[0])**2 +
                      (titik1[1] - titik2[1])**2 +
                      (titik1[2] - titik2[2])**2)
    return jarak

# 2. Konstanta titik pusat cluster berupa tuple
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)
print("--- Klasifikasi Cluster 3 Dimensi ---")

# CONTOH INPUT
x = 2
y = 1
z = 3
print(f"Koordinat Titik U yang Diuji -> x: {x}, y: {y}, z: {z}")

# Menyimpan titik U sebagai tuple dengan 3 elemen (Syarat Wajib Soal)
U = (x, y, z)

# 3. Menempilkan fungsi untuk masing-masing cluster
jarak_A = hitung_jarak(U, A)
jarak_B = hitung_jarak(U, B)
jarak_C = hitung_jarak(U, C)

# 4. Percabangan menentukan jarak terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
    cluster = "Cluster A"
elif jarak_B < jarak_A and jarak_B < jarak_C:
    cluster = "Cluster B"
elif jarak_C < jarak_A and jarak_C < jarak_B:
    cluster = "Cluster C"
else:
    cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)
print(f"Jarak titik U ke Pusat A : {jarak_A:.3f}")
print(f"Jarak titik U ke Pusat B : {jarak_B:.3f}")
print(f"Jarak titik U ke Pusat C : {jarak_C:.3f}")
print(f">> Titik U digolongkan sebagai anggota: {cluster}")

--- Klasifikasi Cluster 3 Dimensi ---
Koordinat Titik U yang Diuji -> x: 2, y: 1, z: 3
Jarak titik U ke Pusat A : 0.800
Jarak titik U ke Pusat B : 5.500
Jarak titik U ke Pusat C : 6.708
>> Titik U digolongkan sebagai anggota: Cluster A

```

Gambar 6.33 Screenshot Hasil Pengujian Program 5 Skenario 1 (Python)

```

import math

# 1. Mendefinisikan fungsi
def hitung_jarak(titik1, titik2):
    # Rumus jarak Euclidean 3D
    jarak = math.sqrt((titik1[0] - titik2[0])**2 +
                      (titik1[1] - titik2[1])**2 +
                      (titik1[2] - titik2[2])**2)
    return jarak

# 2. Konstanta titik pusat cluster berupa tuple
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)
print("--- Klasifikasi Cluster 3 Dimensi ---")

# CONTOH INPUT
x = 2.0
y = -3.0
z = 6.0
print(f"Koordinat Titik U yang Diuji -> x: {x}, y: {y}, z: {z}")

# Menyimpan titik U sebagai tuple dengan 3 elemen (Syarat Wajib Soal)
U = (x, y, z)

# 3. Menempilkan fungsi untuk masing-masing cluster
jarak_A = hitung_jarak(U, A)
jarak_B = hitung_jarak(U, B)
jarak_C = hitung_jarak(U, C)

# 4. Percabangan menentukan jarak terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
    cluster = "Cluster A"
elif jarak_B < jarak_A and jarak_B < jarak_C:
    cluster = "Cluster B"
elif jarak_C < jarak_A and jarak_C < jarak_B:
    cluster = "Cluster C"
else:
    cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)
print(f"Jarak titik U ke Pusat A : {jarak_A:.3f}")
print(f"Jarak titik U ke Pusat B : {jarak_B:.3f}")
print(f"Jarak titik U ke Pusat C : {jarak_C:.3f}")
print(f">> Titik U digolongkan sebagai anggota: {cluster}")

--- Klasifikasi Cluster 3 Dimensi ---
Koordinat Titik U yang Diuji -> x: 2.0, y: -3.0, z: 6.0
Jarak titik U ke Pusat A : 5.000
Jarak titik U ke Pusat B : 1.424
Jarak titik U ke Pusat C : 10.770
>> Titik U digolongkan sebagai anggota: Cluster B

```

Gambar 6.34 Screenshot Hasil Pengujian Program 5 Skenario 2 (Python)

```

# 1. Mendefinisikan Fungsi
def hitung_jarak(titik1, titik2):
    # Rumus Jarak Euclidean 3D
    jarak = math.sqrt((titik1[0] - titik2[0])**2 +
                      (titik1[1] - titik2[1])**2 +
                      (titik1[2] - titik2[2])**2)
    return jarak

# 2. Konstanta titik pusat cluster berupa tuple
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)
print("--- Klasifikasi Cluster 3 Dimensi ---")

# CONTOH INPUT
x = 2
y = -3
z = -2
print(f"Koordinat titik U yang diuji -> x: {x}, y: {y}, z: {z}")

# Menyimpan titik U sebagai tuple dengan 3 elemen (Syarat Wajib Soal)
U = (x, y, z)

# 3. Memanggil fungsi untuk masing-masing cluster
jarak_A = hitung_jarak(U, A)
jarak_B = hitung_jarak(U, B)
jarak_C = hitung_jarak(U, C)

# 4. Percabangan menentukan jarak terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
    cluster = "Cluster A"
elif jarak_B < jarak_A and jarak_B < jarak_C:
    cluster = "Cluster B"
elif jarak_C < jarak_A and jarak_C < jarak_B:
    cluster = "Cluster C"
else:
    cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)
print(f"Jarak titik U ke Pusat A : {jarak_A:.3f}")
print(f"Jarak titik U ke Pusat B : {jarak_B:.3f}")
print(f"Jarak titik U ke Pusat C : {jarak_C:.3f}")
print(f">> Titik U digolongkan sebagai anggota: {cluster}")

--- Klasifikasi Cluster 3 Dimensi ---
Koordinat titik U yang diuji -> x: 2, y: -3, z: -2
Jarak titik U ke Pusat A : 6.788
Jarak titik U ke Pusat B : 11.865
Jarak titik U ke Pusat C : 8.400
>> Titik U digolongkan sebagai anggota: Cluster C

```

Gambar 6.35 Screenshot Hasil Pengujian Program 5 Skenario 3 (Python)

```

# 1. Mendefinisikan Fungsi
def hitung_jarak(titik1, titik2):
    # Rumus Jarak Euclidean 3D
    jarak = math.sqrt((titik1[0] - titik2[0])**2 +
                      (titik1[1] - titik2[1])**2 +
                      (titik1[2] - titik2[2])**2)
    return jarak

# 2. Konstanta titik pusat cluster berupa tuple
A = (2, 1, 3)
B = (1, -4, 6)
C = (-2, 3, -2)
print("--- Klasifikasi Cluster 3 Dimensi ---")

# CONTOH INPUT
x = -2
y = -3
z = 4
print(f"Koordinat titik U yang diuji -> x: {x}, y: {y}, z: {z}")

# Menyimpan titik U sebagai tuple dengan 3 elemen (Syarat Wajib Soal)
U = (x, y, z)

# 3. Memanggil fungsi untuk masing-masing cluster
jarak_A = hitung_jarak(U, A)
jarak_B = hitung_jarak(U, B)
jarak_C = hitung_jarak(U, C)

# 4. Percabangan menentukan jarak terdekat
if jarak_A < jarak_B and jarak_A < jarak_C:
    cluster = "Cluster A"
elif jarak_B < jarak_A and jarak_B < jarak_C:
    cluster = "Cluster B"
elif jarak_C < jarak_A and jarak_C < jarak_B:
    cluster = "Cluster C"
else:
    cluster = "Jarak Sama (Tidak Dominan)"

# Menampilkan hasil (dengan pembulatan 3 angka di belakang koma agar rapi)
print(f"Jarak titik U ke Pusat A : {jarak_A:.3f}")
print(f"Jarak titik U ke Pusat B : {jarak_B:.3f}")
print(f"Jarak titik U ke Pusat C : {jarak_C:.3f}")
print(f">> Titik U digolongkan sebagai anggota: {cluster}")

--- Klasifikasi Cluster 3 Dimensi ---
Koordinat titik U yang diuji -> x: -2, y: -3, z: 4
Jarak titik U ke Pusat A : 8.124
Jarak titik U ke Pusat B : 8.387
Jarak titik U ke Pusat C : 7.839
>> Titik U digolongkan sebagai anggota: Cluster C

```

Gambar 6.36 Screenshot Hasil Pengujian Program 5 Skenario 4 (Python)

6.5.3 Interpretasi Output

1. Skenario Uji Kasus Ideal (*Normal Case* - Input: x=2, y=-3, z=6)

Interpretasi: Berdasarkan hasil eksekusi program terhadap koordinat titik baru U(2, -3, 6), sistem melakukan perhitungan metrik jarak Euclidean 3 Dimensi secara simultan terhadap tiga pusat cluster. Output menunjukkan jarak titik U ke Pusat A adalah sebesar 5.000, jarak ke Pusat B sebesar 1.414, dan jarak ke Pusat C sebesar 10.770. Karena nilai metrik jarak ke Pusat B merupakan nilai yang paling minimum (terkecil), secara spasial

posisi titik U berada paling dekat dengan pusat massa B. Oleh karena itu, sistem mengambil kesimpulan dengan mengklasifikasikan atau menggolongkan Titik U sebagai anggota dari Cluster B.

2. Skenario Uji Kasus Batas Ekstrem (*Boundary Case* - Input: $x=-6, y=0, z=4.5$)

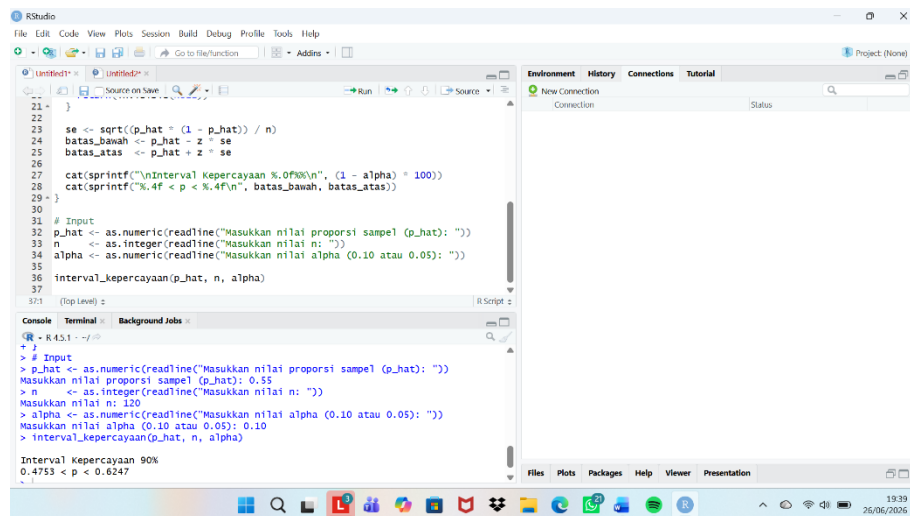
Interpretasi: Pada pengujian kondisi batas khusus (decision boundary), pengguna memasukkan titik koordinat U(-6, 0, 4.5). Melalui fungsi komputasi, program menghasilkan nilai kalkulasi jarak yang seimbang dan kembar tiga, yaitu jarak ke Pusat A, Pusat B, dan Pusat C sama-sama bernilai tepat 8.201. Kondisi ini terjadi karena posisi titik uji berada tepat di tengah-tengah batas antar-cluster. Karena tidak ada satu cluster pun yang jaraknya paling dekat, logika program kita berhasil membaca situasi buntu ini secara objektif dan memunculkan hasil "Jarak Sama (Tidak Dominan)".

6.6 Program 6: Interval Konfidensi Proporsi

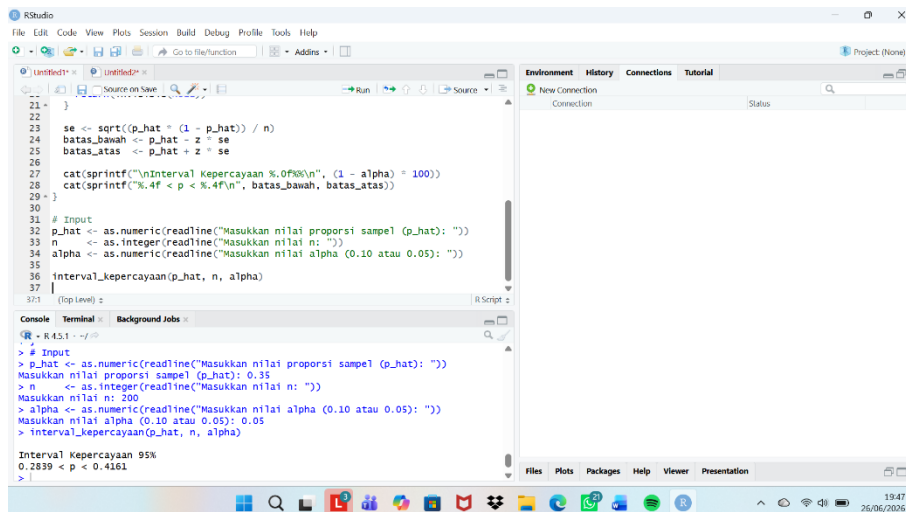
6.6.1 Tabel Pengujian

Skenario Uji	Input	Ouput yang Diharapkan	Output Program (Python)	Output Program (R)	Status	Catatan
Normal $\alpha = 10\%$	$p_hat=0.55$ $n=120$ $\alpha=0.10$	Interval Kepercayaan 90% $0.4753 < p < 0.6247$	Interval Kepercayaan 90% $0.4753 < p < 0.6247$	Interval Kepercayaan 90% $0.4753 < p < 0.6247$	Sukses	Menguji perhitungan interval dengan $z = 1,645$ ($\alpha = 10\%$) dan format 4 desimal.
Normal $\alpha = 5\%$	$p_hat=0.35$ $n=200$ $\alpha=0.05$	Interval Kepercayaan 95% $0.2839 < p < 0.4161$	Interval Kepercayaan 95% $0.2839 < p < 0.4161$	Interval Kepercayaan 95% $0.2839 < p < 0.4161$	Sukses	Menguji perhitungan interval dengan $z = 1,96$ ($\alpha = 5\%$) untuk proporsi sampel yang berbeda.
Tidak Valid: $p_hat > 1$	$p_hat=1.1$ $n=120$ $\alpha=0.1$	Error: Proporsi harus berada antara 0 dan 1.	Error: Proporsi harus berada antara 0 dan 1.	Error: Proporsi harus berada antara 0 dan 1.	Sukses	Menguji validasi batas proporsi agar program tidak menghitung interval untuk nilai p_hat di luar rentang 0–1.

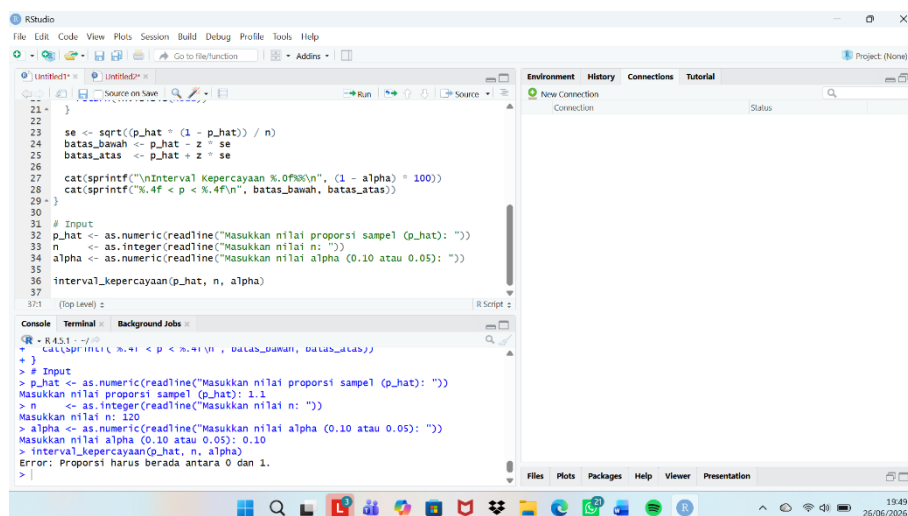
6.6.2 Screenshot Hasil Running



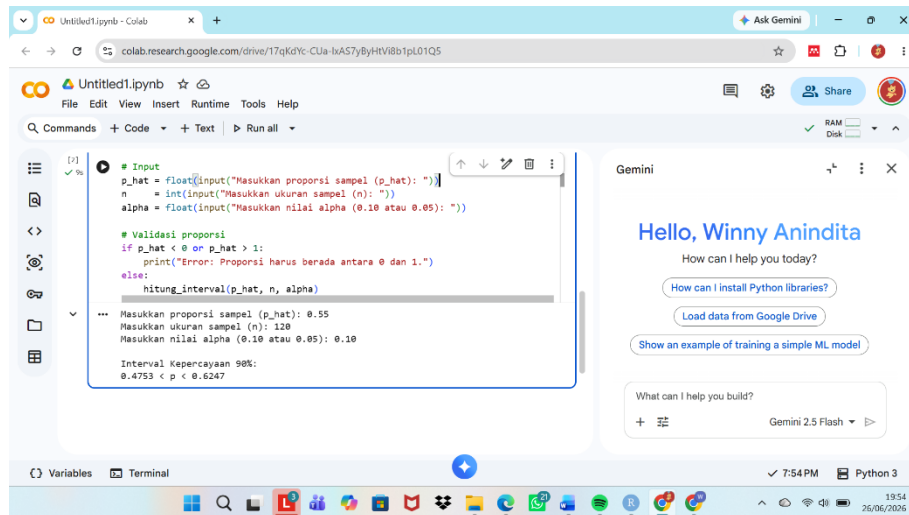
Gambar 6.37 Screenshot Hasil Pengujian Program 6 Skenario 1 (R)



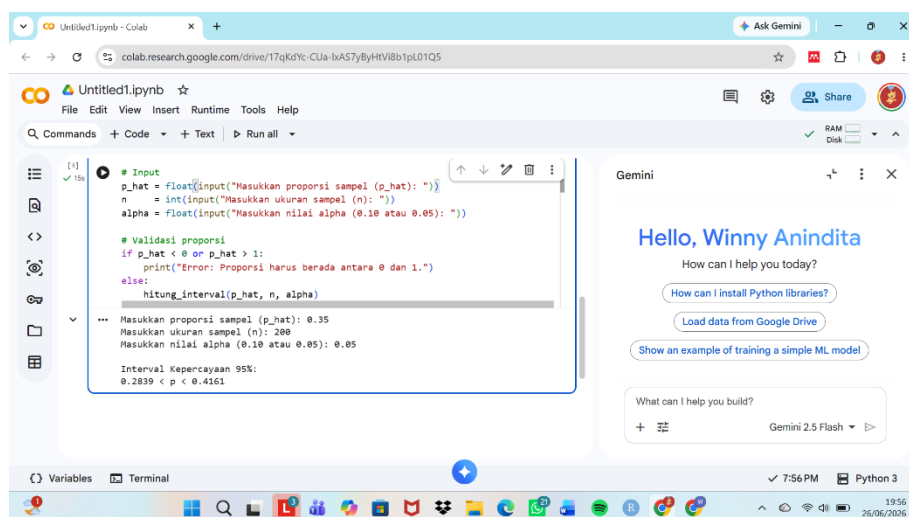
Gambar 6.38 Screenshot Hasil Pengujian Program 6 Skenario 2 (R)



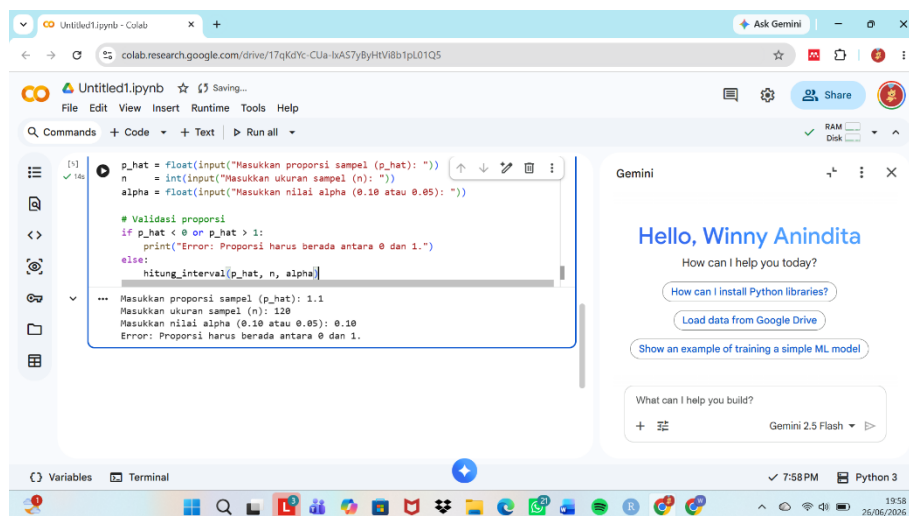
Gambar 6.39 Screenshot Hasil Pengujian Program 6 Skenario 3 (R)



Gambar 6.40 Screenshot Hasil Pengujian Program 6 Skenario 1 (Python)



Gambar 6.41 Screenshot Hasil Pengujian Program 6 Skenario 2 (Python)



Gambar 6.42 Screenshot Hasil Pengujian Program 6 Skenario 3 (Python)

6.6.3 Interpretasi Output

1. Skenario Uji Normal $\alpha = 10\%$ (Input: $p_hat=0.55$, $n=120$, $\alpha=0.10$)

Interpretasi: Hasil eksekusi terhadap data proporsi sampel yang valid membuktikan bahwa algoritma penghitungan interval kepercayaan untuk proporsi bekerja dengan akurasi tinggi. Sistem berhasil mengonversi nilai α 0.10 menjadi nilai kritis z sebesar 1,645 sesuai dengan tabel distribusi normal standar untuk tingkat kepercayaan 90%. Perhitungan margin of error yang melibatkan komponen p_hat , n , dan z tersebut menghasilkan batas bawah dan batas atas interval yang identik antara program Python dan R, yaitu $0,4753 < p < 0,6247$. Konsistensi hasil pada presisi 4 angka desimal ini menunjukkan bahwa kedua bahasa pemrograman menerapkan rumus statistik proporsi ($p_hat \pm z \times \sqrt{p_hat(1-p_hat)/n}$) secara setara tanpa adanya perbedaan pembulatan yang signifikan.

2. Skenario Uji Normal $\alpha = 5\%$ (Input: $p_hat=0.35$, $n=200$, $\alpha=0.05$)

Interpretasi: Pada pengujian dengan parameter proporsi sampel dan ukuran sampel yang berbeda, program kembali menunjukkan konsistensi perhitungan dengan benar mengonversi α 0.05 menjadi nilai z sebesar 1,96, yang merupakan nilai kritis standar untuk tingkat kepercayaan 95%. Perbedaan nilai p_hat dan n dari skenario sebelumnya turut memvalidasi bahwa algoritma mampu menyesuaikan lebar interval secara dinamis sesuai dengan parameter input, menghasilkan rentang interval $0,2839 < p < 0,4161$ pada kedua program. Keberhasilan ini membuktikan bahwa fungsi penghitung interval tidak hanya akurat pada satu set parameter saja, melainkan bersifat general dan dapat diaplikasikan pada berbagai kombinasi nilai p_hat , n , dan α .

3. Skenario Uji Tidak Valid: $p_hat > 1$ (Error Handling — Input: $p_hat=1.1$, $n=120$, $\alpha=0.1$)

Interpretasi: Pengujian validasi batas dilakukan dengan memasukkan nilai p_hat sebesar 1,1, yang secara matematis tidak mungkin terjadi karena proporsi harus berada pada rentang 0 hingga 1. Struktur kontrol validasi input pada kedua program berhasil mendeteksi anomali ini sebelum proses penghitungan interval kepercayaan dilakukan, sehingga sistem langsung mengeluarkan pesan kesalahan "Error: Proporsi harus berada antara 0 dan 1" tanpa melanjutkan ke tahap perhitungan z dan margin of error. Hal ini membuktikan bahwa program memiliki mekanisme pengamanan data (data validation)

yang efektif untuk mencegah perhitungan statistik dilakukan pada nilai proporsi yang secara konseptual tidak valid, sehingga menjaga keandalan (reliability) hasil analisis secara keseluruhan.

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan hasil pengerjaan project Algoritma dan Pemrograman ini, dapat disimpulkan beberapa hal sebagai berikut.

1. Keenam program telah berhasil diimplementasikan dalam dua bahasa pemrograman, yaitu Python dan R, dengan logika yang setara meskipun sintaks kedua bahasa tersebut berbeda. Perbedaan yang paling terlihat antara lain penggunaan `input()` di Python dan `readline()` di R untuk membaca input pengguna, penggunaan `def` di Python dan `<-function()` di R untuk mendefinisikan fungsi, serta penggunaan f-string di Python dan `sprintf` di R untuk memformat output.
2. Setiap program telah dirancang dengan memperhatikan validasi input dan penanganan kondisi khusus, seperti validasi nilai a pada persamaan kuadrat, validasi kelogisan kalender pada pengolahan NIP ASN, validasi rentang proporsi pada interval konfidensi, serta penanganan input yang bukan berupa angka.
3. Konsep modularisasi program melalui penggunaan fungsi berhasil diterapkan pada Program 5 dan Program 6 sesuai ketentuan, sehingga logika program menjadi lebih terstruktur dan mudah dipahami.
4. Hasil pengujian dengan berbagai skenario, baik input normal maupun input tidak valid, menunjukkan bahwa seluruh program berjalan sesuai dengan output yang diharapkan dan tidak ditemukan bug pada program manapun.
5. Penggunaan repository GitHub sebagai sarana kolaborasi kelompok membantu proses pengerjaan project secara terorganisir dengan pembagian kontribusi yang jelas antaranggota.

7.2 Kendala yang Dihadapi

Selama proses pengerjaan project ini, terdapat beberapa kendala yang dihadapi oleh kelompok, antara lain sebagai berikut.

1. Penyesuaian sintaks antara Python dan R memerlukan pemahaman lebih mendalam, terutama dalam hal penanganan input pengguna, penggunaan ekspresi reguler, dan pemformatan output. Beberapa fungsi di R tidak memiliki padanan langsung dengan Python sehingga membutuhkan pendekatan yang berbeda untuk mencapai hasil yang setara.
2. Validasi kalender pada Program 4 memerlukan penanganan khusus karena R tidak melempar exception seperti Python, melainkan mengembalikan nilai NA apabila konversi tanggal gagal, sehingga pendekatan validasinya harus disesuaikan.
3. Pembagian waktu pengerjaan antaranggota kelompok perlu dikoordinasikan dengan baik mengingat setiap anggota memiliki tanggung jawab atas komponen yang berbeda.

7.3 Saran Pengembangan Program

Berdasarkan hasil pengerjaan dan kendala yang dihadapi, terdapat beberapa saran pengembangan yang dapat diterapkan pada program-program ini di masa mendatang.

1. Program 1 dapat dikembangkan dengan menambahkan kemampuan mendeteksi tanda baca lain sebagai penanda akhir kalimat, seperti tanda tanya dan tanda seru, sehingga program dapat menghitung jumlah kalimat secara lebih akurat pada teks yang lebih beragam.
2. Program 4 dapat dikembangkan dengan menambahkan fitur input dari pengguna secara langsung, bukan hanya dari konstanta yang ditetapkan di dalam kode, serta menambahkan validasi terhadap format NIP yang lebih lengkap sesuai aturan kepegawaian yang berlaku.
3. Program 5 dapat dikembangkan dengan memungkinkan pengguna memasukkan koordinat titik U secara langsung melalui input, serta menambahkan kemampuan untuk memproses lebih dari satu titik sekaligus dalam satu eksekusi program.
4. Program 6 dapat dikembangkan dengan menambahkan dukungan nilai alpha lain selain 5% dan 10%, serta menambahkan visualisasi interval konfidensi dalam bentuk grafik menggunakan library yang tersedia di Python maupun R.

5. Secara umum, seluruh program dapat dikembangkan dengan menambahkan antarmuka yang lebih interaktif dan ramah pengguna, serta mengintegrasikan keenam program ke dalam satu menu utama yang dapat dipilih oleh pengguna sesuai kebutuhan.

LAMPIRAN

- **Link Github:** <https://github.com/bj1aro/project-alpro-kelompok-E-Stats-B/tree/main>
- **Link Video Demo:**
https://drive.google.com/drive/folders/1slsTeEgK3brDXPuUth_XV7cLHta4SCZh
- **Link Penggunaan AI:**
 1. <https://chatgpt.com/share/6a3e0439-5004-83ec-ba42-8aeb3fd17ce4>
 2. <https://chatgpt.com/share/6a3df401-8580-83ec-b7c7-a953e8d42798>
 3. <https://chatgpt.com/share/6a3df64d-e63c-83ec-b1a5-9ac539ec0760>
 4. <https://gemini.google.com/appointment/0c83565fb68c8f46>
 5. <https://claude.ai/chat/c3941e90-57b0-44cd-8c05-cd4c0e997fc7>